Physics-Inspired Optimization of the Quadratic Assignment Problem on GPU

Case Study: Graph Matching for Human Brain Connectomes
Master thesis by Moritz Klein
Date of submission: June 30, 2026

1. Review: Prof. Dr. Arjan Kuijper
2. Review: Dr. Matthias Heller
Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Erklärung zur Nutzung von KI-gestützten Hilfsmitteln

Generative Sprachmodelle (bereitgestellt über *fhgenie*) wurden unterstützend beim Verfassen (Formulierung und Wortwahl) eingesetzt. Jede Aussage der Sprachmodelle wurde vom Autor kritisch geprüft und überarbeitet. Alle endgültigen Entscheidungen bezüglich Inhalt und Formulierung wurden vom Autor getroffen. Der Autor trägt die alleinige Verantwortung für den Inhalt dieser Arbeit.

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Moritz Klein, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§38 Abs.2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.



Darmstadt, 30.06.2026

Moritz Klein

Abstract

The Quadratic Assignment Problem (QAP) is one of the most challenging combinatorial optimization problems. Quantum and quantum-inspired methods often rely on Quadratic Unconstrained Binary Optimization (QUBO) encodings, which introduce a severe 2^{n^2} search-space explosion and fragile penalty terms when applied to the QAP.

Rather than accepting these limitations, the key insight of this thesis is that physics-inspired dynamics can be reformulated to operate directly on integer permutations, preserving feasibility by construction and bypassing QUBO entirely.

To address this challenge, this thesis makes five contributions. First, we reformulate Simulated Annealing (SA), Simulated Quantum Annealing (SQA), Momentum Annealing (MA), Parallel Tempering (PT), and Population Annealing (PA) to operate natively in permutation space for the QAP. Second, we implement these methods from scratch in `ASSA.jl`, a custom GPU-accelerated solver framework in Julia. Third, we introduce efficient permutation-native accelerations, including a novel caching strategy for MA. Fourth, we propose PT-SQA, a hybrid architecture combining replica exchange with tunneling-like multi-slice dynamics. Fifth, we evaluate all solvers on QAPLIB and connectome graph matching. PA consistently outperforms Tabu Search across the Taillard benchmark families under a strict 150-second time limit. We establish a new best-known solution for `tai175e01` (59286 vs. 59732) and demonstrate scaling to brain connectome instances of $n = 1000$.

Contents

1 Introduction **1**

2 Problem **3**
2.1 Quadratic Assignment Problem . . . 3
2.2 Graph Matching . . . 6
2.3 Quadratic Unconstrained Binary Optimization . . . 7
2.4 Encoding the QAP as QUBO . . . 8
2.4.1 Constraint Penalties . . . 8
2.4.2 Alternative Encoding . . . 9
2.4.3 Why not use the QUBO for QAP? . . . 10

3 State of the Art **11**
3.1 Exact Methods . . . 11
3.2 Classical Heuristics . . . 11
3.3 Continuous Relaxation Methods . . . 12
3.4 Quantum and Quantum-Inspired Methods . . . 13
3.5 Summary and Motivation . . . 14

4 Heuristics and Methods **15**
4.1 Base Move: 2-Opt . . . 15
4.2 Simulated Annealing . . . 17
4.2.1 Cooling Schedules . . . 17
4.2.2 Application to the QAP . . . 18
4.2.3 Efficient Cost Difference Evaluation via Cache . . . 19
4.3 Simulated Quantum Annealing . . . 21
4.3.1 Application to the QAP . . . 22
4.3.2 Beyond Single Time Dimension . . . 24
4.4 Momentum Annealing . . . 25
4.4.1 Application to the QAP . . . 26
4.5 Parallel Tempering . . . 29
4.5.1 Temperature Ladder . . . 30
4.5.2 Cluster Moves . . . 30
4.5.3 Application of Cluster Moves to the QAP . . . 31
4.5.4 Combining Heuristics . . . 33
4.5.5 Parallel Tempering in Two Dimensions . . . 34
4.6 Population Annealing . . . 35

5 Implementation **37**
5.1 Simulated Annealing . . . 40
5.2 Simulated Annealing Cached . . . 41
5.3 Simulated Quantum Annealing . . . 42

5.3.1 Ineffectiveness of Caching for SQA . . . 43
5.4 Momentum Annealing . . . 43
5.5 Momentum Annealing Cached . . . 44
5.6 Parallel Tempering . . . 45
5.7 Population Annealing . . . 45
5.8 Summary . . . 46

6 Results and Discussion **47**
6.1 QAPLIB . . . 47
6.1.1 Simulated Annealing . . . 48
6.1.2 Simulated Quantum Annealing . . . 50
6.1.3 Momentum Annealing . . . 54
6.1.4 Population Annealing . . . 56
6.1.5 Parallel Tempering . . . 58
6.1.6 QAPLIB-wide Benchmark Summary . . . 62
6.1.7 Comparison to External Solvers . . . 67
6.2 Human Brain Connectomes . . . 69

7 Conclusion and Outlook **72**
7.1 Future . . . 74
7.1.1 Seeded Graph Matching . . . 74
7.1.2 SQA: QAP-aware Transverse Field . . . 75
7.1.3 SQA and MA Crossover . . . 76
7.1.4 `ASSA.jl` Improvements . . . 76

Epilogue **77**

A Appendix **79**
A.1 Reservoir sampling on GPU . . . 79
A.2 SA Always using Cache . . . 80
A.3 SQA on Torus: Dimension + 2 . . . 81
A.4 PT2D for MA . . . 81
A.5 Parameters for Adaptive Temperature Schedule in PT . . . 82

1 Introduction

Imagine you're designing a hospital with 20 departments and 20 rooms. Nurses walk between rooms hundreds of times a day. Where do you put the *intensive care unit* relative to the *operating theatre*, or the *pharmacy* relative to the *wards*? Frequently interacting units should be located in close proximity, thereby minimizing the total travel time. This is the Quadratic Assignment Problem (QAP).
The same structure, however, reaches far beyond physical layouts: the QAP also models aligning two subjects' connectomes (each with up to $n = 1000$ brain regions) to detect structural similarities associated with cognitive development or disease.

The QAP is one of the most challenging combinatorial optimization problems, and finding high-quality solutions efficiently has attracted decades of algorithmic effort. Classical heuristics such as Simulated Annealing (SA) and Tabu Search operate directly on permutations and can scale well, but remain sensitive to parameter tuning; the best-known solution for a single $n = 100$ instance required months of CPU time [1].

Quantum and quantum-inspired methods have recently emerged as an alternative, with hardware such as *D-Wave* offering annealing dynamics that can in principle escape local minima more efficiently. However, applying these methods to the QAP requires encoding it as a Quadratic Unconstrained Binary Optimization (QUBO), and this encoding introduces a fundamental difficulty that undermines the quantum advantage.

The QUBO encoding inflates an already intractable problem into an astronomically larger one. For $n = 14$ there are $14! \approx 8.2 \times 10^{10}$ valid assignments. That is about the number of microseconds (μs) per day. The QUBO encoding balloons this to $2^{14^2} = 2^{196}$ possible states, roughly 10^{35} times larger than the age of the universe in microseconds. Yet of all those QUBO states, only a negligible fraction represents feasible solutions.
The root cause is structural: permutations enforce a globally coupled feasibility constraint that binary models cannot capture natively. Swapping two facilities changes two assignment positions at once, preserving the one-hot structure implicitly. Binary encodings treat assignment variables as independent bits, so single-bit flips break feasibility and must be corrected through heavy penalty terms. This penalty-tuning problem is itself NP-hard, and hardware embedding overhead restricts practical problem sizes on current quantum annealers to roughly $n \approx 15$.

Instead of forcing the QAP into a binary framework, this thesis adapts spin-glass–inspired annealing dynamics to operate directly in the natural permutation space. The key insight that makes this possible is that every 2-opt swap (exchanging two assignments) produces a valid permutation by construction, with no penalty terms needed and no search-space inflation. This allows the full expressive power of thermal, tunneling, and population-based dynamics to be directed at the QAP objective rather than wasted on constraint satisfaction.

Three research questions guide this thesis:

1. *How can optimization mechanisms designed for spin-glasses be reformulated for integer permutations?*
2. *How does quantum-inspired tunneling compare to classical thermal fluctuations in permutation space?*
3. *How can permutation-based annealing be efficiently parallelized on GPUs while maintaining correctness?*

To answer these questions, we develop permutation-space variants of the following solvers:

- **SA**: classical thermal fluctuations with swap-based moves — at high temperatures moves are accepted freely, while at low temperatures the system settles into a near-frozen configuration;
- **Simulated Quantum Annealing (SQA)**: multi-replica coupling serving as permutation tunneling — parallel copies are coupled across an imaginary-time dimension, enabling them to pass through barriers that would trap a single replica;
- **Momentum Annealing (MA)**: bipartite remapping for massive parallelism in dense problems — the assignment is split into two coupled copies that are optimized alternately, gradually converging toward a shared solution;
- **Parallel Tempering (PT)**: replica exchange with permutation cluster moves — a replica trapped in a local minimum can escape by temporarily exchanging its configuration with a hotter, more exploratory replica;
- **Population Annealing (PA)**: evolutionary resampling with Boltzmann-weighted selection — low-energy replicas are duplicated proportionally, while high-energy ones are eliminated;
- **PT-SQA**: a novel hybrid using quantum tunneling for cluster moves within PT.

The central claim of this thesis is that physics-inspired annealing dynamics transfer more effectively to the QAP when reformulated in permutation space than through QUBO encodings. This structural choice (operating in $n!$ rather than 2^{n^2}) eliminates penalty tuning entirely and yields a family of GPU-accelerated solvers that are competitive and scalable to connectome instances at $n = 1000$.

Empirically, we evaluate all solvers on QAPLIB. Under a fixed sweep time limit, we find that PA consistently outperforms Tabu Search (among the strongest available CPU baseline) across the Taillard benchmark families. Crucially, PA and Parallel Tempering with cluster moves (PTC) compete with commercial hybrid solvers like D-Wave under the same strict time limits.
Both PA and PTC establish a new best-known solution for `tai175e01` (59286 vs. 59732). All solvers scale to brain connectome instances of $n = 1000$, with MA providing the fastest throughput and adaptive PA achieving the best final solution quality.

The thesis is structured as follows: **Chapter 2** introduces the QAP, its complexity, and the QUBO mapping. **Chapter 3** surveys the state of the art, covering exact methods, classical heuristics, continuous relaxations, and quantum-inspired approaches. **Chapter 4** develops annealing dynamics and their permutation-space reformulations. **Chapter 5** describes the GPU implementation within the `ASSA.jl` framework. **Chapter 6** presents the benchmark results, including connectome graph matching, and discusses the findings. Finally, **Chapter 7** concludes the thesis and outlines future directions.

2 Problem

This chapter formally introduces the QAP, shows its mathematical equivalence to the Graph Matching Problem (GMP), and explains why we avoid the standard QUBO formulation in favor of a permutation-native approach.

2.1 Quadratic Assignment Problem

Koopmans and Beckmann [2] first introduced the QAP in 1957 as an economic location problem. Since then, it has found a wide range of applications, including: designing electrical circuits to minimize connections between components; job scheduling; typewriter keyboard layout design; and facility layout problems such as hospital layout to minimize patient travel distance, university campus planning, and forest zoning management [3–8]. It also appears in telecommunications planning including IoT networks, network traffic balancing, and UAV routing [9].

Beyond these direct applications, classic combinatorial problems such as the Traveling Salesman Problem (TSP), graph partitioning, and maximum clique can also be formulated as instances of the QAP [10].

Before we write a single formula: what does the QAP actually ask?
You have n facilities and n locations. Each pair of facilities has a certain "flow" between them (*how often do they interact?*). Each pair of locations has a certain "distance". You want to place facilities into locations so that high-traffic pairs end up close together.
The cost of an assignment is: for every pair of facilities, multiply their flow by the distance between their assigned locations. Sum everything up. Minimize that.
Importantly, distances could also be asymmetric if we, e.g., measure the time taken to travel between them and one is on a hill and the other is in a valley.

Now, given a flow matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$ and a distance matrix $\boldsymbol{B} \in \mathbb{R}^{n \times n}$, the goal is to find an assignment $\boldsymbol{\pi} : \{1, \ldots, n\} \to \{1, \ldots, n\}$ mapping facilities to locations that minimizes the total cost [10]:[1]

$$z(\boldsymbol{\pi}) = \sum_{i}^{n} \sum_{j}^{n} \boldsymbol{A}_{ij} \boldsymbol{B}_{\boldsymbol{\pi}(i)\boldsymbol{\pi}(j)}. \tag{2.1}$$

Alternatively, using a binary permutation matrix $\boldsymbol{X} \in \{0, 1\}^{n \times n}$ where rows and columns sum to 1, the cost function is equivalent to:

$$\operatorname{tr}\left(\boldsymbol{A}\boldsymbol{X}\boldsymbol{B}\boldsymbol{X}^{\top}\right). \tag{2.2}$$

[1] The Generalized Quadratic Assignment Problem (GQAP) also includes the matrix $\boldsymbol{C}$ which defines the cost of assigning facility i to location $\boldsymbol{\pi}(i)$ and thereby extending Equation 2.1 by $\sum \boldsymbol{C}_{i\boldsymbol{\pi}(i)}$.

With the following constraints:

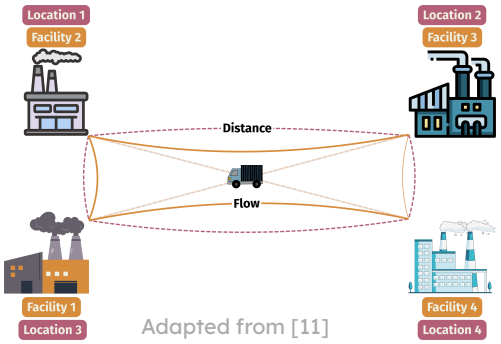
$$\text{one 1 per column:} \quad \sum_i X_{ij} = 1, \qquad j \in 1, 2, \ldots, n \quad \text{and} \tag{2.3.a}$$

$$\text{one 1 per row:} \quad \sum_j X_{ij} = 1, \qquad i \in 1, 2, \ldots, n. \tag{2.3.b}$$

Examples

A **small example** is depicted on the right, where facility 1 is assigned to location 3, facility 2 to location 1, facility 3 to location 2, and facility 4 to location 4, yielding $\pi = [3, 1, 2, 4]$. We can write this as a matrix X where $X_{ij} = 1$ if facility i is assigned to location j.

$$X = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$



Adapted from [11]

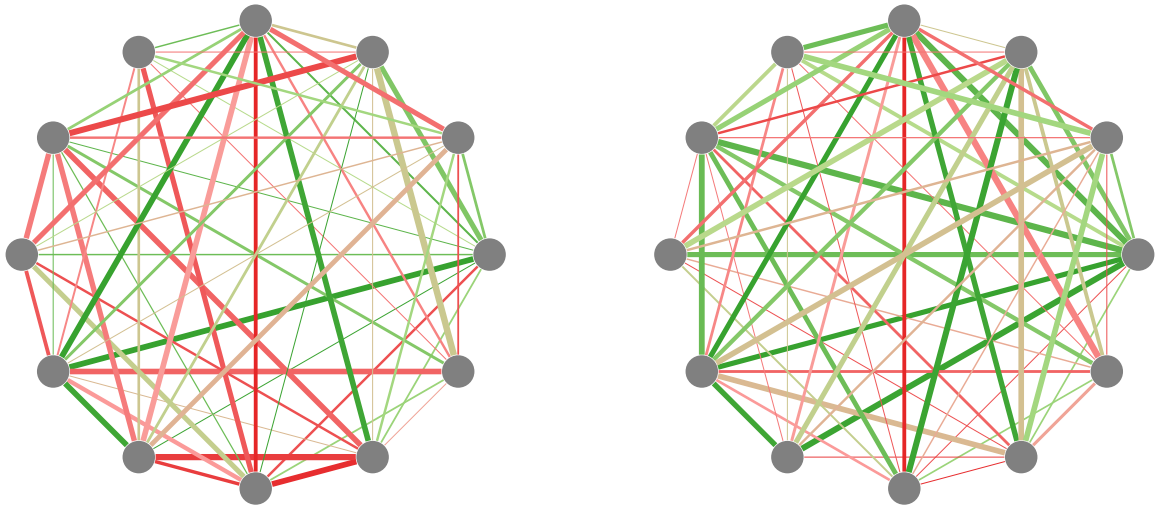
The **TSP** asks: *"Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?"* To express this within the QAP framework, one uses the distance matrix B as the intercity cost matrix and encodes the sequence in the flow matrix as $A_{i,i+1} = 1$ and $A_{n,1} = 1$, otherwise 0 [10].

Assigning offices to employees: A_{ij} is the frequency with which employee i meets employee j and B_{uv} is the travel time from office u to office v. Assigning office v to employee j and office u to employee i is equal to $X_{iu} = 1$ and $X_{jv} = 1$. The goal is to find X s.t. the time lost is minimized [12].

The objective fundamentally aims to avoid assigning high flows to long distances. Figure 2.1 shows this by contrasting a poor assignment with the optimal one. On the left, a random assignment connects large distances with high flow facilities. That is expensive. On the right, the optimal configuration: the busiest departments are now neighbors.

The QAP is widely regarded as one of the most challenging combinatorial optimization problems [13]. Typically, instances with more than 30 variables cannot be exhaustively solved to optimality within a reasonable time [14]. Sahni and Gonzalez [15] proved the QAP to be NP-hard. Today, the QAP is often considered one of the hardest of the hard [16].

The factorial explosion is visualized in Figure 2.2, and the shaded gap foreshadows the central inefficiency of the QUBO encoding analyzed in Section 2.4.



(a) Random Assignment **(b)** Optimal Assignment

Figure 2.1: Visualizing QAP objective quality: a random assignment (left) places high-flow facility pairs (red) on long-distance location pairs (thick lines), incurring high cost. The optimal assignment (right) aligns high flows with short distances (thin lines), minimizing total cost. Color encodes flow magnitude (red = high, green = low); line thickness encodes distance.

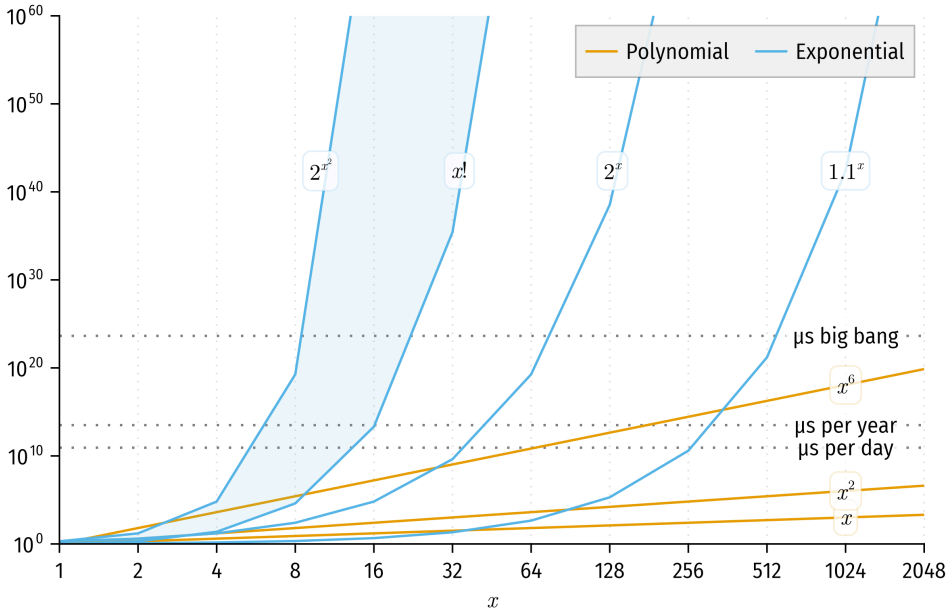


Figure 2.2: Search-space growth as a function of problem size n: polynomial scaling (e.g., n^2) remains tractable, while the factorial growth of QAP permutations ($n!$) and the binary QUBO search space (2^{n^2}) explode super-exponentially. The shaded area between $n!$ and 2^{n^2} illustrates the additional infeasible states introduced by the QUBO encoding. (Adapted from [17].)

2.2 Graph Matching

In graph matching, the goal is to find a relabeling of one graph's nodes so that it looks as much like the other as possible. Here, *looks like* means their adjacency matrices A and B are close, and *close* maps to a small Frobenius norm (think of it as the sum of squared differences between all entries). It is widely used in computer vision, network alignment, and medical imaging (Figure 2.3).

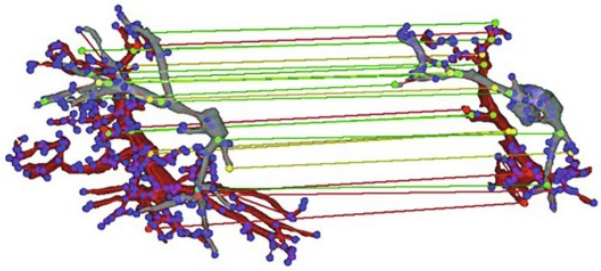


Figure 2.3: Graph matching applied to medical imaging: nodes represent anatomical regions of a liver, and edges encode structural relationships. The task is to find a permutation that aligns the pre-treatment graph (left) with the post-treatment graph (right), minimizing structural disagreement. (Figure from [18].)

Crucially: minimizing $\|A - XBX^\top\|^2$ turns out to be equivalent to maximizing $\operatorname{tr}(A^\top XBX^\top)$. This is also the QAP objective.

$$\text{GMP} = \arg\min_X \ \|A - XBX^\top\|_F^2, \tag{2.4}$$

where $\|A\|_F^2 = \sum_{i,j} a_{ij}^2$ is the squared Frobenius norm, which is equivalent to taking $\operatorname{tr}(A^\top A)$. Thus the GMP can be expressed as

$$\begin{aligned}
\|A - XBX^\top\|_F^2 &= \operatorname{tr}\Big((A - XBX^\top)^\top (A - XBX^\top)\Big) && \text{2.5.a}\\
&= \operatorname{tr}\big((A^\top - XB^\top X^\top)(A - XBX^\top)\big) && \text{2.5.b}\\
&= \operatorname{tr}(A^\top A) - \operatorname{tr}(A^\top XBX^\top) - \operatorname{tr}(XB^\top X^\top A) + \operatorname{tr}(XB^\top X^\top XBX^\top), && \text{2.5.c}
\end{aligned}$$

where $X^\top X = I$, so that the last term first reduces to $\operatorname{tr}(XB^\top BX^\top)$ and by using the cyclic and transpose invariant properties of the trace:

$$\begin{aligned}
\operatorname{tr}(ABCD) &= \operatorname{tr}(DABC)\\
\operatorname{tr}(M) &= \operatorname{tr}(M^\top),
\end{aligned}$$

it further simplifies $\operatorname{tr}(XB^\top BX^\top) = \operatorname{tr}(B^\top B)$; and since $\operatorname{tr}(A^\top XBX^\top) = \operatorname{tr}(XB^\top X^\top A)$, the two middle terms combine to $-2\operatorname{tr}(A^\top XBX^\top)$. Thus, the expression finally simplifies to:

$$\begin{aligned}
&= \operatorname{tr}(A^\top A) + \operatorname{tr}(B^\top B) - 2\operatorname{tr}(A^\top XBX^\top) && \text{2.6.a}\\
&\simeq -\operatorname{tr}(A^\top XBX^\top). && \text{2.6.b}
\end{aligned}$$

Since A and B are constant matrices, $\operatorname{tr}(A^\top A)$ and $\operatorname{tr}(B^\top B)$ are invariant under the choice of X. Therefore, minimizing Equation 2.4 is mathematically equivalent to minimizing $-\operatorname{tr}(A^\top XBX^\top) \rightsquigarrow -z(\boldsymbol{\pi})$ for the QAP [19, 20].

2.3 Quadratic Unconstrained Binary Optimization

Before proceeding, we briefly introduce the Quadratic Unconstrained Binary Optimization (QUBO) formulation, which serves as a foundation for the quantum-inspired heuristics discussed later.

A QUBO problem seeks to minimize a quadratic objective over binary variables:

$$\min_{x \in \{0,1\}^n} \boldsymbol{x}^\top \boldsymbol{Q} \boldsymbol{x} + \boldsymbol{c}^\top \boldsymbol{x}. \tag{2.7}$$

QUBOs are closely tied to the Ising model from spin glass theory. Physicists use it to describe phenomena such as ferromagnetism: for example, iron loses its permanent magnetism above the Curie temperature (∼770 °C), a phase transition where heat randomizes the spin orientations, destroying their collective order [21, 22].

Why do we care about magnets? Because the math is identical. Minimizing the energy of a spin system (finding which spins point up or down) is the same problem as minimizing a QUBO objective. Any optimizer built for spin systems can (in principle) solve any problem we express as a QUBO.

In the Ising model, spins $\boldsymbol{s} \in \{-1, +1\}^n$ represent states, and its energy (also called the Hamiltonian of the system) is expressed as:

$$\mathcal{H} = \sum_{\{i,j\}}^{n} \boldsymbol{J}_{ij} \cdot \boldsymbol{s}_i \cdot \boldsymbol{s}_j + \sum_{i}^{n} \boldsymbol{h}_i \cdot \boldsymbol{s}_i + c \tag{2.8.a}$$

$$= \boldsymbol{s}^\top \boldsymbol{J} \boldsymbol{s} + \boldsymbol{h}^\top \boldsymbol{s} + c. \tag{2.8.b}$$

Minimizing this Hamiltonian corresponds to finding the system's ground state, which solves the optimization problem.

An example can be seen in Figure 2.4 where the edges correspond to the weights J and the nodes to the spins s.

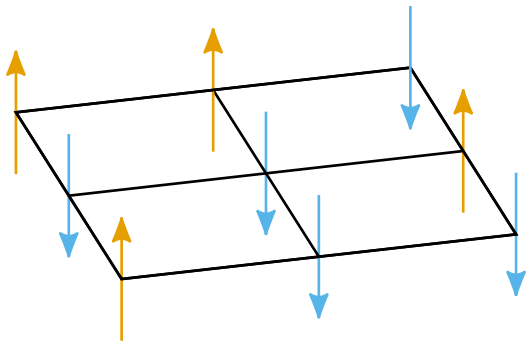


Figure 2.4: A small 2D Ising lattice: nodes represent binary spins $s_i \in \{-1, +1\}$ and edges carry pairwise interaction weights J_{ij}. The ground-state configuration minimizes the Hamiltonian (Equation 2.8).

Mappings between binary and Ising variables allow us to translate between both formulations:

$$s_i = 2x_i - 1 \quad \text{(binary to Ising)}, \qquad x_i = \frac{1 + s_i}{2} \quad \text{(Ising to binary)}. \tag{2.9}$$

This equivalence enables quantum-inspired optimizer methods, such as quantum annealing, to tackle QUBO instances. Notably, many NP-hard discrete optimization problems admit QUBO formulations, making them a natural problem class for quantum computing, though often only at the cost of auxiliary variables and penalty terms, which can render this approach impractical [23].

2.4 Encoding the QAP as QUBO

We now demonstrate how the QAP can be formulated as a QUBO. Doing so highlights the inherent challenges of using a QUBO solver for the QAP, thereby motivating our permutation-based approach. Section 2.4.3 explains why we ultimately discard this approach entirely.

The QAP trace formulation $\operatorname{tr}(\boldsymbol{AXBX}^\top)$ can be vectorized into $\boldsymbol{x}^\top \boldsymbol{Q}\boldsymbol{x}$. First, we flatten the permutation matrix $\boldsymbol{X}$ into a binary vector $\boldsymbol{x}$ and then construct the corresponding matrix $\boldsymbol{Q}$ by taking the Kronecker product

$$\boldsymbol{Q} = \boldsymbol{A} \otimes \boldsymbol{B} = \begin{pmatrix} a_{11}\boldsymbol{B} & \dots & a_{1n}\boldsymbol{B} \\ \vdots & \ddots & \vdots \\ a_{m1}\boldsymbol{B} & \dots & a_{mn}\boldsymbol{B} \end{pmatrix}. \tag{2.10}$$

Computing $\boldsymbol{x}^\top \boldsymbol{Q}\boldsymbol{x}$ is now equivalent to evaluating $\operatorname{tr}(\boldsymbol{AXBX}^\top)$.

In this formulation, the solver deals with n^2 binary variables and explores a search space of 2^{n^2} possible solutions. However, most of them are infeasible because the permutation matrix constraints are not respected, since QUBO problems are inherently unconstrained. The huge gap between the number of feasible and infeasible solutions is illustrated in Figure 2.2 by the shaded area.

The key difficulty is that QUBO solvers do not know about permutations. They just flip bits. So we need to *teach* the solver that X must have exactly one 1 per row and column, by making any violation expensive. We add a penalty term: the more you violate the one-hot constraint, the worse the energy. The solver then avoids violations automatically. This sounds elegant, but it comes at a price described below.

2.4.1 Constraint Penalties

Constraint violations can be measured by the number of times the one-hot condition is not met.

We define a constraint matrix $\boldsymbol{C}$ and target vector $\boldsymbol{b} = \mathbf{1}$:

$$\boldsymbol{C} = \begin{pmatrix} \boldsymbol{I} \otimes \mathbf{1}^\top \\ \mathbf{1}^\top \otimes \boldsymbol{I} \end{pmatrix}, \qquad \boldsymbol{b} = \mathbb{1}, \tag{2.11}$$

where I is the $n \times n$ identity matrix and $\mathbf{1}$ is a vector of ones. The first n rows of C enforce that the columns of X sum up to 1, while the second half enforces that the rows sum up to 1 [24].

The total amount of violations can then be expressed as the squared Euclidean norm

$$\#\text{violations}^2 = \|\boldsymbol{C}\boldsymbol{x} - \boldsymbol{b}\|^2. \tag{2.12}$$

Example

For $n = 3$

$$\boldsymbol{C} = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 & 1 \\ & & & & & & & & \\ 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 1 \\ 1 \\ \\ 1 \\ 1 \\ 1 \end{pmatrix}$$

and a violating example, where X violates two column constraints and one row constraint:

$$\boldsymbol{X} = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 0 & 1 \end{pmatrix} \rightarrow \boldsymbol{x}^\top = (1 \;\; 0 \;\; 0 \;\; 1 \;\; 1 \;\; 1 \;\;\; 0 \;\; 0 \;\; 1)$$

would yield

$$\|\boldsymbol{C}\boldsymbol{x} - \boldsymbol{b}\|^2 = \|(0 \;\; 2 \;\; 0 \;\; 1 \;\; 0 \;\; 1)\|^2 = 6.$$

Expanding the squared norm gives

$$\|\boldsymbol{C}\boldsymbol{x} - \boldsymbol{b}\|^2 = \boldsymbol{x}^\top \boldsymbol{C}^\top \boldsymbol{C}\boldsymbol{x} - 2\boldsymbol{b}^\top \boldsymbol{C}\boldsymbol{x} + \boldsymbol{b}^\top \boldsymbol{b}. \tag{2.13}$$

By Lagrangian relaxation, the one-hot constraints are incorporated directly into the QUBO objective as a penalty, instead of being enforced strictly. This yields

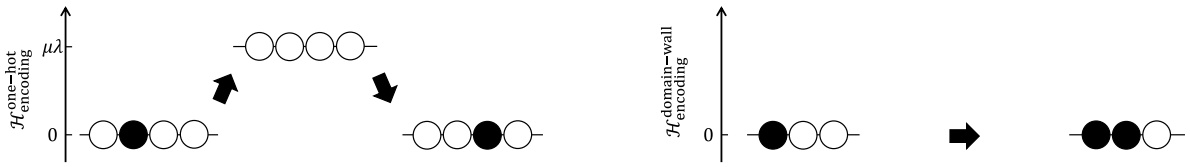
$$\boldsymbol{x}^\top (\boldsymbol{Q} + \lambda \boldsymbol{C}^\top \boldsymbol{C})\boldsymbol{x} + (\boldsymbol{c} - 2\lambda \boldsymbol{b}^\top C)\boldsymbol{x}, \tag{2.14}$$

where λ controls the penalty strength.

Finding an optimal λ is NP-hard (the Big-M problem) [25]: if too low, solutions are invalid; if too high, the energy landscape becomes artificially steep and difficult to traverse.

2.4.2 Alternative Encoding

Unary (or domain-wall) encoding represents an integer k by setting the first k bits to 1 (e.g., for a 5-bit system $k = 3$ is `11100`) [26]. It guarantees fewer infeasible intermediate states during local search compared to one-hot encoding (Figure 2.5), but it still relies on Big-M penalties and increases implementation complexity.



(a) One-Hot Encoding **(b)** Unary Encoding

Figure 2.5: Two binary encodings for an integer variable over five values: one-hot encoding (left) activates a single bit per value; domain-wall encoding (right) sets the k first bits to 1. Single-bit flips in domain-wall encoding can yield adjacent valid states, unlike one-hot. (Figure from [27].)

2.4.3 Why not use the QUBO for QAP?

The QUBO formulation introduces crippling inefficiencies for the QAP:

- **Search space explosion**: A problem of size $n = 10$ has $10! \approx 3.6 \times 10^6$ valid permutations, but the QUBO encodes $2^{10^2} = 2^{100} \approx 1.27 \times 10^{30}$ binary states.
- **Memory scaling**: Storing A and B takes $2n^2$ elements. Storing Q takes n^4 entries [28].
- **Penalty tuning**: Resolving the Big-M problem is computationally difficult and distorts the optimization landscape.
- **Hardware limits**: Due to qubit embedding overhead in limited-connectivity topologies, modern quantum annealers like D-Wave restrict fully connected problems to roughly $n \approx 15$ [29].

Therefore, we discard the QUBO encoding and instead use native permutation operators that inherently maintain feasibility, a conclusion supported by earlier attempts to binary-encode the QAP [30].

3 State of the Art

Because the QAP is NP-hard, no single algorithm dominates across all instance families, and decades of algorithmic effort have produced a diverse landscape of complementary strategies. This chapter surveys four major lines of work: exact methods, classical metaheuristics, continuous relaxation approaches, and quantum-inspired methods. For each line, we identify the technical limitation most relevant and finally synthesize the resulting gap.

3.1 Exact Methods

As established in Section 2, complete enumeration is infeasible. Classical exact methods such as **Branch and Bound** attempt to mitigate this by reducing the search space via *branching*, *bounding*, and *selection* [10]. Branch and Bound works by exploring move trees. At each step, it either *branches* (tries a subproblem) or *bounds* (proves a subtree cannot beat the current best, and prunes it). In theory, this avoids checking every permutation, but in practice, the pruning is insufficient for QAP: the lower bounds leave too many branches alive.
While increasingly sophisticated lower-bounding techniques have been developed, these methods typically scale poorly and quickly become impractical. In practice, exact solvers are therefore limited to relatively small instances [31].
Beyond this threshold, the search must rely on heuristics that trade optimality guarantees for tractability.

3.2 Classical Heuristics

Classical heuristics trade optimality for tractability by accepting solutions that are locally, but not globally, optimal.

SA was among the earliest successful approaches for the QAP permutation space [32] and remains actively studied [33]. Its core limitation is schedule sensitivity: cooling too fast traps the search in local minima, while cooling too slowly is computationally prohibitive.
Tabu Search [34, 35] addresses SA's local-minima problem by augmenting neighborhood search with a short-term memory structure that forbids recently visited configurations. This prevents cycling and allows the search to accept non-improving moves. Tabu Search is the strongest available CPU baseline for the QAP used in this thesis. In

2019 [1], it produced a new best-known solution for tai100a [2], albeit requiring months of computation and careful parameter tuning.
Despite their different mechanisms, Tabu Search and SA show comparable performance depending on instance structure and tuning [36].
In 2025, **Parallel Tempering** (PT) with caching was applied to solve the QAP. A detailed description is provided in Section 4.2.3 and Section 4.5 [28].

Beyond these, numerous ensemble and hybrid methods have been explored, including **memetic** and **evolutionary** algorithms [37]; **swarm-based** approaches such as ant colony, particle swarm and bat optimization [38]; and **hybrid** combinations of metaheuristics with genetic algorithms [39, 40]. **Deep learning** approaches have also been explored, including temperature-softmax relaxations for assignments [41] and reinforcement learning methods [42].

Despite this diversity, all methods in this family are sensitive to hyperparameter tuning and none provides systematic escape from local minima at scale. This motivates the continuous relaxation and quantum-inspired methods reviewed next.

3.3 Continuous Relaxation Methods

Continuous optimization has long been studied in mathematics and is increasingly applied to combinatorial problems. For the QAP, continuous relaxations are used both to derive stronger lower bounds and to approximately solve the assignment problem itself.

On the lower-bound side, [43] and [44] use continuous relaxations to obtain tighter bounds.
For approximate solving, the Fast Approximate QAP Algorithm (FAQ) [20] (implemented in SciPy [45]) relaxes the permutation constraint $\mathcal{P}$ to a convex hull $\mathcal{D}$:

$$\mathcal{P} = \left\{P \in \{0,1\}^{n\times n} : P^\top \mathbf{1} = P\mathbf{1} = \mathbf{1}\right\} \tag{3.1.a}$$

$$\mathcal{D} \coloneqq \mathcal{D}_n = \left\{P \in \mathbb{R}^{n\times n} : P^\top \mathbf{1} = P\mathbf{1} = \mathbf{1}, P \succeq 0 \text{ (element-wise inequality)}\right\} \tag{3.1.b}$$

This relaxation enables the use of continuous optimization techniques. In this smooth landscape, gradient descent works, and at the end, it snaps the continuous solution back to the nearest permutation. It is generally fast. However, it is not guaranteed to be optimal, as it can get stuck in local optima. Specifically, the FAQ applies the Frank-Wolfe method to the relaxed search space $\mathcal{D}$ to find a maximizing permutation as follows:

[2] tai100a is a well-known, highly challenging QAP instance that serves as a core benchmark in Section 6.

Initialize: $P^0 \in \mathcal{D}$, barycenter $(\frac{1}{n}\mathbf{1}_n \times \mathbf{1}_n^\top)$

for $i = 1, 2, 3, \ldots$ **do**:

(1) Compute $\nabla z\left(P^i\right) = AP^iB^\top + A^\top P^iB$.

(2) Compute $Q^i = \arg\min \ \operatorname{tr}\left(Q^\top \nabla z\left(P^i\right)\right)$ over $Q \in \mathcal{D}$ via Hungarian algorithm.

(3) Compute step size $\alpha^i = \arg\min \ z\left(\alpha P^i + (1-\alpha)Q^i\right)$ with $\alpha \in [0,1]$

(4) Set $P^{i+1} = \alpha^i P^i + \left(1-\alpha^i\right)Q^i$

end

return $\hat{Q} = \arg\max \ \operatorname{tr}\left(Q^\top \nabla z\left(P^{\text{final}}\right)\right)$ over $Q \in \mathcal{P}$ via Hungarian algorithm.

Saad-Eldin et al. [19] improved the FAQ method by replacing the Hungarian algorithm used in step *(2)* with the faster *Lightspeed Optimal Transport* method. But even with this improvement, FAQ remains sensitive to initialization and can still become trapped in local minima [18, 20]. Because the relaxation operates in a continuous doubly-stochastic space rather than the discrete permutation space, gradient descent cannot exploit the discrete structure of the QAP. This limitation motivates the discrete, physics-inspired dynamics explored in the following section.

3.4 Quantum and Quantum-Inspired Methods

Several quantum-inspired methods use QUBO formats:

- *QuCOOP* [29] iteratively approximates the QAP objective by a sequence of local QUBOs, using a 2-cycle parametrization with $\frac{n(n-1)}{2}$ binary variables. On simulated annealing it achieves 5.53% mean error on 72 QAPLIB instances ($n \leq 50$). On real D-Wave hardware, however, valid permutations are only reliably recovered up to $n \approx 12$.
- *QuAnt* [46] trains a neural network to regress QUBO matrices directly from problem instances, replacing analytical derivations entirely. This enables compact solution encodings, significantly reducing qubit requirements. The key caveat is that the learned QUBO is a black box.

Other methods operate in different spaces:

- *QuPer* [47] uses a variational quantum circuit based on the Bruhat decomposition to output doubly-stochastic matrices (similar to FAQ), requiring only $2\log_2(n)$ qubits via amplitude encoding. Without ancilla qubits ($m = 0$), the circuit is classically simulable but only spans a tiny fraction of all permutations (e.g. 1344 out of 40320 for $n = 8$); adding ancilla qubits introduces quantum behaviour and significantly enlarges the span. All experiments are nonetheless classically simulated, feasible only because n stays small enough. They demonstrate competitiveness with FAQ on QAPLIB instances up to $n = 256$ (20 qubits, including $m = 2$ ancilla qubits).
- Daly et al. [48] reduce the QAP to a Maximum Weighted Independent Set problem on Unit Disk Graphs, enabling execution on Rydberg neutral atom arrays. The QAP is encoded in $(n-1)^2$ binary variables (vs. n^2 one-hot). However, the approach remains purely theoretical, as no hardware experiments have been conducted yet.

Although focused on Ising models rather than the QAP directly, in 2025, S. Chowdhury *et al.* [49] provided a systematic study for PT and SQA, analyzing the impact of replica count, adaptive temperature ladders and isoenergetic cluster moves.
IsingFormer [50] augments PT with transformer-learned move proposals for Ising systems, demonstrating that learned heuristics can improve mixing without abandoning the PT framework.

The commercial solver nonlinear-program (*NL*) by *D-Wave* avoids the QUBO formulation and is used as an external solver in our comparison discussed in Section 6.1.7. However, its internal mechanism is proprietary.

Crucially, to our knowledge, neither SQA nor MA has ever been formulated to operate directly in the permutation space of the QAP.

3.5 Summary and Motivation

No single method dominates across all QAP instances. Exact methods guarantee optimality but are limited to $n \lessapprox 30$. Classical heuristics such as SA and Tabu Search scale well but are highly sensitive to parameter tuning and instance structure: the best-known solution for `tai100a` required months of computation. Continuous relaxation methods like FAQ are fast but depend heavily on initialization and frequently converge to poor local optima. Quantum and quantum-inspired methods potentially avoid some of these pitfalls but often rely on the QUBO encoding (introducing the search space explosion).

This motivates a permutation-native approach which operates directly in the feasible space of $n!$ assignments. While SA has long been applied directly to the QAP in permutation space, we are not aware of an analogous formulation of either SQA or MA for the QAP in this representation. SQA has been applied to the related TSP [51]; feasibility there was maintained by restricting Monte Carlo updates to valid tours via 2-opt moves within an Ising-style formulation.

This survey reveals a consistent gap: while SA has been applied natively in permutation space since 1984, neither SQA nor MA has ever been formulated for the QAP without binary encoding (to our knowledge). Chapter 4 closes this gap by adapting physics-inspired annealing dynamics to operate directly on integer permutations, eliminating penalty tuning and search-space inflation by construction.

4 Heuristics and Methods

> All main algorithmic techniques for the exact solution of NP-hard problems have been used for attacking the QAP: decomposition, branch-and-bound, and branch-and-cut. The QAP, however, bravely resisted. After decades of attacks, the results obtained on its optimal solution are far from being satisfactory [...] [52].
>
> Not only that the QAP cannot be solved efficiently but it even cannot be approximated efficiently within some constant approximation ratio. Furthermore, finding local optima is not a trivial task even for simply structured neighborhoods like the 2-opt neighborhood [53].
>
> — R. E. Burkard

We attack the QAP with physics-inspired heuristics, drawing analogies from metallurgical annealing, quantum tunneling, and statistical mechanics. This chapter introduces the base neighborhood operators and adapts various trajectory and ensemble-based methods (see Figure 4.1) directly to the QAP's permutation space.

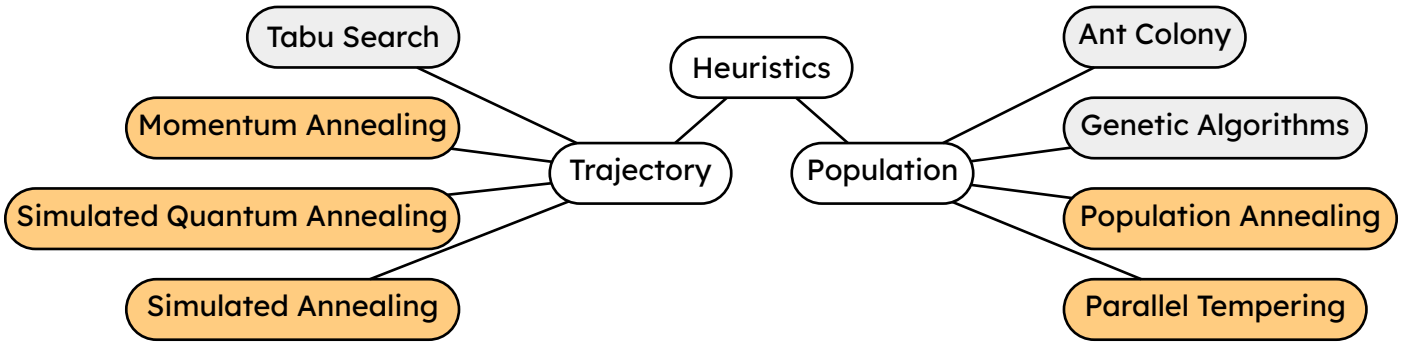


Figure 4.1: Taxonomy of metaheuristic optimization strategies: divided into trajectory-based methods (which optimize a *single* solution over time) and population-based methods (which evolve a set of solutions). Algorithms implemented and evaluated in this thesis are highlighted; Tabu Search, Genetic Algorithms, and Ant Colony Optimization are included for context but not implemented.

4.1 Base Move: 2-Opt

Every solver in this thesis speaks a common language: the 2-opt swap. It is the simplest move that keeps a permutation valid: swap two assignments. Everything else is just a different strategy for deciding when and how to apply this atomic operation.

Local search methods iteratively improve an existing solution by applying small modifications (producing a neighbor) and form the backbone of most heuristic algorithms. This framework parallels continuous optimization, where gradient descent is used to find optima; however, in discrete spaces lacking gradients, local search relies on discrete neighborhood operators [12].

Unlike spin systems where single bits flip independently, permutation spaces require swapping two assignments to maintain feasibility. We treat the 2-opt swap as the atomic move for the QAP.

Given a permutation $\boldsymbol{\pi}$, a 2-opt move swaps the elements at positions i and j, yielding $\boldsymbol{\pi}' = \boldsymbol{\pi} \oplus p_{ij}$. As an example, see Figure 4.2 in which the 2-opt move $p_{3,7}$ is illustrated.

$$\boldsymbol{\pi} = \begin{pmatrix} 1 & 2 & 4 & 3 & 7 & 6 & 5 & 8 \end{pmatrix}$$

$$\boldsymbol{\pi}' = \boldsymbol{\pi} \oplus p_{3,7} = \begin{pmatrix} 1 & 2 & 5 & 3 & 7 & 6 & 4 & 8 \end{pmatrix}.$$

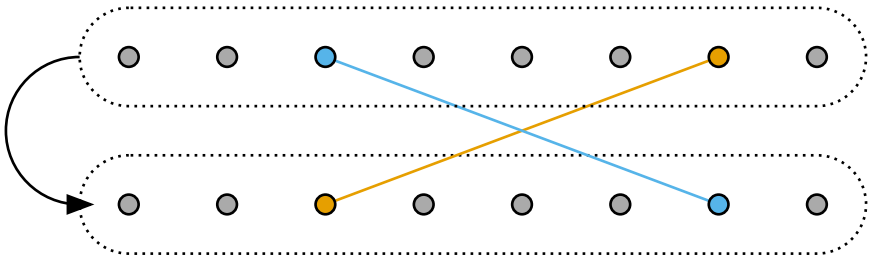


Figure 4.2: A 2-opt swap applied to positions 3 and 7 of the permutation $\boldsymbol{\pi}$: the elements at indices 3 and 7 (values 4 and 5) are exchanged, yielding $\boldsymbol{\pi}'$. All other positions remain unchanged, and the resulting sequence is still a valid permutation.

For the QAP the cost difference for this swap is given by:

$$\Delta z(\boldsymbol{\pi}, i, j) = z(\boldsymbol{\pi}') - z(\boldsymbol{\pi}). \tag{4.1}$$

Instead of calculating this difference naively by recomputing the full double sum in the QAP, Burkard and Rendl [32] show that for asymmetric A and B it reduces to

$$\Delta z = \sum_{k=1,k\neq i,j}^{n} \left[\left(A_{jk} - A_{ik} \right) \left(B_{\pi_i \pi_k} - B_{\pi_j \pi_k} \right) + \left(A_{kj} - A_{ki} \right) \left(B_{\pi_k \pi_i} - B_{\pi_k \pi_j} \right) \right] \tag{4.2.a}$$

$$+ \left(A_{jj} - A_{ii} \right) \left(B_{\pi_i \pi_i} - B_{\pi_j \pi_j} \right) + \left(A_{ji} - A_{ij} \right) \left(B_{\pi_i \pi_j} - B_{\pi_j \pi_i} \right). \tag{4.2.b}$$

For symmetric A and B, this further simplifies to:

$$\Delta z = 2 \sum_{k=1,k\neq i,j}^{n} \left(A_{jk} - A_{ik} \right) \left(B_{\pi_i \pi_k} - B_{\pi_j \pi_k} \right) \tag{4.3.a}$$

$$+ \left(A_{jj} - A_{ii} \right) \left(B_{\pi_i \pi_i} - B_{\pi_j \pi_j} \right). \tag{4.3.b}$$

The cost difference formula avoids redundant computation, making the 2-opt move much more efficient. Concretely it takes time proportional to $O(n)$ instead of $O\left(n^2\right)$. In Section 4.2.3 we will further reduce this to $O(1)$ by precomputing a cache matrix.

4.2 Simulated Annealing

Simulated Annealing (SA) is inspired by a process in physics. In materials such as metals, the internal structure changes depending on how quickly they are cooled after heating. If the material is cooled rapidly, its atoms do not have enough time to settle into the usual low-temperature arrangement, so it forms many small grains with different orientations. This is called quenching, and it is used, for example, to harden some steels. If the cooling is slow, however, the atoms have time to organize into larger crystals, which correspond to a lower-energy state [12].

In 1983, Kirkpatrick et al. [54] first introduced SA for optimization of combinatorial problems (TSP, chip placement). By starting at high temperatures, SA can explore the energy landscape with a higher likelihood of accepting worse solutions. As the temperature decreases over time, the algorithm gradually focuses on improving solutions.
The key insight of SA is that sometimes accepting a worse solution is good, as it prevents getting permanently stuck.
The probability of transitioning between states is governed by the Metropolis condition [54]:

$$p = \min\left(1, e^{-\beta \Delta E}\right), \tag{4.4}$$

where $\beta = \frac{1}{T}$ is the inverse temperature and ΔE is the energy difference. Notably, transitions with $\Delta E < 0$ are always accepted. Worsening transitions being accepted depends both on the amount of change and the current temperature.

4.2.1 Cooling Schedules

Selecting a temperature schedule involves balancing broad exploration with strict convergence. While theoretical schedules guarantee global optimality, they require infinite time.

Common practical schedules include:

$$t_{k+1} = t_k - \alpha, \qquad \alpha > 0 \qquad (\textbf{Linear}) \tag{4.5.a}$$

$$t_{k+1} = t_k \cdot \alpha = t_0 \cdot \alpha^k, \qquad \alpha < 1 \qquad (\textbf{Geometric}) \tag{4.5.b}$$

$$t_{k+1} = \frac{1}{\beta_{k+1}}, \quad \beta_{k+1} = \alpha + \beta_k, \qquad \alpha > 0 \qquad (\textbf{Inverse Linear}) \tag{4.5.c}$$

$$t_{k+1} = \frac{t_k}{1 + \alpha t_k}, \qquad \alpha < 1 \qquad (\textbf{Lundy \& Mees}). \tag{4.5.d}$$

The schedules are illustrated in Figure 4.3.

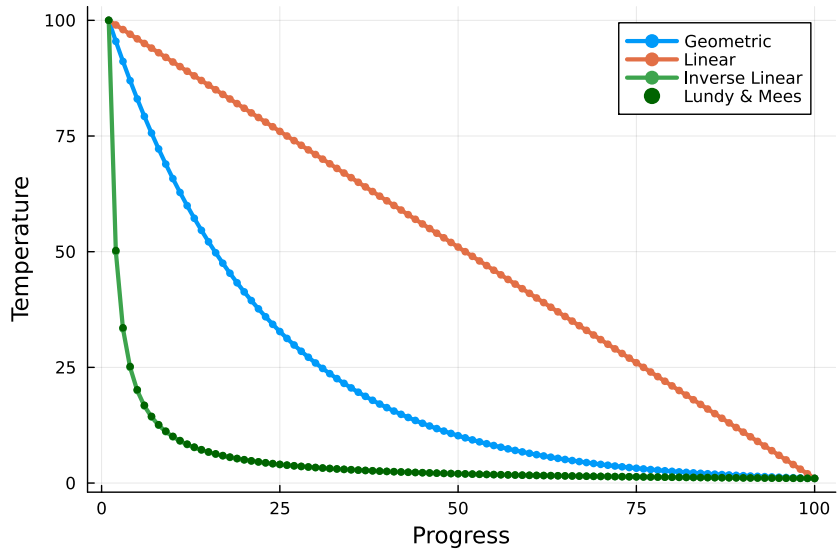


Figure 4.3: Four cooling schedules: each reducing temperature from 100 to 1 over 100 steps. Linear cooling decreases at a constant rate; geometric cooling decays multiplicatively, spending more time near low temperatures; inverse-linear and Lundy & Mees schedules are algebraically equivalent and cool the fastest. The choice of schedule governs the trade-off between exploration (high T) and exploitation (low T).

However, all schedules face a fundamental dilemma: slow cooling requires excessive iterations, while fast cooling traps the system in local minima. While restart mechanisms can partially address this, we largely circumvent this tuning problem in later sections by using Parallel Tempering (Section 4.5), which exchanges configurations across fixed temperatures rather than cooling them over time.

4.2.2 Application to the QAP

Franzin and Stützle [33] reviewed several SA design choices for the QAP, including:

- **Neighborhood exploration**: their study suggests following Connolly [55] by sequentially testing all pairs of indices i and j for potential swaps. One complete pass requires evaluating all $\binom{n}{2} = \frac{n(n-1)}{2}$ possible swaps. This is in contrast to random sampling of neighbors, which may miss promising improvements [33].
- **Temperature extremes**: Misevičius [56] proposed a method to set the initial temperature based on the average cost difference of random swaps:

$$t_0 = (1 - \lambda_1 - \lambda_1')\Delta_{\min} + \lambda_1\Delta_{\text{avg}} + \lambda_1'\Delta_{\max} \tag{4.6.a}$$

$$t_f = (1 - \lambda_2 - \lambda_2')\Delta_{\min} + \lambda_2\Delta_{\text{avg}} + \lambda_2'\Delta_{\max}. \tag{4.6.b}$$

  Misevičius [56] settled on $(\lambda_1 = 0.5, \lambda_2 = 0.05, \lambda_1' = \lambda_2' = 0)$. We adopted this scheme but reduced $\lambda_2 = 0.01$ for even lower final temperatures. Connolly [55] did not consider the average cost differences but only the minimum and maximum, which is more sensitive to outliers.
- **Restart mechanisms**: To mitigate premature freezing, some form of reheating or restart mechanism is recommended. For instance, Connolly [55] combined the *Lundy & Mees* cooling schedule with reheating, resetting the temperature to the level at

which the best solution had been found after a predefined number of unsuccessful trials.

See Algorithm 4.1 for a general framework of SA applied to the QAP.

Simulated Annealing for the QAP

```julia
x = randperm(qap.n); E = z(x); best = (x=x, E=E)

for step ∈ 1:n_steps
  # connolly neighborhood exploration
  for i in 1:qap.n, j in i+1:qap.n
    ΔE = Δz(qap, x, i, j)

    # metropolis condition
    if rand() ⩽ exp(-ΔE / temp_schedule(step))
      x[[i,j]] .= x[[j,i]]  # perform swap
      E += ΔE

      # update best found
      if E < best.E; best = (x=x, E=E); end
    end
  end
end
```

Algorithm 4.1: SA flow for the QAP (adapted from [57])

4.2.3 Efficient Cost Difference Evaluation via Cache

Paul [58] first introduced a cache matrix for the QAP. More recently, Boltzmann Machine Caching, which adapted the notion of local fields from QUBO models to integer-variable optimization problems, was first applied to the QAP in [59].

The central idea is to precompute a cache matrix $\boldsymbol{H}$ in $\mathcal{O}\left(n^2\right)$, so that the cost of swapping two elements in the permutation can then be evaluated in $\mathcal{O}(1)$.

The cache matrix is defined as

$$\boldsymbol{H} = \boldsymbol{H}(\boldsymbol{X}) := \begin{cases} \boldsymbol{A}^\top \boldsymbol{X} \boldsymbol{B} & = \boldsymbol{A}^\top \boldsymbol{B}_{\boldsymbol{\pi},*} & \text{if symmetric} \\ \boldsymbol{A}^\top \boldsymbol{X} \boldsymbol{B} + \boldsymbol{A} \boldsymbol{X} \boldsymbol{B}^\top & = \boldsymbol{A}^\top \boldsymbol{B}_{\boldsymbol{\pi},*} + \boldsymbol{A} \boldsymbol{B}^\top_{*,\boldsymbol{\pi}} & \text{if asymmetric} \end{cases} \tag{4.7}$$

where $\boldsymbol{X}$ is the permutation matrix corresponding to assignment $\boldsymbol{\pi}$.

Bagherbeik et al. [59] derived formulas for Δz using the cache, achieving $\mathcal{O}(1)$ evaluation.
However, their formulas assumed that diagonal entries of $\boldsymbol{A}$ and $\boldsymbol{B}$ vanish. We extend this to arbitrary diagonals (used e.g., for keyboard layout optimization [5]):

$$\Delta z_{\text{symmetric}(\boldsymbol{\pi},i,j)} = \Big[\boldsymbol{H}_{i,\boldsymbol{\pi}_j} + \boldsymbol{H}_{j,\boldsymbol{\pi}_i} - \boldsymbol{H}_{i,\boldsymbol{\pi}_i} - \boldsymbol{H}_{j,\boldsymbol{\pi}_j} \qquad \text{(calculates loop)} \tag{4.8.a}$$

$$-\left(\boldsymbol{A}_{j,i}-\boldsymbol{A}_{i,i}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_i}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_i}\right) \qquad \text{(subtract for } k=i\text{)} \quad 4.8.\text{b}$$

$$-\left(\boldsymbol{A}_{j,j}-\boldsymbol{A}_{i,j}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_j}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_j}\right)\Big]\cdot 2 \qquad \text{(subtract for } k=j\text{)} \quad 4.8.\text{c}$$

$$-\left(\boldsymbol{A}_{i,i}-\boldsymbol{A}_{j,j}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_i}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_j}\right) \qquad \text{(after loop).} \quad 4.8.\text{d}$$

The first line computes the full loop contribution of Equation 4.3.a, including the cases $k \in \{i,j\}$. This corresponds to the vectorized dot product

$$\left(\boldsymbol{A}_{j,*}-\boldsymbol{A}_{i,*}\right)^{\top}\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}}\right).$$

The following two lines subtract the said cases and the final term is identical to Equation 4.3.b.

The asymmetric (Equation 4.2) works analogously:

$$\Delta z_{\text{asymmetric}(\boldsymbol{\pi},i,j)} = \boldsymbol{H}_{i,\boldsymbol{\pi}_j}+\boldsymbol{H}_{j,\boldsymbol{\pi}_i}-\boldsymbol{H}_{i,\boldsymbol{\pi}_i}-\boldsymbol{H}_{j,\boldsymbol{\pi}_j} \qquad \text{(calculates loop)} \quad 4.9.\text{a}$$

$$-\left(\boldsymbol{A}_{j,i}-\boldsymbol{A}_{i,i}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_i}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_i}\right) \qquad \text{(subtract for } k=i\text{)} \quad 4.9.\text{b}$$

$$-\left(\boldsymbol{A}_{i,j}-\boldsymbol{A}_{i,i}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_i}-\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_j}\right) \qquad \text{(subtract for } k=i\text{)} \quad 4.9.\text{c}$$

$$-\left(\boldsymbol{A}_{j,j}-\boldsymbol{A}_{i,j}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_j}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_j}\right) \qquad \text{(subtract for } k=j\text{)} \quad 4.9.\text{d}$$

$$-\left(\boldsymbol{A}_{j,j}-\boldsymbol{A}_{j,i}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_i}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_j}\right) \qquad \text{(subtract for } k=j\text{)} \quad 4.9.\text{e}$$

$$-\left(\boldsymbol{A}_{i,i}-\boldsymbol{A}_{j,j}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_i}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_j}\right) \qquad \text{(after loop)} \quad 4.9.\text{f}$$

$$-\left(\boldsymbol{A}_{i,j}-\boldsymbol{A}_{j,i}\right)\left(\boldsymbol{B}_{\boldsymbol{\pi}_i,\boldsymbol{\pi}_j}-\boldsymbol{B}_{\boldsymbol{\pi}_j,\boldsymbol{\pi}_i}\right) \qquad \text{(after loop).} \quad 4.9.\text{g}$$

After performing an accepted swap $\boldsymbol{\pi}(i) \leftrightarrow \boldsymbol{\pi}(j)$, the cache matrix can be updated without recomputing it from scratch [28, 59].

For symmetric QAP:

$$\Delta\boldsymbol{A} = \boldsymbol{A}_{j,*}-\boldsymbol{A}_{i,*} \qquad \Delta\boldsymbol{B} = \boldsymbol{B}_{\boldsymbol{\pi}_i,*}-\boldsymbol{B}_{\boldsymbol{\pi}_j,*} \quad 4.10.\text{a}$$

$$\boldsymbol{H} = \boldsymbol{H}+\Delta\boldsymbol{A}^{\top}\otimes_{\text{Outer}}\Delta\boldsymbol{B}. \quad 4.10.\text{b}$$

For asymmetric QAP, we must additionally add

$$\Delta\boldsymbol{A} = \boldsymbol{A}_{*,j}-\boldsymbol{A}_{*,i} \qquad \Delta\boldsymbol{B} = \boldsymbol{B}_{*,\boldsymbol{\pi}_i}-\boldsymbol{B}_{*,\boldsymbol{\pi}_j} \quad 4.11.\text{a}$$

$$\boldsymbol{H} = \boldsymbol{H}+\Delta\boldsymbol{A}\otimes_{\text{Outer}}\Delta\boldsymbol{B}^{\top}. \quad 4.11.\text{b}$$

Because cache updates cost $O\left(n^2\right)$, this method is only advantageous at low acceptance rates (e.g., late-stage SA or cold PT replicas).

For Simulated Annealing with Caching (SAC), we evaluate all possible swaps with index i in parallel, but only accept at most one. This resembles the scheme used by the Digital Annealer [60].

4.3 Simulated Quantum Annealing

We use Simulated Quantum Annealing (SQA) only as algorithmic inspiration: the method is a purely heuristic, permutation-native construction, not a physical simulation. We still adopt some physics terminology, such as *worldlines* or *Trotter slices*, for convenience, but these terms should be understood as analogies rather than literal physical phenomena.

As Figure 4.4 illustrates, tunneling enables barrier crossings regardless of temperature, whereas thermal jumps become exponentially unlikely as the system cools [61].

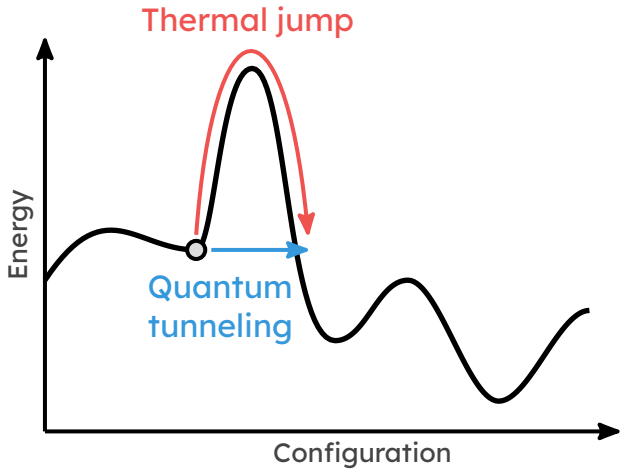


Figure 4.4: Comparison of classical and quantum escape mechanisms: classical thermal fluctuations (red arrow) rely on random statistical chance to jump over the barrier, which becomes exponentially unlikely at low temperatures. Quantum tunneling (blue arrow) allows the system to pass through the barrier, enabling more efficient escapes from local minima. This is the key motivation for Quantum Annealing and its simulated analogue SQA. (Adapted from [62].)

The key difference between SQA and SA lies in their state representation: whereas SA runs a single permutation through time, SQA runs P copies simultaneously (called Trotter slices) connected in a ring. Each slice evolves like standard SA, but neighbors are penalized for disagreeing. At the start, coupling is weak: slices roam freely. Gradually, the coupling strengthens, forcing slices to align into a single consensus. The *tunneling* happens when the coupled *superstate* can shift through an energy barrier that no individual slice could cross alone.
This advantage is sharpest for barriers that are tall but narrow: energetically costly to cross step by step, yet separated by only a few assignment changes from the other side. A single SA replica must climb over the full height. The SQA superstate can span across: slices that have found a better assignment pull their neighbors toward it; the reverse can happen too, though less readily.

Formally, this is motivated by the time-dependent Hamiltonian of Quantum Annealing (QA):

$$\mathcal{H}_q = -\sum_{\{i,j\}\in\mathcal{V}} \boldsymbol{J}_{ij}\sigma_i^z\sigma_j^z - \sum_i \boldsymbol{h}_i\sigma_i^z - \Gamma(t)\sum_i \sigma_i^x, \tag{4.12}$$

where σ_i^z and σ_i^x denote the Pauli z and x operators and $\Gamma(t)$ is the transverse field.

QA can be simulated on a classical computer by mapping the quantum Hamiltonian $\mathcal{H}_q$ to a $(d+1)$-dimensional classical Ising Hamiltonian via the Suzuki–Trotter [63] decomposition:

$$\mathcal{H}_{d+1} = \sum_{k}^{P} \left[\sum_{\{i,j\}\in\mathcal{V}}^{N} \boldsymbol{J}_{ij} \boldsymbol{x}_i^{[k]} \boldsymbol{x}_j^{[k]} + \sum_{i}^{N} \boldsymbol{h}_i \boldsymbol{x}_i^{[k]} + J_\perp \sum_{i}^{N} \boldsymbol{x}_i^{[k]} \boldsymbol{x}_i^{[k+1]} \right], \tag{4.13}$$

where P is the number of Trotter slices and $J_\perp$ is the coupling.

This Suzuki–Trotter decomposition introduces an imaginary-time axis, allowing classical Monte Carlo to approximate the annealing processes. Increasing the number of slices P improves the imaginary-time resolution, but SQA typically uses finite P, yielding systematic discretization errors of order $\Delta\tau = \frac{\beta}{P}$ contrary to continuous Quantum Monte Carlo [64].

4.3.1 Application to the QAP

Standard SQA operates on binary spin variables and has a physical interpretation in terms of quantum annealing.

We reformulate the $(d+1)$-dimensional Hamiltonian from Equation 4.13 for integers by replacing binary variables $x_{ij}^{[k]}$ with integer variables $\boldsymbol{\pi}_i^{[k]}$:

$$\mathcal{H}_{d+1} = \sum_{k=1}^{P} \operatorname{tr}\left(A X_{[k]} B X_{[k]}^\top \right) + J_\perp \left(n - \operatorname{tr}\left(X_{[k]} X_{[k+1]}^\top \right) \right) \tag{4.14.a}$$

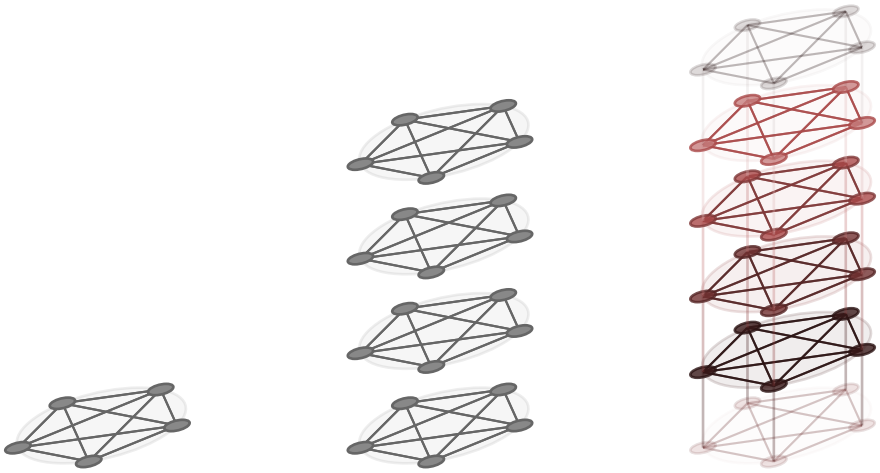
$$= \underbrace{\sum_{k=1}^{P} z\left(\boldsymbol{\pi}^{[k]}\right)}_{\text{Intra Slice}} + \underbrace{J_\perp \sum_{i=1}^{N} 1 - \delta\left(\boldsymbol{\pi}_i^{[k]}, \boldsymbol{\pi}_i^{[k+1]}\right)}_{\text{Inter Slice: } J_\perp \cdot g(\boldsymbol{\pi}^{[k]}, \boldsymbol{\pi}^{[k+1]})}, \tag{4.14.b}$$

where δ is the Kronecker delta (1, if the two assignments equal and 0 otherwise).

The Inter-Slice term $J_\perp g\left(\boldsymbol{\pi}^{[k]}, \boldsymbol{\pi}^{[k+1]}\right)$ is the permutation-space analogue of the spin-spin coupling in the original Ising formulation $J_\perp \sum_i^N \boldsymbol{x}_i^{[k]} \boldsymbol{x}_i^{[k+1]}$ (Equation 4.13). In the binary case, this coupling counts the number of positions where two adjacent slices agree (both spin-up or both spin-down). For permutations, agreement at position i means $\boldsymbol{\pi}_i^{[k]} = \boldsymbol{\pi}_i^{[k+1]}$, i.e., both slices assign the same facility to location i. The Hamming distance $g\left(\boldsymbol{\pi}^{[k]}, \boldsymbol{\pi}^{[k+1]}\right) = \sum_{i=1}^{N} 1 - \delta\left(\boldsymbol{\pi}_i^{[k]}, \boldsymbol{\pi}_i^{[k+1]}\right)$ therefore counts the number of disagreements, making it the natural permutation analogue of the binary mismatch penalty. As $J_\perp$ increases, adjacent slices are increasingly penalized for differing assignments, forcing the worldlines to collapse into a single classical permutation, mirroring the freeze-out of quantum fluctuations in standard QA.

The asymmetry noted above now becomes apparent: moving toward a better assignment earns both the coupling reward and a lower intra-slice QAP cost simultaneously. Moving toward a worse assignment still earns the coupling reward, but pays the intra-slice QAP cost in return, one reward against one penalty, rather than two rewards at once.

Figure 4.5 shows three regimes. A single slice **(a)** is just SA and multiple uncoupled slices **(b)** is just running SA many times in parallel with no information sharing. Only with coupling **(c)** does something new happen: the slices form a superstate that can move collectively. Without the coupling, additional slices provide no information exchange; it is precisely the inter-slice penalty $J_\perp$ that transforms independent runs into a coordinated superstate capable of tunneling-like dynamics.



(a) Single Slice **(b)** Uncoupled Slices **(c)** Coupled Slices

Figure 4.5: From single SA-slice to SQA-slice topology: **(a)** a single permutation slice as in standard SA, where one classical state evolves independently; **(b)** multiple uncoupled slices, equivalent to running SA in parallel with no information exchange; and **(c)** multiple slices coupled by the inter-slice penalty $J_\perp$ of SQA, forming a superstate whose worldlines are forced to progressively align. (Inspired by [65].)

The ring topology in Figure 4.6 shows the periodic boundary conditions and the even/odd *coloring* directly maps to a GPU parallelization used in our implementation.

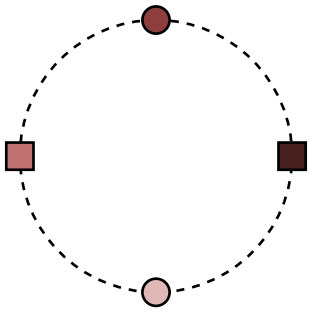


Figure 4.6: Ring topology of Trotter slices under periodic boundary conditions: each slice $\boldsymbol{\pi}[k]$ is coupled to its two direct neighbors $\boldsymbol{\pi}[k-1]$ and $\boldsymbol{\pi}[k+1]$, and the last slice wraps back to the first. The sequence of assignments $\{\boldsymbol{\pi}[1], \ldots, \boldsymbol{\pi}[P]\}$ forms the worldline of facility i.

To optimize the $\mathcal{H}_{d+1}$, we apply stochastic Monte Carlo updates similar to SA. During the annealing we gradually increase the coupling $J_\perp$ to align the Trotter slices. As is common for SQA, we employ two types of Monte Carlo updates:

- **Local sweeps** propose 2-opt swaps within a single Trotter slice $\boldsymbol{\pi}^{[k]}$, similar to standard SA.

- **Global sweeps**: simultaneous equivalent swaps across all slices, effectively flipping entire worldlines to promote tunneling [66].

Originally, SQA only increases the coupling $J_\perp$ and keeps a constant low temperature T. We conducted our experiments with a schedule that decreases the temperature T while increasing $J_\perp$ to combine thermal jumping and quantum tunneling.
We again followed a similar scheme of Misevičius [56] , using sampled Δ for picking penalty extremes ($\lambda_1, \lambda_1' = 0.75, 0.05$):

$$J_{\perp_{\max}} = \left[\frac{\Delta_{\min}}{5} + \frac{3\Delta_{\text{avg}}}{4} + \frac{\Delta_{\max}}{20}\right] \cdot 2.5 \tag{4.15.a}$$

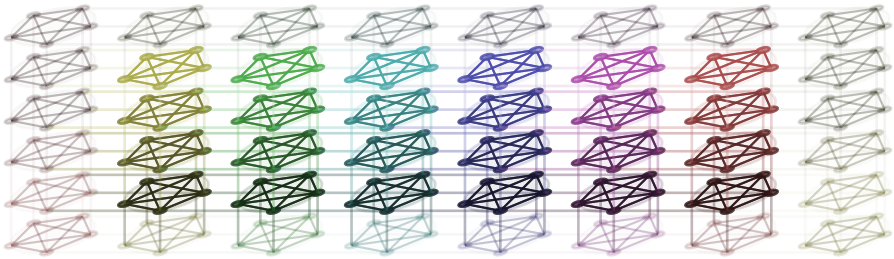
$$J_{\perp_{\min}} = \frac{\Delta_{\min}}{2}. \tag{4.15.b}$$

4.3.2 Beyond Single Time Dimension

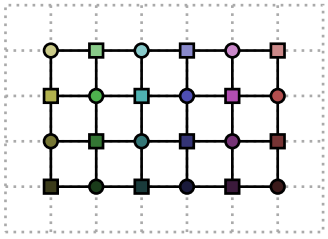
While classical SQA is limited to a one-dimensional ring of Trotter slices, we extend the topology to a $(d+2)$-dimensional grid (a torus, Figure 4.7), coupling each replica $\boldsymbol{\pi}^{[k,\ell]}$ to its four spatial neighbors:

$$\mathcal{H}_{d+2} = \sum_{k=1}^{P_1}\sum_{\ell=1}^{P_2}\left[z\left(\boldsymbol{\pi}^{[k,\ell]}\right) + J_\perp g\left(\boldsymbol{\pi}^{[k,\ell]}, \boldsymbol{\pi}^{[k+1,\ell]}\right) + J_\top g\left(\boldsymbol{\pi}^{[k,\ell]}, \boldsymbol{\pi}^{[k,\ell+1]}\right)\right], \tag{4.16}$$

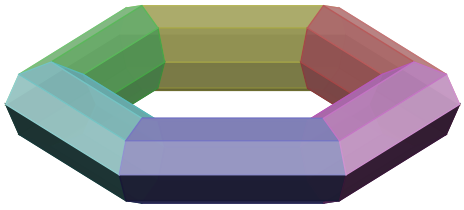
where $J_\perp$ and $J_\top$ control coupling strength in each dimension. In practice, we set $J_\perp = J_\top$ for simplicity.



(a) Slices



(b) Grid



(c) Torus

Figure 4.7: Extension of SQA to a two-dimensional Trotter space: **(a)** the 2D Trotter space, where each slice $\boldsymbol{\pi}[k,\ell]$ interacts with its four spatial neighbors; **(b)** the $P_1 \times P_2$ checkerboard coloring (squares/circles) identifies two independent sets that can be updated in parallel; **(c)** due to the periodic boundary conditions, the grid is forming a torus.

The additional dimension may provide more pathways for quantum tunneling, potentially accelerating escape from local minima.

However, we note that the rigorous connection to quantum mechanics via path-integral formulation becomes weaker in this setting and the computational cost increases significantly with the added dimension.

The checkerboard pattern in Figure 4.7b directly generalizes the even/odd coloring of the ring in two dimensions and allows us to apply the same GPU parallelization of the 1D ring to the $P_1 \times P_2$ grid.

4.4 Momentum Annealing

Momentum Annealing (MA) asks: what if two copies of the permutation, a left and a right, were optimized alternately, while penalizing disagreement between them?
The key insight is that because the left copy is held fixed while the right is updated, and vice versa, every position on the active side can be updated independently and without conflict. This decoupling is precisely what enables GPU parallelism for dense QAP instances.
Intuitively, the two copies behave like two mirrors facing each other, each reflecting the other's current state, and gradually converging toward a consistent image. The trade-off is straightforward: the search space multiplies, but the gain is massive parallelism.

The original formulation of MA was proposed by Okuyama et al. [67] for Ising models. MA facilitates massive parallelism by remapping dense interactions into a bipartite graph [67]. Formally, the bipartite structure maps each original variable to a left and a right copy, with cross-interactions capturing the original objective and a penalty term forcing eventual agreement, directly mirroring the two-mirror intuition above. The system is duplicated into left (X_L) and right (X_R) configurations, minimizing:

$$\mathcal{H}_{\text{MA}} = \boldsymbol{x_L}^\top \boldsymbol{J} \boldsymbol{x_R} + \boldsymbol{h}^\top (\boldsymbol{x_L} + \boldsymbol{x_R}) - \lambda \boldsymbol{x_L}^\top \boldsymbol{x_R}. \tag{4.17}$$

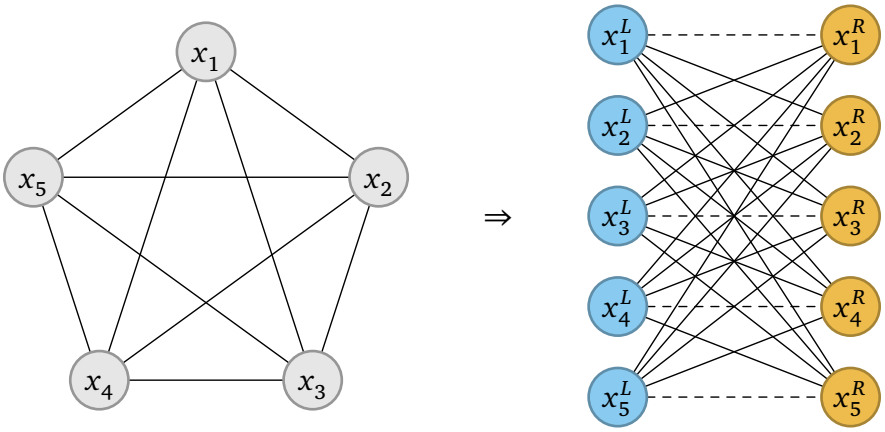


Figure 4.8: Bipartite remapping in Momentum Annealing: the original n variables are duplicated into left (x_L) and right (x_R) copies. Solid edges represent the original interaction structure between opposite sides; dashed edges represent the penalty coupling λ that forces the two sides to converge to the same solution over time. Spins within the same side do not interact directly (same color), enabling all n left (or right) variables to be updated simultaneously while keeping the other side fixed.

The penalty term $\lambda \boldsymbol{x}_L{}^\top \boldsymbol{x}_R$ forces both sides to eventually converge to the same configuration by increasing λ over time.

Figure 4.8 shows this structure. Doubling the solution space to 2^{2n} is the price paid, but for dense QAP instances where all-to-all interactions otherwise force sequential updates, this bipartite decomposition is what makes GPU-scale parallelism feasible.

The algorithm alternates between optimizing the left and right configurations.

The original scheme of MA for picking the penalties does not transfer to QAPs. Instead we again followed the scheme of Misevičius [56] based on Δ_{avg}.

4.4.1 Application to the QAP

We adapt this bipartite framework to integer permutations, similar to the Independent Quadratic Assignment Problem (IQAP) formulation:

$$\min f(\Phi, \pi) = \sum_{i}^{m} \sum_{j}^{n} A_{ij} B_{\pi_i \varphi_j}. \tag{4.18}$$

Yang et al. [68] also use the alternating algorithm where they repeatedly fix one side (π or Φ) and optimize the other (popular in bilinear programming).

Whereas the IQAP does not require π and φ to be equal, the QAP does; so we solve for:

$$\mathcal{H}_{\text{MA}} = \operatorname{tr}\left(A X_L B X_R^\top\right) + \lambda\left(n - \operatorname{tr}\left(X_L X_R^\top\right)\right) \tag{4.19.a}$$

$$= \underbrace{\sum_{i} \sum_{j} A_{ij} B_{\boldsymbol{\pi}_i^L \boldsymbol{\pi}_j^R}}_{z_{\text{MA}}(\boldsymbol{\pi}^L, \boldsymbol{\pi}^R)} + \lambda g\left(\boldsymbol{\pi}^L, \boldsymbol{\pi}^R\right), \tag{4.19.b}$$

where we enforce $X_L = X_R$ at the end by increasing the coupling λ.

2-Opt for Momentum Annealing

The calculation of Δz (Equation 4.1) changes slightly when updating only one side of the permutations.

Because MA decouples the left and right permutations, a local swap in $\boldsymbol{\pi}^L$ only affects the left side of the interaction. Therefore, the standard QAP Δz equation must be split.

When swapping indices i and j in the left permutation:

$$\Delta z_L\left(\boldsymbol{\pi}^L, \boldsymbol{\pi}^R, i, j\right) = \sum_{k=1, k\neq i,j}^{n} \left(A_{jk} - A_{ik}\right)\left(B_{\boldsymbol{\pi}_i^L \boldsymbol{\pi}_k^R} - B_{\boldsymbol{\pi}_j^L \boldsymbol{\pi}_k^R}\right) \tag{4.20.a}$$

$$+\left(A_{ji} - A_{ii}\right)\left(B_{\boldsymbol{\pi}_i^L \boldsymbol{\pi}_i^R} - B_{\boldsymbol{\pi}_j^L \boldsymbol{\pi}_i^R}\right) \tag{4.20.b}$$

$$+\left(A_{jj} - A_{ij}\right)\left(B_{\boldsymbol{\pi}_i^L \boldsymbol{\pi}_j^R} - B_{\boldsymbol{\pi}_j^L \boldsymbol{\pi}_j^R}\right). \tag{4.20.c}$$

The right-side evaluation mirrors this symmetrically:

$$\Delta z_R\left(\boldsymbol{\pi}^L, \boldsymbol{\pi}^R, i, j\right) = \sum_{k=1, k\neq i,j}^{n} \left(A_{kj} - A_{ki}\right)\left(B_{\boldsymbol{\pi}_k^L \boldsymbol{\pi}_i^R} - B_{\boldsymbol{\pi}_k^L \boldsymbol{\pi}_j^R}\right) \tag{4.21.a}$$

$$+\left(A_{ij} - A_{ii}\right)\left(B_{\boldsymbol{\pi}_i^L \boldsymbol{\pi}_i^R} - B_{\boldsymbol{\pi}_i^L \boldsymbol{\pi}_j^R}\right) \tag{4.21.b}$$

$$+\left(A_{ji} - A_{jj}\right)\left(B_{\boldsymbol{\pi}_j^L \boldsymbol{\pi}_j^R} - B_{\boldsymbol{\pi}_j^L \boldsymbol{\pi}_i^R}\right) \tag{4.21.c}$$

The total Hamiltonian difference includes the penalty shift:

$$\Delta H_{\mathrm{MA,L}} = \Delta z_L\left(\boldsymbol{\pi}^L, \boldsymbol{\pi}^R, i, j\right) + \lambda \Delta g\left(\boldsymbol{\pi}^L, \boldsymbol{\pi}^R, i, j\right) \tag{4.22.a}$$

$$\Delta H_{\mathrm{MA,R}} = \Delta z_R\left(\boldsymbol{\pi}^L, \boldsymbol{\pi}^R, i, j\right) + \lambda \Delta g\left(\boldsymbol{\pi}^R, \boldsymbol{\pi}^L, i, j\right), \tag{4.22.b}$$

where

$$\Delta g\left(\boldsymbol{\pi}^a, \boldsymbol{\pi}^b, i, j\right) = \overbrace{\left[2 - \delta\left(\boldsymbol{\pi}_i^a, \boldsymbol{\pi}_j^b\right) - \delta\left(\boldsymbol{\pi}_j^a, \boldsymbol{\pi}_i^b\right)\right]}^{\text{after swap}} \tag{4.23.a}$$

$$- \underbrace{\left[2 - \delta\left(\boldsymbol{\pi}_j^a, \boldsymbol{\pi}_j^b\right) - \delta\left(\boldsymbol{\pi}_i^a, \boldsymbol{\pi}_i^b\right)\right]}_{\text{before swap}}. \tag{4.23.b}$$

Efficient Cost Difference Evaluation via Side-Conditioned Cache

Unlike standard QAP local search, MA fixes one side while updating the other. This makes the interaction term linear in the active permutation and allows an especially simple cache.

When sweeping the left side, the right permutation $\boldsymbol{\pi}^R$ is fixed, so we build the cache

$$\boldsymbol{H}^L = \boldsymbol{A}\boldsymbol{X}_R\boldsymbol{B}^\top = \boldsymbol{A}\boldsymbol{B}^\top_{*,\boldsymbol{\pi}_R},$$

whose entries directly yield Δz_L in $\mathcal{O}(1)$. Symmetrically, right sweeps use

$$\boldsymbol{H}^R = \boldsymbol{A}^\top\boldsymbol{X}_L\boldsymbol{B} = \boldsymbol{A}^\top\boldsymbol{B}_{\boldsymbol{\pi}_L,*}.$$

Then the cost of swapping i and j in $\boldsymbol{\pi}^L$ (or symmetrically $\boldsymbol{\pi}^R$) reduce to:

$$\Delta z_{L(\boldsymbol{\pi}^L,i,j)} = \boldsymbol{H}^L_{i,\boldsymbol{\pi}_j^L} + \boldsymbol{H}^L_{j,\boldsymbol{\pi}_i^L} - \boldsymbol{H}^L_{i,\boldsymbol{\pi}_i^L} - \boldsymbol{H}^L_{j,\boldsymbol{\pi}_j^L}, \tag{4.24.a}$$

$$\Delta z_{R(\boldsymbol{\pi}^R,i,j)} = \boldsymbol{H}^R_{i,\boldsymbol{\pi}_j^R} + \boldsymbol{H}^R_{j,\boldsymbol{\pi}_i^R} - \boldsymbol{H}^R_{i,\boldsymbol{\pi}_i^R} - \boldsymbol{H}^R_{j,\boldsymbol{\pi}_j^R}. \tag{4.24.b}$$

Thus, once the appropriate cache has been built in $\mathcal{O}\left(n^2\right)$, local MA swap evaluations reduce from $\mathcal{O}(n)$ to $\mathcal{O}(1)$. Crucially, during a left sweep $\boldsymbol{H}_L$ remains valid because it depends only on the fixed right permutation $\boldsymbol{\pi}^R$. Likewise, $\boldsymbol{H}_R$ remains valid throughout a right sweep. We therefore rebuild the cache only when switching the active side.

Sweep Strategies

By default MA performs local sweeps, where we propose 2-opt swaps within one side while keeping the other side fixed [67].

For binary spin systems this is straightforward as flipping all spins from one side contributes a valid move. However, for permutations, we must ensure that the proposed swaps maintain the feasibility of the solution (i.e., no duplicate assignments).

To systematically propose vertex-disjoint swaps, we employ a round-robin tournament scheme.

Figure 4.9 shows the round-robin structure. Completing all $n-1$ rounds constitutes a full neighborhood traversal (equivalent to Connolly's sequential sweep) but with each individual round executed in parallel. This is the key efficiency gain over naive sequential 2-opt.

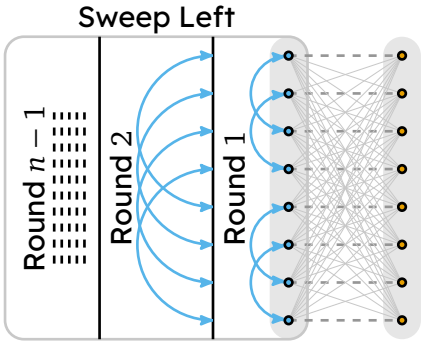
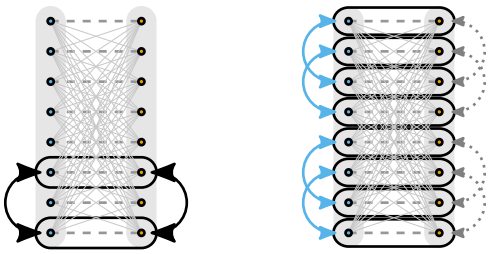


Figure 4.9: Round-robin tournament scheme for the MA local sweep: in each of the $n-1$ rounds, the n left-side indices are partitioned into $\binom{n}{2}$ disjoint pairs (paired differently per round). Because the $\left\lfloor \frac{n}{2} \right\rfloor$ pairs do not share indices, all proposals within one round are conflict-free and can be evaluated and committed in parallel. The right side remains fixed throughout, so no coupling conflicts arise across rounds.

Inspired by SQA, we introduce two new types of sweeps for MA. Figure 4.10 shows both. The design tension they resolve is between exploration and speed by parallelism. For the pseudo-global sweeps we use a synchronous frozen-round scheme: all proposals of one round-robin match are evaluated against the same pre-round state, accept/reject decisions are made, and only then are the accepted one-sided swaps applied in parallel.



(a) Global **(b)** Pseudo-Global

Figure 4.10: Two novel global sweep types introduced for MA: **(a)** a global sweep proposes the same 2-opt swap (i, j) simultaneously on both the left and right permutations, mirroring the worldline-flipping global move of SQA (at the cost of invalidating the cache); **(b)** a pseudo-global sweep proposes a disjoint swap on one side only but evaluates acceptance using the full two-sided Hamiltonian, approximating a global move while limiting cache invalidation to side-switches.

Dropout: Because global moves invalidate the cache, we use them only occasionally to encourage exploration.
Inspired by dropout in neural networks, Okuyama et al. [67] introduced a mechanism that occasionally sets $\lambda = 0$ with probability 5%. This temporarily decouples the two sides, reduces coupling-induced inertia, and allows them to evolve more independently, helping the search escape local minima. We found the same mechanism effective for the QAP as well: local sweeps can exploit the cache aggressively, while occasional uncoupled updates preserve the exploratory power of global moves.

4.5 Parallel Tempering

Parallel Tempering (PT), also known as Replica Exchange Monte Carlo, was introduced by Hukushima and Nemoto [69] in 1996 for Ising problems. It addresses a fundamental limitation of SA: once the temperature has cooled significantly and the system becomes trapped in a local minimum, thermal fluctuations are often insufficient to escape.

The method simulates R independent replicas of the system simultaneously, each at a different temperature:

$$T_1 > T_2 > \ldots > T_R \quad \Leftrightarrow \quad \beta_1 < \beta_2 < \ldots < \beta_R. \tag{4.25}$$

Periodically, replicas at adjacent temperatures attempt to swap configurations, accepted with probability:

$$p_{\text{swap}} = \min\left(1, e^{\Delta\beta\Delta E}\right), \tag{4.26}$$

where $\Delta E = E_i - E_j$ is the energy difference and $\Delta\beta = \beta_i - \beta_j$ is the difference between their inverse temperatures.

Figure 4.11 illustrates this. The key implication for the QAP is that PT converts SA's main weakness (irreversible freezing) into a recoverable state: a cold replica that has stagnated can temporarily move up the ladder, roam freely, and return for refinement without any manual restarts.

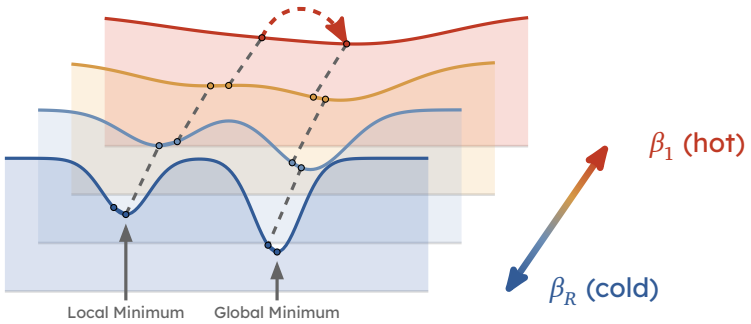


Figure 4.11: Escape mechanism of Parallel Tempering: replicas at different inverse temperatures $\beta_1 < \ldots < \beta_R$ explore the energy landscape simultaneously. A hot replica (small β) can freely explore broad regions including the basin of the global minimum, while a cold replica (large β) refines solutions locally but risks trapping. Periodic replica exchanges allow a low-energy configuration found at high temperature to percolate to cold replicas for exploitation, and a trapped cold replica to escape by temporarily moving to a higher temperature. (Inspired by [70].)

4.5.1 Temperature Ladder

The effectiveness of PT depends critically on the temperature ladder design. If temperatures are spaced too widely, neighboring replicas have little energy distribution overlap, resulting in very low swap acceptance rates. If spaced too closely, computational resources are wasted on nearly redundant temperatures.

A simple baseline approach is to use geometrically spaced temperatures: $T_{i+1} = \alpha T_i$ [71].

It is beneficial to place additional temperatures near the system's critical temperature (such as phase-transitions, e.g., ferromagnetic to paramagnetic), where equilibration slows down dramatically.

However, identifying the critical temperature a priori is often problem-dependent, nontrivial and requires huge computational cost [72].

Adaptive PT introduces temperature ladders based on energy overlaps which try to equalize swap probability across replicas to avoid bottlenecks. The algorithm can be found in the appendix of [49].

4.5.2 Cluster Moves

Beyond temperature-based swaps, PT can be enhanced with Cluster Moves [49, 73]. In this variant, each temperature maintains at least two replicas, and the largest cluster of correlated spins is swapped as seen in Figure 4.12.

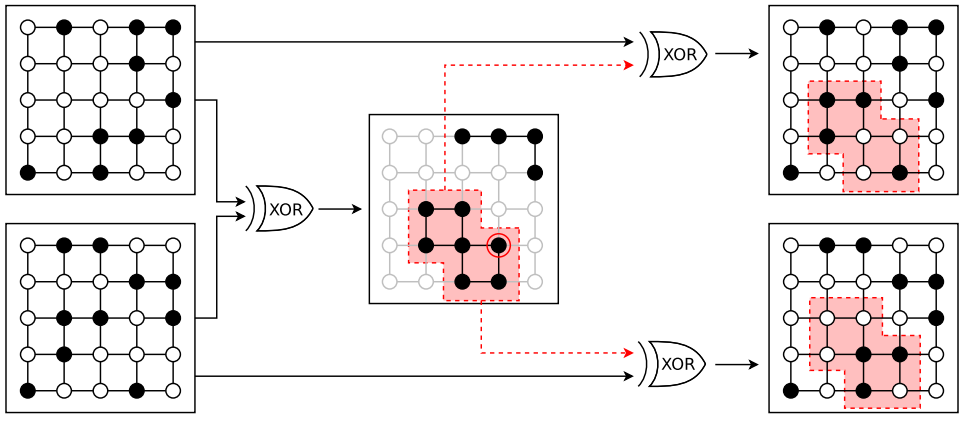


Figure 4.12: Isoenergetic cluster move on a 2D Ising lattice: a spatially connected cluster of spins (highlighted) is identified and simultaneously flipped between two replicas at the same temperature. Because the energy contribution of the cluster depends only on its boundary interactions, this exchange is isoenergetic ($\Delta E = 0$) and therefore always accepted. For the dense QAP, such moves cannot be constructed directly, thus motivating the cyclic and path-relinking adaptations described below. (Figure from [74].)

The always-accepted nature of isoenergetic moves is their defining advantage: large-scale coordinated updates bypass the Metropolis barrier entirely [75]. This cluster-based recombination can dramatically accelerate sampling by allowing large-scale coordinated updates across many spins simultaneously. The remaining question is whether analogous moves exist for fully-connected permutation spaces.

4.5.3 Application of Cluster Moves to the QAP

Identifying meaningful clusters becomes nontrivial for dense, fully-connected interaction graphs like those arising in the QAP. Furthermore, permutation-based representations lack the natural definition of recombination present in spin systems.

In sparse lattice spin systems, isoenergetic cluster moves exist because the energy contribution of a cluster depends only on its boundary interactions with the rest of the system. Flipping an entire cluster therefore leaves the total energy unchanged. For the dense QAPs, however, the cost function $z(\boldsymbol{\pi}) = \sum \boldsymbol{A}_{ij}\boldsymbol{B}_{\boldsymbol{\pi}_i\boldsymbol{\pi}_j}$ is fully connected: reassigning any single facility affects its interaction with all $n-1$ other facilities simultaneously. We were unable to construct isoenergetic cluster moves for dense QAP instances, and conjecture none exist in general.

To the best of our knowledge, adapting crossover operators (such as cyclic moves and path relinking) as cluster moves within PT has not been previously explored for the QAP.

Cyclic Moves: To adapt cluster moves to permutations, we use cyclic moves. They naturally exchange large groups of elements between solutions and are common crossover permutation operators in genetic algorithms [76].

A 2-opt swap exchanges two elements. A cyclic move exchanges a whole chain of elements between two permutations simultaneously and still guarantees a valid permutation. This is the permutation analogue of flipping an entire cluster in a spin system.

Given two permutations $\boldsymbol{\pi}_1$ and $\boldsymbol{\pi}_2$, a cycle is formed by tracing the elements at the same indices across both permutations until the sequence returns to the starting element. For instance, consider the following two permutations:

$$\boldsymbol{\pi}_1 = \begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{pmatrix}$$
$$\boldsymbol{\pi}_2 = \begin{pmatrix} 5 & 9 & 1 & 8 & 2 & 6 & 4 & 7 & 3 \end{pmatrix}$$

If we start at the first index, $\boldsymbol{\pi}_1$ has 1 and $\boldsymbol{\pi}_2$ has 5. We then look for 5 in $\boldsymbol{\pi}_1$ (which is at the fifth index), where $\boldsymbol{\pi}_2$ has 2. Following this chain yields the sequence of elements $1 \to 5 \to 2 \to 9 \to 3 \to 1$, forming a cycle.
All cycles between $\boldsymbol{\pi}_1$ and $\boldsymbol{\pi}_2$ are given by $C = \{(1\ 5\ 2\ 9\ 3), (4\ 8\ 7), (6)\}$.

By exchanging only the elements of this cycle between $\boldsymbol{\pi}_1$ and $\boldsymbol{\pi}_2$, we can perform a large-scale, coordinated move that guarantees the resulting sequences remain valid permutations without any missing or duplicated assignments.

Applying a cyclic move $c \in C$ is asymmetric. For $\boldsymbol{\pi}_1$, we directly apply c. For $\boldsymbol{\pi}_2$, the cycle elements reside at different indices, requiring the inverted cycle c'.

The cluster move evaluates each $c \in C$:

$$\Delta E_1 = \Delta z(\boldsymbol{\pi}_1, c) \tag{4.27.a}$$
$$\Delta E_2 = \Delta z(\boldsymbol{\pi}_2, c') \qquad c' = (c_1\ \ c_n\ \ c_{n-1}\ \ \cdots\ \ c_2). \tag{4.27.b}$$

The joint move is accepted with probability $e^{-\beta_i(\Delta E_1 + \Delta E_2)}$.
In practice, we evaluate all cycles in parallel and apply only the largest accepted move.

Path Relinking Moves: Path Relinking provides an alternative method to recombine permutations [77]. The underlying expectation is that systematically exploring the trajectory between two high-quality permutations will uncover intermediate states that successfully combine the best elements of both. In this process, one permutation acts as the *initial* solution π^I and the other as the *guiding* solution π^G. Figure 4.13 shows one such path. Different strategies exist for selecting the best move, such as best-improvement (greedy) or first-improvement, but we use the former for simplicity.

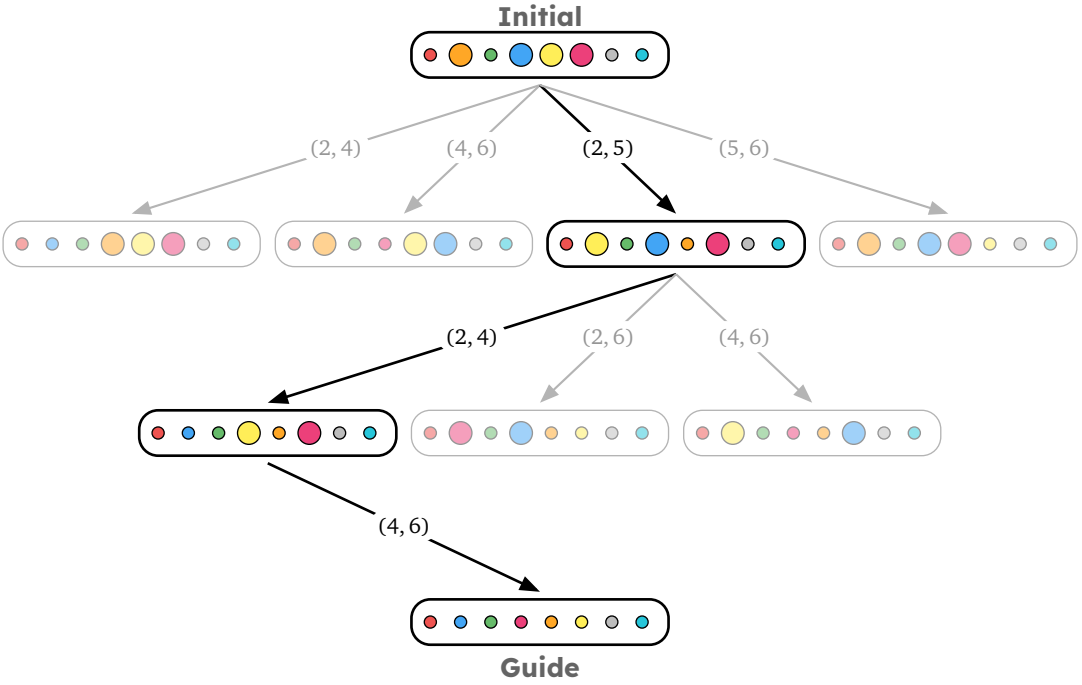


Figure 4.13: Path relinking between an initial permutation and a guide permutation: large circles mark positions where the two permutations disagree. At each step, the 2-opt move that reduces the Hamming distance from π_I (top) to π_G (bottom) by the most is applied. The intermediate solutions along this path are evaluated, and the best one is returned as the proposed new state. Truncated relinking (stopping before reaching π_G) prevents replica collapse by ensuring proposed states remain distinct. (Adapted from [78].)

To execute a joint cluster move between two replicas, we perform relinking in both directions: first using π_1 as the initial and π_2 as the guide, and then vice versa. The best intermediate permutations found along each path become the new proposed states.

Crucially, completing the full relinking path often results in the proposed state simply becoming the guide solution. Accepting these moves would rapidly propagate identical solutions across replicas, leading to a catastrophic loss of population diversity (replica collapse). To prevent this, we use *truncated path relinking*, terminating the path generation prematurely (e.g., after closing 75% of the initial Hamming distance) to ensure the proposed states remain distinct.

We again accept the joint move with probability $e^{-\beta_i(\Delta E_1+\Delta E_2)}$. Note that this move is always accepted when returning only the best intermediate states (including the initial state).

4.5.4 Combining Heuristics

In the following, we combine the above methods into two hybrid algorithms: PT-SQA and PT2D. This allows us to leverage PT for SQA and MA.

PT-SQA: Implicit Cluster Moves by Tunneling

We propose a novel hybrid method that combines SQA with PT through a jointly structured replica ladder. Each replica r is characterized by a parameter pair $(T_r, J_r^{\perp})$, where the temperature T_r decreases and the inter-slice coupling $J^{\perp}$ increases across the ladder. This construction induces a smooth transition from a thermally dominated regime at the top of the ladder (characterized by high temperature and weak inter-slice coupling) to a quantum-dominated regime at the bottom, where low temperature and strong coupling prevail.

Within each replica, the SQA dynamics over Trotter slices admit an interpretation as implicit cluster updates. At high temperatures, the weak inter-slice coupling allows individual slices to evolve largely independently, maintaining high configurational diversity. As the temperature decreases and the coupling strengthens, slices are increasingly forced into alignment, enabling tunneling-like collective updates that span the entire worldline.

To improve mixing across the ladder, we introduce a slice-wise replica exchange mechanism. Rather than swapping entire replicas as in standard PT, individual Trotter slices are exchanged between neighboring replicas. This enables localized transfer of configurations across the ladder. The probability of swapping slice k of replica i with slice ℓ of replica j is given by:

$$P_{\text{slice}_{ik} \leftrightarrow \text{slice}_{j\ell}} = \min\Big(1, e^{(\beta_j - \beta_i)(E_{j\ell} - E_{ik})}\Big), \tag{4.28}$$

where replica i is hotter than replica j, i.e. $\beta_i < \beta_j$.

Because individual Trotter slices are coupled to their neighbors within a replica, a slice-wise exchange modifies the local coupling structure by breaking and re-establishing inter-slice interactions at the boundary of the exchanged slice. To maintain a consistent energy accounting for the full system, the replica energies must be updated to reflect both the direct slice energy change and the modified coupling contributions:

$$E_i^{d+1} \stackrel{+}{=} E_{ik} - E_{j\ell} + \Big(g\big(\text{spins}_i\big) - g_i^{\text{old}}\Big) J_i^{\perp} \tag{4.29.a}$$

$$E_j^{d+1} \stackrel{+}{=} E_{j\ell} - E_{ik} + \Big(g\Big(\text{spins}_j\Big) - g_j^{\text{old}}\Big) J_j^{\perp} \tag{4.29.b}$$

These slice exchanges allow configurations originating in the high-temperature thermal regime to propagate toward low-temperature replicas, where the strong inter-slice coupling enables them to spread information via the implicit cluster structure. The combination of implicit cluster dynamics within SQA and fine-grained slice-wise replica exchange provides an efficient mechanism for exploring complex energy landscapes. We refer to this hybrid as *PT-SQA*.

Together, these components define the PT-SQA architecture: a replica ladder that simultaneously spans a thermal and a quantum regime, coupled through slice-wise exchanges. The full structure is depicted in Figure 4.14.

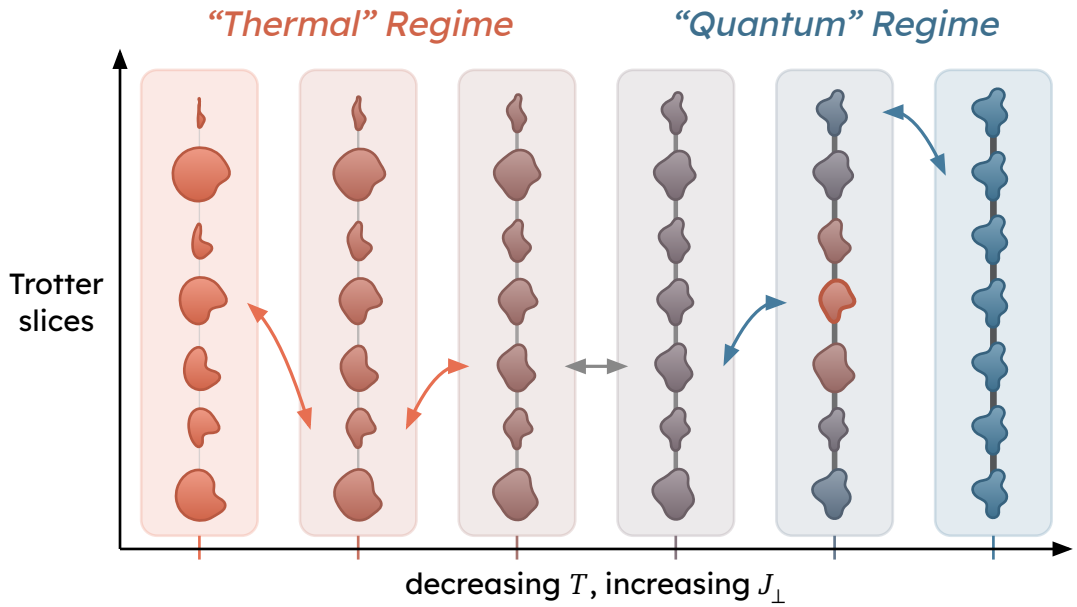


Figure 4.14: Architecture of the PT-SQA hybrid: replicas are arranged along a ladder with jointly decreasing temperature T_r and increasing inter-slice coupling $J_{\perp_r}$. Hot replicas (left) operate in a thermally dominated regime with weak coupling, allowing Trotter slices to explore independently. Cold replicas (right) operate in a quantum-dominated regime where strong coupling forces slices into alignment via tunneling-like dynamics. Similar shapes within a replica hint at growing slice consensus. Individual Trotter slices are exchanged between neighboring replicas (rather than swapping full multi-slice states), enabling fine-grained propagation of configurations across the thermal-to-quantum transition.

4.5.5 Parallel Tempering in Two Dimensions

Parallel Tempering in Two Dimensions (PT2D) extends the one-dimensional temperature ladder of standard PT to a two-dimensional grid, adding a second axis for the penalty parameter λ [79, 80]. Each replica is now characterized by a pair (T_r, λ_r), and replica exchanges are proposed along both dimensions independently.

Figure 4.15 shows the replica grid. A single (T, λ) annealing path is forced to cross the effective optimization region in a highly schedule-dependent way; PT2D replaces this with a population that can approach the region from any direction, making the method substantially more robust, at the cost of a multiplicatively larger replica grid.
We note that PT2D is computationally demanding, as it requires maintaining a two-dimensional replica grid.

As a precursor to PT2D, we also experimented with self-adaptive penalties [80], which adjust λ dynamically based on the current Hamming distance. However, PT2D provides a more principled alternative by treating λ as an independent thermodynamic axis.

For future work, it could also be interesting to consider this framework for multi-objective optimization, where each objective function is associated with its own penalty axis.

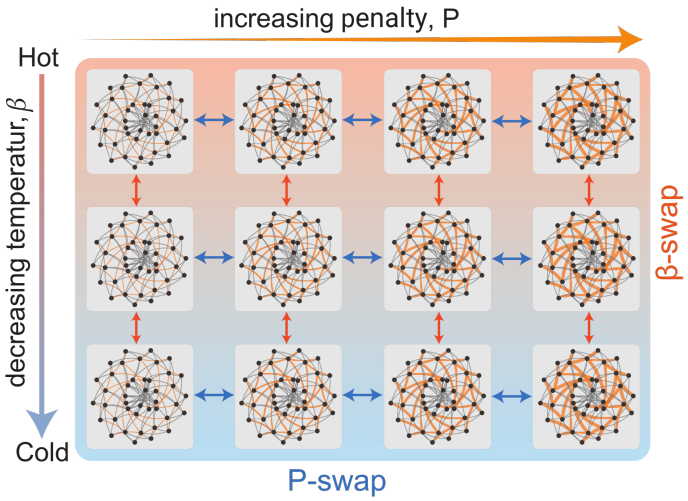


Figure 4.15: Two-dimensional Parallel Tempering: replicas are arranged on a grid parameterized by temperature T (horizontal axis) and penalty λ (vertical axis). Replica exchanges are proposed independently along both axes, decoupling the two schedules and allowing configurations to explore the joint (T, λ) landscape. This resolves the narrow symbiosis regime in which temperature and penalty must simultaneously be in an effective range, which is a key limitation of single-schedule penalty-based methods. (Figure from [81].)

4.6 Population Annealing

Population Annealing (PA) is survival of the fittest without crossover. Running 100 replicas in parallel, at each temperature step the method duplicates low-energy replicas proportionally and eliminates high-energy ones. Diversity comes from subsequent Monte Carlo steps, not from combining solutions. The physics framing is thin; the algorithmic idea is elegant.

PA was introduced by Hukushima [82] in 2003 as an alternative to PT. Like PT, PA simulates multiple replicas to overcome trapping in local minima, but it does so differently: instead of swapping configurations between fixed temperatures, PA evolves R replicas. In contrast to PT, which maintains replicas at multiple temperatures simultaneously, PA advances all replicas together through the temperature schedule.

At each temperature step, replicas are reweighted based on their fitness [83]:

$$r_j = \frac{e^{-(\beta_i - \beta_{i-1})E_j}}{Q_i}, \qquad Q_i = \frac{1}{R_{i-1}} \sum_j^{R_{i-1}} e^{-(\beta_i - \beta_{i-1})E_j}. \tag{4.30}$$

A resampling step duplicates low-energy replicas proportionally to r_j and eliminates high-energy ones. Diversity comes strictly from subsequent Monte Carlo updates, not crossover. This avoids the difficulty of defining viable recombinations for integer permutations.

Resampling can be performed with either constant or variable population size R_i. Constant population size is especially attractive for GPU implementations: it maintains a fixed memory footprint and requires no dynamic allocation during resampling.

While many resampling schemes exist, *systematic resampling* has been shown to perform best for constant population sizes [84]. One round of this scheme is shown in Figure 4.16. Compared to independent multinomial sampling, this scheme maintains a fixed population size, making it well-suited for GPU implementations with constant memory footprint.

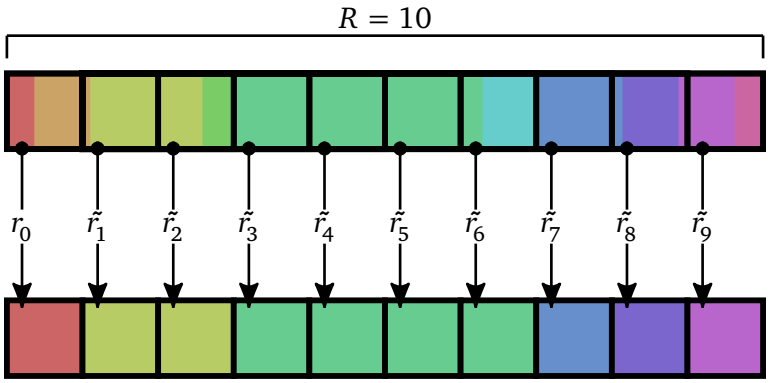


Figure 4.16: Systematic resampling for Population Annealing with 10 replicas: a single random offset $r_0 \sim U[0,1]$ determines the starting point, and R equally spaced pointers $\tilde{r}_i = r_0 + i$ are placed across the cumulative weight distribution. Each pointer selects one replica proportional to its Boltzmann weight, duplicating fit (low-energy) replicas and eliminating poor ones. (Adapted from [84].)

Temperature schedules should be chosen adaptively to prevent population collapse due to aggressive cooling. A constant temperature step $\Delta\beta$ is generally sub-optimal; instead, one should enforce a constant energy histogram overlap between successive temperatures, which can be estimated from the reweighting factors:

$$\alpha(\beta_{i-1}, \beta') = \frac{1}{R_{i-1}} \sum_{j}^{R_{i-1}} \min\left(1, \frac{R e^{-(\beta\prime - \beta_{i-1})E_j}}{R_{i-1} Q(\beta_{i-1}, \beta')}, \right) \tag{4.31}$$

Finding $\alpha \in [0.5, 0.9]$ is a good rule of thumb and can be done by, for example, bisection search [83].

The population size controls a fundamental tradeoff between statistical accuracy and computational cost. Larger populations better approximate the Boltzmann distribution but require more energy evaluations. PA exhibits low communication overhead, as replicas evolve independently between resampling steps and do not require pairwise exchanges.

5 Implementation

This chapter answers a practical question: how do you turn the algorithms from Chapter 4 into something that actually runs on a GPU at scale?

All methods were implemented from scratch in the Julia [85] programming language, resulting in the custom solver framework `ASSA.jl` [3] developed specifically for this thesis. GPU kernels are implemented using `CUDA.jl` [86, 87]. We adopted a CUDA-specific implementation to retain full access to low-level warp primitives, which are required by several solver kernels and were not yet supported by `KernelAbstractions.jl` [88] at the time of writing. The trade-off is code duplication between the GPU and CPU paths; a future migration to `KernelAbstractions.jl` would eliminate this duplication while preserving performance across backends.



Figure 5.1: `ASSA.jl`, the custom Julia solver framework developed for this thesis: **A**nnealing **S**uite **S**weet **A**nnealing.

We implemented the five solver families (SA, SQA, MA, PT, PA), all operating natively in permutation space.
The framework fulfills the purpose of enabling accurate and efficient annealing simulations and is designed around three principles: correctness (exact energy tracking), efficiency (GPU-parallel sweeps), and modularity (a composable solver hierarchy). These requirements were chosen because they correspond to the most important practical goals of the framework: producing correct results, running efficiently on available hardware, and remaining easy to extend and maintain.

In practice, this composability is only partially realized: the framework currently provides specialized implementations for `PT-SA`, `PT-SQA`, `PT(2D)-MA`, and `PA-SA`. Other combinations, such as `PT-PA` or `PA-MA`, are conceptually possible but not yet supported.

The framework's architecture is illustrated using `ParallelTempering` and `SimulatedQuantumAnnealer` as examples in Figure 5.2. The key design decision is that all solvers share one interface (`AbstractTrajectoryAnnealer`), but specialize only what differs. PT wraps any trajectory solver: swap SA for SQA and you get PT-SQA for little overhead.

An object-oriented approach would store each replica as a self-contained object containing its permutation, energy, and best-found solution. This is the Array-of-Structures

[3] Logo designed by prompting *fhgenie*.

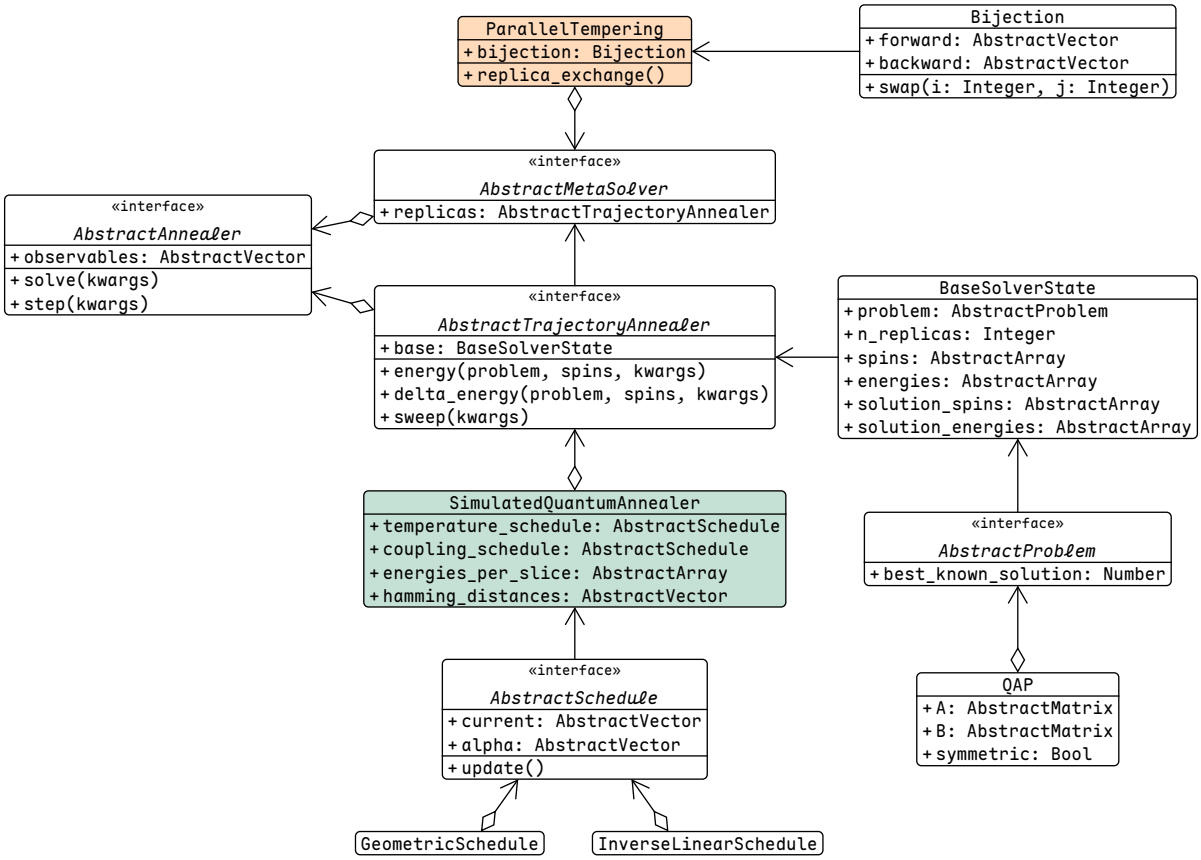


Figure 5.2: Class diagram of the ASSA.jl solver hierarchy: shown for ParallelTempering and SimulatedQuantumAnnealer as representative examples. AbstractTrajectoryAnnealer defines the shared interface for all single-chain solvers (SA, SQA, MA), each wrapping a BaseSolverState that stores permutations, energies, and best-found solutions in a Structure-of-Arrays (SoA) layout with dimensions $\left(n_{\text{spins}} \times n_{\text{trotter}} \times n_{\text{replicas}}\right)$. ParallelTempering implements AbstractMetaSolver and wraps a single AbstractTrajectoryAnnealer holding all replicas in one batched state; logical replica identities are tracked via a Bijection that swaps temperature indices rather than permutation arrays, eliminating memory bandwidth overhead. AbstractSchedule instances manage temperature and coupling updates independently per solver.

(AoS) layout. On a GPU this is problematic: when many threads access the same field across replicas, the memory addresses are non-contiguous, causing costly scattered reads.
ASSA.jl instead stores all permutations in one contiguous array and all energies in another, following a Structure-of-Arrays (SoA) layout. This makes indexing less intuitive but enables coalesced memory access, which is critical for GPU performance.

All trajectory solvers implement solve! kernels that fuse sweep! and schedule updates, which reduces kernel launch and synchronization overhead. Observables are evaluated periodically on the CPU and written to TensorBoard for runtime monitoring, such as energy, Hamming distance, and swap acceptance rates.

The control flow of solve! is illustrated in Figure 5.3, which shows the main execution path and specialized step! implementations are utilized as shown in Figure 5.4.

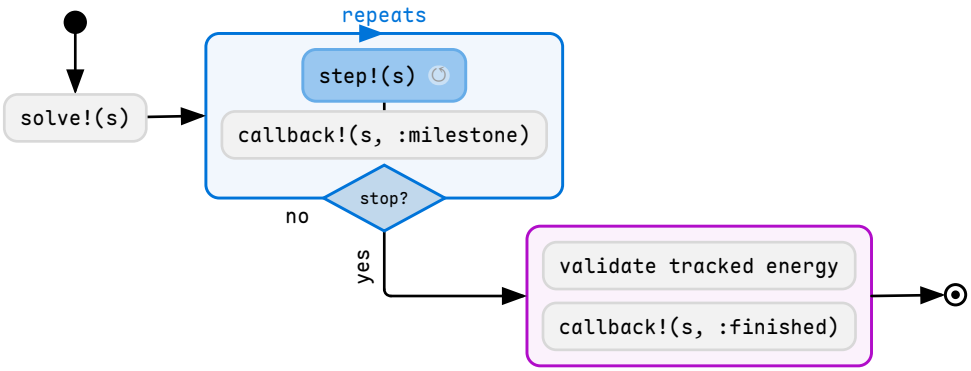


Figure 5.3: Generic control flow of the `solve!` entry point shared by all `ASSA.jl` solvers. Each call to `step!` advances the solver by one algorithmic step (sweep plus schedule update); a milestone callback periodically evaluates observables on the CPU and writes them to `TensorBoard`. The loop terminates either when the step budget is exhausted or when an early-exit condition is met (e.g., the best-known solution energy is reached). After termination, a correctness check recomputes the full energy from scratch and validates it against the tracked value, providing a runtime safeguard against bookkeeping bugs.

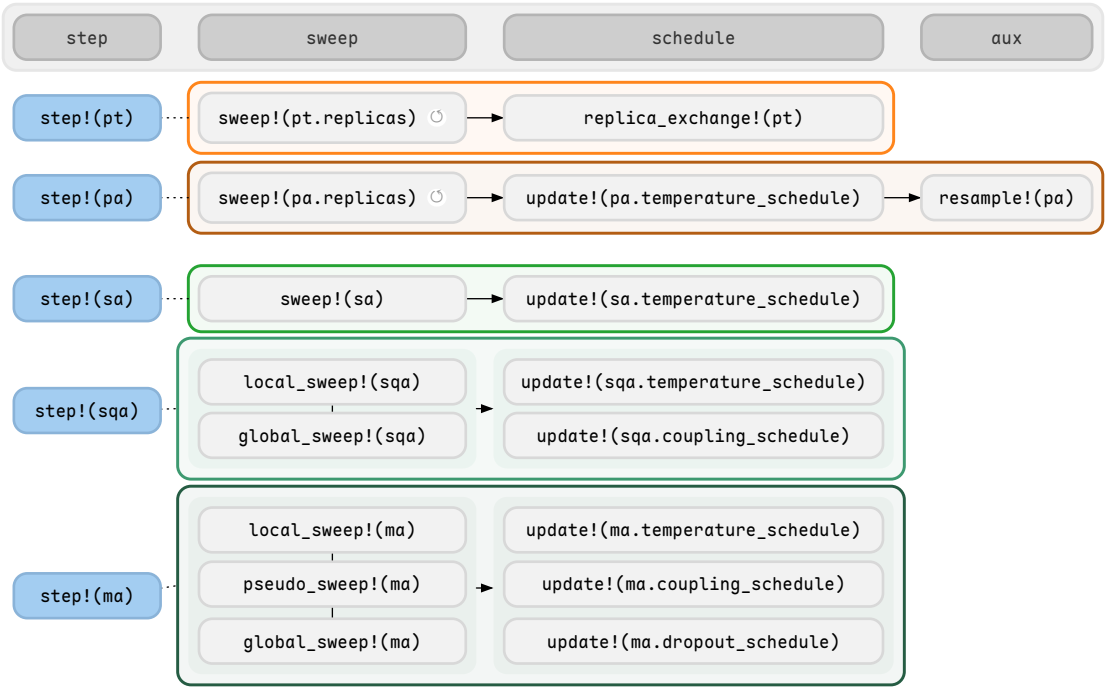


Figure 5.4: Specialized `step!` implementations for each solver family. Meta-solvers PT and PA delegate inner work to a wrapped trajectory solver: PT calls `sweep!` on all replicas and then performs `replica_exchange!`; PA additionally updates the temperature schedule and triggers `resample!` after each sweep batch. Trajectory solvers SA, SQA, and MA perform sweep and schedule updates. SQA alternates local sweeps (odd/even slice coloring) with optional global worldline-flipping sweeps and updates both temperature and coupling schedules. MA additionally manages a dropout schedule and supports three sweep variants (local, pseudo-global, global), with the ↺ symbol indicating that sweep phases may repeat within a single step.

For all implemented solvers, the correctness of the iterative energy bookkeeping is verified after every solve by comparing the tracked energy against a full evaluation from scratch. While this introduces additional computational overhead, it provides a robust

safeguard for correctness. This safeguard matters more than it may appear, since a single off-by-one error in the cache update would silently corrupt every result. The full recomputation afterwards acts as a mathematical proof-of-correctness at runtime: if the numbers disagree, there is a bug.

Every GPU kernel requires two decisions: how many blocks to launch (grid size) and how many threads per block. A high-level overview of the CUDA kernel layouts used across the framework is provided in Table 5.1.

Algorithm	Operation	Grid Size (Blocks)	Block Size (Threads)
SA	Standard Sweep	n_{replica}	32
SQA	Local Sweep	$n_{\text{replica}} \times \frac{n_{\text{trotter}}}{2}$	32
SQA	Global Sweep	n_{replica}	$32 \times n_{\text{trotter}}$
MA	All Sweeps	n_{replica}	$\frac{n_{\text{QAP}}}{2}$
PT	Standard Exchange	1	$\frac{n_{\text{replica}}}{2}$
PT-SQA	Slice Exchange	$\frac{n_{\text{replica}}}{2} \times n_{\text{trotter}}$	32
PT	Cyclic Cluster Moves	n_{cycles}	32

Table 5.1: Summary of CUDA grid and block configurations for the implemented solvers.

The following sections document the key implementation decisions for each solver.

5.1 Simulated Annealing

The GPU implementation of SA computes the energy difference ΔE for a proposed swap (i, j) in parallel across the warp, which distributes the $O(n)$ dot product via warp-level reductions.

While explicit shared memory optimizations for the problem matrices were considered, empirical testing showed little to no performance improvement. This aligns with observations from the `CUDA.jl` authors[4], but a more systematic profiling study is left for future work.

The kernel follows the Connolly neighborhood traversal scheme, sequentially iterating over all pairs (i, j) with $i < j$ in a deterministic order [55]. Only thread 1 evaluates the Metropolis acceptance criterion and commits the swap, ensuring race-free updates to the shared permutation array and energy scalar.

The implementation supports the Connolly adaptive cooling schedule as a special case. When active, thread 1 additionally evaluates a schedule-level acceptance criterion, increments the failure counter on rejection, and resets it upon finding a new best solution. This logic is entirely absent from the simpler `sweep!` variant, which is used as the inner kernel for PT and omits the schedule update entirely.

Solution tracking is performed inside the kernel: whenever an accepted swap yields a new best energy E_{sol}, thread 1 updates E_{sol} and all 32 threads cooperatively copy the

[4] The underlying LLVM NVPTX compiler often optimizes global memory reads and utilizes the hardware caches effectively on its own *(online discussion)*

current permutation into π_{sol} in a strided loop. An early-exit mechanism allows the kernel to terminate immediately once the best-known solution energy is reached, avoiding unnecessary computation in the remaining steps.

Additionally, the `accept_only_one_j_per_i` flag provides an optional variant of the sweep for more efficiency: when enabled, the inner loop breaks after the first accepted swap for each i, advancing to the next i immediately. This reduces the number of evaluations per sweep at the cost of a less thorough neighborhood exploration. This matches the behavior of the cached variant more, but accepts only the first Metropolis-accepted swap rather than a random accepted swap.

5.2 Simulated Annealing Cached

Our implementation of the simulated annealing algorithm utilizes a cache matrix H (following [28] and [59]), ensuring that the energy difference ΔE can be computed in $O(1)$ time. Updating the cache requires $O\left(n^2\right)$ operations.
This implementation removes the limiting assumption that the diagonals of the distance and flow matrices are always zero.

We followed the GPU-hints given in [28] and show pseudocode in Algorithm 5.1, which evaluates all possible swaps for index i in parallel but only accepts at most one swap.

Pseudocode of Cached SA – inner loop

```
for i in 1:n
  ΔEs = [Δz(qap, x, i, j, H) for j in 1:qap.n] # do in parallel
  possible_swaps = rand(qap.n) .≤ exp.(-ΔEs ./ temp_schedule(step))
  swap_flags = 1:n .* possible_swaps           # j if accepted, otherwise 0
  j = pick_random_nonzero(swap_flags)
  if !isnan(j)
    x[[i,j]] .= x[[j,i]]                        # perform swap
    E += ΔEs[j]
    update_cache!(H, qap, x, i, j)              # do in parallel
  end
end
```

Algorithm 5.1: Cached SA inner loop for the QAP.

Furthermore, when the problem size n exceeds the maximum block size, individual threads are assigned to evaluate multiple j indices. To uniformly and randomly select one accepted swap from all valid candidates, each thread employs local reservoir sampling in Algorithm A.1. We implemented the naive parallel prefix sum [89] to select a global winner, with probabilities proportional to the number of valid candidates discovered by each thread.

The challenge is: after evaluating all n possible swaps in parallel, we want to pick exactly one accepted swap uniformly at random. On a CPU, this is trivial: filter, then sample. On a GPU with fixed block sizes, we can't know how many candidates exist ahead of time. The solution: each thread independently runs reservoir sampling on its

local candidates, then a parallel prefix sum selects a global winner proportional to each thread's candidate count.

5.3 Simulated Quantum Annealing

The SQA implementation is more complex, as it must manage both intra-slice and inter-slice energy contributions across an array of Trotter slices and replicas.

Because inter-slice coupling prevents adjacent slices from being updated simultaneously, local sweeps launch blocks over only half of the Trotter slices. The warps are used to parallelize the Δz computation. For global sweeps, the layout changes to a two-dimensional thread group: the x-dimension handles the warp reduction for ΔE and the y-dimension iterates over all slices in parallel.

Coloring of the slices is used to safely parallelize local sweeps. For the standard ring topology, even- and odd-indexed slices form independent sets: updating all even slices simultaneously introduces no coupling conflicts, and the same holds for odd slices. The two sets are swept sequentially with an explicit synchronization barrier between them. For the torus topology, a checkerboard pattern generalizes this to the two-dimensional grid structure. To use this odd/even coloring with cyclic boundaries we require an even number of n_{trotter} slices.

Energy decomposition is a key design feature of the local sweep kernel. The `Δenergy` function returns three distinct quantities for each proposed swap (i, j) in slice k:

$$(\delta E, \delta z, \delta g) = \Delta\mathcal{H}(\pi, k, i, j, J\perp), \tag{5.1}$$

where δE is the total energy change including the inter-slice coupling contribution (used for the Metropolis acceptance criterion), δz is the intra-slice QAP cost change only (used to update the per-slice energy), and δg is the change in Hamming distance to the neighboring slices (used to update the coupling overlap g). This decomposition is necessary because the full replica energy is tracked as $E = \sum E_k + J_\perp g$, requiring separate bookkeeping of the QAP and coupling contributions. Energy updates are applied atomically across concurrently executing slice blocks.

For global sweeps, the y-dimension covers all n_{trotter} rather than half, since no coupling conflicts arise across slices in a global move. We do not utilize extra threads for energy calculation because this would often exceed the block size limit. For each proposed swap (i, j) the energy change is computed independently per slice and stored in a dynamically allocated shared memory array of size n_{trotter}. The total energy change $\Delta E = \sum \Delta E_i$ is then reduced across slices using `reduce_warp` before a single acceptance decision is made. If accepted, all slices are updated simultaneously. This implements the worldline-flipping global move described, where a swap is accepted only if it improves the total energy summed across all Trotter slices.

During early development, we experimented with a single fused kernel that performs the local sweeps (odd and even) and the global sweep, using `sync_threads()` between phases. However, attempting to synchronize one replica per block in a single complex kernel degraded performance. Instead, the `sweep!` function explicitly synchronizes the phases by launching three separate kernels per sweep type (`sweep_odd`, `sweep_even`, `sweep_global`).

Solution tracking is performed per-slice: `solution_energy_per_slice[slice_idx, replica_idx]` is updated inside the kernel whenever a swap yields a new per-slice best.

5.3.1 Ineffectiveness of Caching for SQA

While we extended the caching mechanism to SQA, we ultimately opted not to use it based on the distinct algorithmic dynamics of the quantum-inspired simulation.

In standard SA, the acceptance rate drops asymptotically towards zero as the temperature cools, making it highly efficient to use caching. In contrast, SQA relies on a coupling parameter that typically increases during the late stages of the anneal to force consensus among the Trotter slices. This increasing penalty actively drives state flips as the slices are forced to align with one another. Consequently, the local acceptance rate is expected to remain relatively high even at low temperatures. Because the low acceptance threshold required to make caching computationally beneficial is likely never reached, we will not utilize the cache for SQA in our experiments.

5.4 Momentum Annealing

The GPU implementation of Momentum Annealing supports three sweep types `LocalSweep`, `PseudoGlobalSweep`, and `GlobalSweep`. Each replica is assigned a CUDA block, and each round-robin match within a sweep is assigned a thread.

Local and Pseudo-Global Sweeps share a common kernel dispatcher. Because MA operates on a bipartite graph representation, the left and right side must be updated independently. The kernel executes in two sequential phases (first sweeping the left side, then the right) with an explicit `sync_threads()` barrier between them to prevent race conditions across the shared permutation array.

Within each phase, threads execute round-robin tournament rounds in parallel. For local sweeps, a key design decision is whether to perform all $n - 1$ rounds within a single kernel launch. When enabled, this Connolly-inspired strategy evaluates the entire neighborhood within one launch, making the GPU implementation significantly more competitive against the CPU baseline by amortizing kernel launch overhead. When disabled, only the round corresponding to the current step is executed. Pseudo-global sweeps instead operate one frozen round at a time, since the cache depending on the mutated side must be rebuilt between rounds.

Local sweeps and pseudo-global sweeps share the round-robin pairing scheme, but not the same execution semantics. For local sweeps, disjoint swaps are independent because the opposite side is fixed, so accepted moves can be committed directly in parallel. For pseudo-global sweeps, this is not valid: the acceptance score uses the combined two-sided energy and therefore depends on the full active permutation. We therefore implement pseudo-global sweeps as frozen rounds. In phase 1, all disjoint proposals are evaluated against the same pre-round state and their accept/reject decisions are stored. After synchronization, the accepted one-sided swaps are applied in parallel. The bookkeeping then uses the one-sided `ΔE` and `Δg`, which are additive across disjoint swaps.

The accepted one-sided contributions are reduced to block-level totals using `CUDA.reduce_block` before updating the shared energy E and coupling overlap g:

$$E \overset{+}{=} \sum_{\text{threads}} \Delta E_i \tag{5.2.a}$$

$$g \overset{+}{=} \sum_{\text{threads}} \Delta g_i \tag{5.2.b}$$

The pseudo-global sweep uses the combined two-sided energy only for frozen-round acceptance, while the actual bookkeeping uses the one-sided `ΔE` and `Δg` of the applied swaps.

Global Sweeps employ a simpler kernel that iterates over all pairs (i, j) within a single thread block. Only thread 1 evaluates the Metropolis criterion and commits swaps; a `sync_threads()` barrier follows each accepted move to ensure consistency. While inherently sequential and therefore slower, this sweep provides a worldline-flipping exploration analogous to the global sweep in SQA.

5.5 Momentum Annealing Cached

The bipartite structure of MA enables a particularly efficient cache.

Unlike the cached SA implementation, the caches remain valid throughout an entire local sweep because each depends only on the fixed opposite side. Thus, the algorithm rebuilds the left cache H^L only when switching to a left sweep and rebuilds H^R only when switching to a right sweep. This makes caching substantially more natural for MA than for SA, where the cache must be updated after accepted moves. The pseudocode for the GPU implementation of the cached local sweep is shown in Algorithm 5.2.

Pseudocode of cached local_sweep

```
# tid, stride, replica_idx = threadIdx().x, blockDim().x, blockIdx().x
for side in (:left, :right)
    for round_i in 1:(qap.n - 1)
        for match_i in tid:stride:(qap.n ÷ 2)
            i, j = round_robin(qap.n, round_i, match_i)
            ΔE = Δenergy(side, πL, πR, i, j, cache, λ * (rand() < dropout_rate))
            should_accept(ΔE, T) && swap!(π_side, i, j)
        end
        sync_threads()
    end
    rebuild_opposite_cache!(cache, qap, πL, πR, side)
end
```

Algorithm 5.2: Cached MA local sweep with dropout

For pseudo-global sweeps, the situation is slightly different. During a frozen left round, H^L remains valid for the one-sided bookkeeping term, but the acceptance surrogate additionally depends on the current right-side cache H^R. Because the left permutation changes after the round, H^R must be rebuilt before the next frozen left round. The right-sweep case is symmetric.

5.6 Parallel Tempering

Naive implementations of Parallel Tempering would swap entire state arrays between replicas in memory. Instead, our algorithm leverages a `Bijection` mapping to swap only their temperature parameters. The physical memory arrays remain stationary, while the logical temperatures are exchanged, vastly reducing memory bandwidth overhead.

For **standard PT**, the solver evaluates and attempts pairwise $(\text{temp}_i, \text{temp}_{i+1})$ swaps.

For **PT-SQA**, Trotter slices within a replica are stored contiguously, so swapping logical indices via the bijection would lead to non-contiguous memory accesses. Instead, we utilize the warps to physically exchange the spins, ensuring coalesced memory access. The target slices for these swaps are randomized each round to prevent systematic bias. For the swap we utilize the `energies_per_slice` of the SQA structure.

Cyclic cluster moves are implemented as a hybrid CPU-GPU procedure. The cycle decomposition is performed on the CPU: finding all cycles of the permutation $\sigma = \pi_1^{-1} \circ \pi_2$ that transforms one replica's assignment into another's. Cycles shorter than 3 (which reduce to 2-opt moves) and longer than $\frac{n_{\text{QAP}}}{2}$ (which would exchange the majority of the assignment, risking replica collapse) are filtered out.
The energy change $\Delta z(\pi, c)$ for each cycle c is evaluated on the GPU. The kernel decomposes the computation into two cases. For pairs (i, j) where both indices lie within the cycle, the contribution is parallelized over the cycle elements. For pairs where only one index lies within the cycle, the outer loop is parallelized over all n positions rather than over the cycle length, since cycles are typically small relative to n. All three shared data structures (the cycle, the shifted cycle (encoding the post-move assignment), and a boolean membership array) are stored in dynamic shared memory to minimize global memory traffic. Multiple cycles are evaluated in parallel by assigning one CUDA block per cycle, with results returned as a vector of energy deltas.

Path relinking itself is a sequential procedure: the iterative path traversal and move selection cannot be parallelized across steps. However, the most expensive inner operation, evaluating Δz for all valid moves at each step, is offloaded to the GPU.

5.7 Population Annealing

Our Population Annealing implementation requires a numerically stable computation of the Boltzmann reweighting factors $e^{-\Delta\beta E_i}$, as naive evaluation risks floating-point overflow or underflow for large energies. To address this, we normalize by the mean energy $\bar{E}$ and compute the partition function estimate Q in log-space.

The original estimator is $Q = \frac{1}{R} \sum e^{-\Delta\beta E_i}$. To stabilize this, we instead compute

$$q = \sum e^{-\Delta\beta(E_i - \bar{E})}, \tag{5.3}$$

which is equivalent up to a constant factor. This follows from:

$$e^{-\Delta\beta(E_i - \bar{E})} = e^{-\Delta\beta E_i} e^{\Delta\beta \bar{E}}, \tag{5.4}$$

yielding

$$q = e^{\Delta\beta\bar{E}} \sum e^{-\Delta\beta E_i} \tag{5.5.a}$$

$$= e^{\Delta\beta\bar{E}} RQ. \tag{5.5.b}$$

Solving for $\log Q$ and reverting the mean-shift gives:

$$\log Q = \log q - \Delta\beta\bar{E} - \log R. \tag{5.6}$$

The normalized per-replica weights are then computed as:

$$w_i = \frac{e^{-\Delta\beta E_i}}{Q} \Rightarrow e^{-\Delta\beta E_i - \log Q} \tag{5.7}$$

The normalized form avoids any intermediate exponentiation of large values. This stabilization technique was discovered by inspecting the source code of [83], where it is implemented but not explicitly documented in the accompanying paper.

5.8 Summary

In summary, `ASSA.jl` implements five solver families (SA, SQA, MA, PT, and PA), all operating natively in permutation space with GPU-parallel sweep kernels. Correctness is guaranteed by full energy recomputation after every solve. The SoA memory layout and warp-level reductions enable coalesced GPU access across all solver types. The framework currently supports specialized compositions PT-SA, PT-SQA, PT(2D)-MA, and PA-SA; further hybrid combinations are architecturally feasible.

6 Results and Discussion

This chapter evaluates five solvers across dozens of benchmark instances. The evaluation proceeds in three stages: a systematic comparison on QAPLIB (the standard benchmark suite for the QAP), a per-solver behavioral analysis, and finally a real-world application matching brain networks of 1000 regions.

To compare solvers across various problems, we report the optimality gap (in percent), defined as:

$$\text{gap} = \frac{\text{found}}{\text{best known}} - 1. \tag{6.1}$$

Because the primary goal of this thesis is to compare the fundamental behavior of different permutation-space annealing dynamics (e.g., thermal fluctuations vs. quantum tunneling vs. population resampling), we intentionally avoid aggressive instance-specific hyperparameter tuning. Instead, we use a sweep-budgeted baseline configuration across solvers to compare their qualitative behavior under a controlled amount of algorithmic work, highlighting their relative strengths, weaknesses, and convergence patterns.

In the main experiments, all of our solvers are run under a fixed sweep budget, which gives a consistent measure of algorithmic work across instance sizes. For the external solver comparison, we additionally switch to a strict wall-clock budget so that practical performance can be judged under equal time limits. We do not directly compare rankings across these regimes; instead, each isolates a different question.

All computational experiments were conducted on a consumer computer equipped with an `AMD Ryzen 7 9800X3D 8-core` processor (5.2 GHz), 32 GB of system memory (5600 MT/s), and an `NVIDIA RTX 5070 Ti` GPU.

6.1 QAPLIB

QAPLIB, introduced in 1991, provides a standardized collection of benchmark instances for the quadratic assignment problem [10, 90].

Although QAPLIB contains many instance sets, we focus on the Taillard (`tai`) family because it is widely used in the literature, covers a broad range of problem sizes, and includes some of the largest QAPLIB instances.

The Taillard instances are divided into three families, each posing a distinct challenge:

- `taiA`: uniform instances with random points on a circle and uniformly distributed flow values; no exploitable structure, making them the hardest family overall.

- `taiB`: clustered distance matrices with exponentially distributed flow values, intended to resemble realistic structure; the clustered geometry creates narrow high barriers, which we find to favor tunneling-based methods.
- `taiC`: gray-pattern instances with sparse flow matrices, including the largest QAPLIB case (`256`) — low density makes large instances tractable, though they are considered especially difficult [91].

Because of these differences, `taiA` instances are generally regarded as the most challenging among the three families [92].

6.1.1 Simulated Annealing

The parameters used for the SA experiments are summarized in Table 6.1.
For each run, the algorithm was executed with 100 parallel replicas, and the minimum value across replicas was recorded as the best-found solution. To avoid instance-specific tuning, we fixed the number of sweeps at 3300 for all problems, which corresponds to about 150 seconds on `tai150b`. Each experiment was repeated for 10 independent rounds.

Replicas	Runs	Sweeps
100	10	3300

Table 6.1: Parameters for SA.

Since Figure 6.1 shows that the final quality is nearly identical, the choice of schedule reduces to a runtime question. Connolly's per-step bookkeeping overhead makes it generally more expensive in absolute wall-clock time (and code readability).
Reference Solver: We therefore use SA with the geometric schedule as the reference solver for the remainder of this chapter.

The Connolly schedule does improve the initial convergence rate, likely because of the underlying Lundy and Mees update rule rather than the restart mechanism itself. However, since the final results remain very similar across these schedules, Connolly does not provide a strong enough advantage to justify adapting its reheating mechanics to the other annealing algorithms. We therefore omit it from further evaluation.

Figure 6.2 reveals a systematic mismatch between the default temperature bounds and problem structure. On `tai100a`, both schedules stagnate early, indicating $T_{\min}$ is too low; on `tai150b`, the gap decreases throughout the full run, indicating $T_{\min}$ is too high and prematurely ends exploitation.

We also explored cache-based variants to better understand the runtime/quality trade-off. The fully cached version is discussed in Appendix A.2, while the adaptive cache activation study is discussed below.

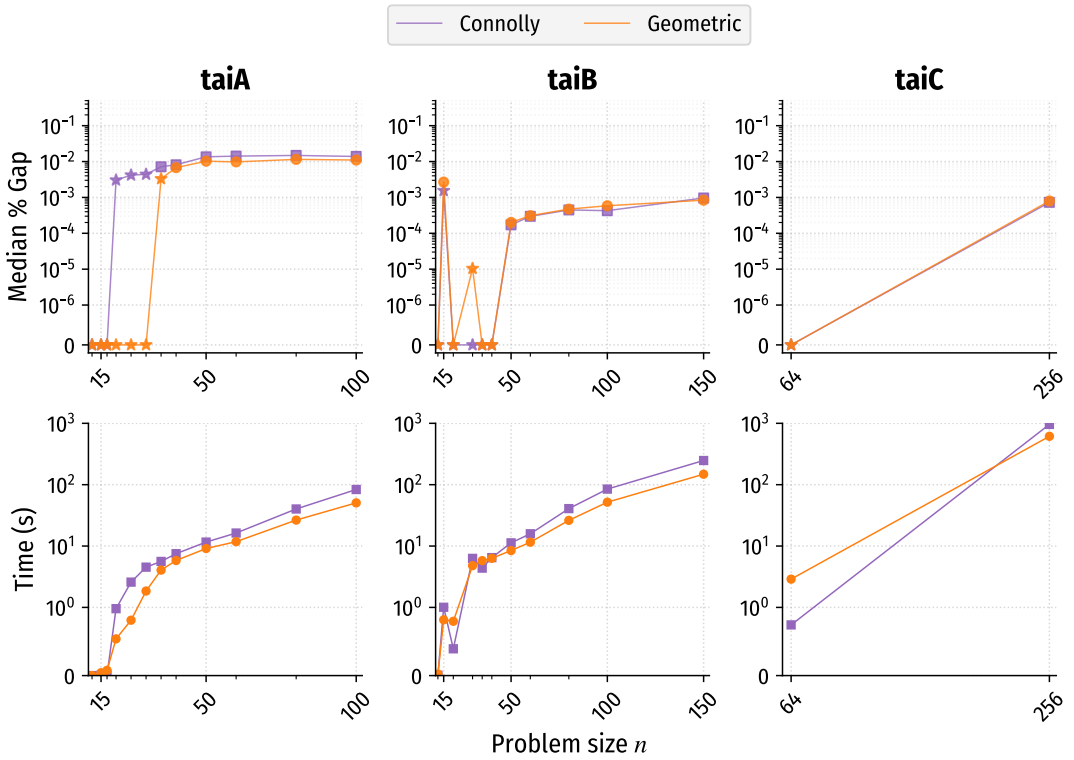


Figure 6.1: Solution quality of SA across all three Taillard families (`taiA`, `taiB`, `taiC`). Under a fixed budget of 3300 sweeps with 100 parallel replicas and 10 independent runs, each point shows the median optimality gap; star markers indicate instances where the best-known solution was reached at least once. Geometric and Connolly cooling achieve similar final solution quality, with `taiA` instances remaining the hardest overall. The small-instance outlier `tai15b` is the only case where the geometric schedule consistently fails to reach the optimum.

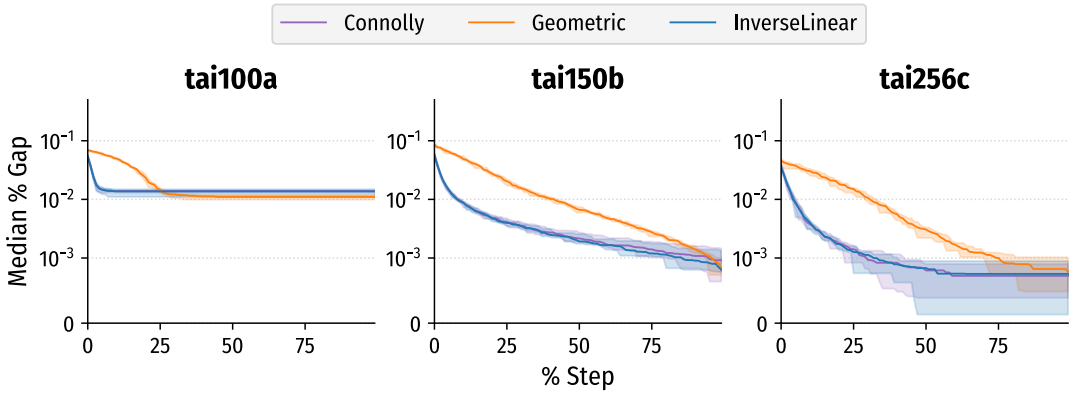


Figure 6.2: Convergence trajectories of SA on the three largest instances of each family. On `tai100a`, both schedules stagnate rapidly in a local minimum, suggesting that $T_{\min}$ is set too low for this rugged landscape. On `tai150b`, the gap decreases continuously throughout the run, indicating that $T_{\min}$ is too high and cuts off exploitation prematurely. On `tai256c`, the behavior is intermediate. The Connolly schedule improves early convergence rate due to its Lundy & Mees update rule, but final solution quality is nearly identical across schedules.

Caching after acceptance rate threshold: The cache is only efficient at low temperatures. Following [58], we activate it once the acceptance rate drops below $\frac{1}{3n_{\text{QAP}}}$. To evaluate this hybrid approach, both the baseline and cached solvers were run for 3300 sweeps on `tai100a`.

The spike at activation shown in Figure 6.3 is the concern: it can temporarily worsen the tracked energy before the cached regime stabilizes. In production, a short warm-up period after activation (holding temperature constant) could smooth this transition.

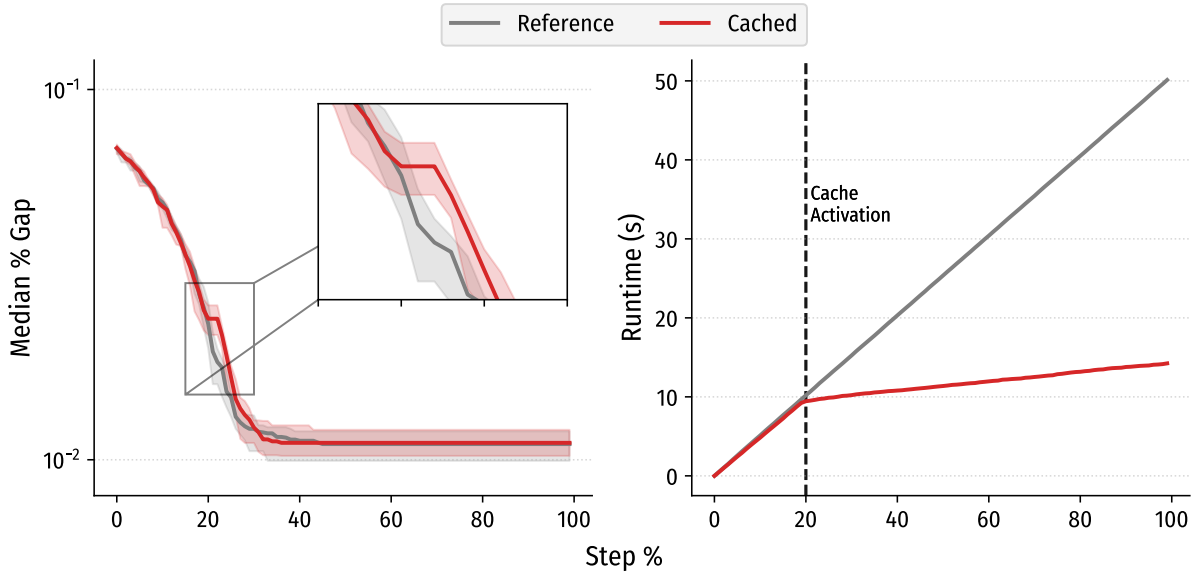


Figure 6.3: Effect of adaptive cache activation on `tai100a` over 3300 sweeps: the cache is enabled once the swap acceptance rate drops below $1 \div (3n)$. Left panel: energy trajectory showing a brief spike at the activation threshold, caused by the abrupt transition from sequential accept to exhaustive single-choice evaluation. Right panel: wall-clock runtime comparison between baseline SA and the adaptive-cache variant, demonstrating a sharp reduction in per-sweep cost after activation while preserving final solution quality. This strategy provides no advantage on `tai150b` and `tai256c`, which do not reach the threshold within their temperature schedule.

Notably, this adaptive caching strategy provided no advantage for `tai150b` and `tai256c`, since those instances did not reach the threshold within the schedule.

6.1.2 Simulated Quantum Annealing

To keep the total slice count comparable to SA, SQA is configured with 10 parallel replicas and 10 Trotter slices per replica. In practice, fewer slices per replica tend to yield higher sweep rates within the same time budget, likely because the associated kernels execute more quickly.

A single SQA sweep requires roughly three times the work of an SA sweep. The local updates are split into odd and even slice phases, followed by a global sweep. Accordingly, one would expect about $\frac{3300}{3} = 1100$ SQA sweeps within the 150-second runtime budget used for SA, but empirically we observe only around 550 sweeps.

Runs	Replicas	Trotter Slices	Sweeps	Time (tai150b)
			3300	≈ 850s
10	10	10	1100	≈ 290s
			550	≈ 135s

Table 6.2: Parameters for SQA.

Figure 6.4 illustrates the core failure mode of monotonic SQA: the solver freezes. The sharp drop at approximately 75% of the run marks the moment all Trotter slices collapse into a single classical permutation, after which no further progress is made.
Figure 6.6 shows the effect of adaptive coupling: the solver remains active throughout. The staircase pattern in the objective value reflects successive collapse events, each followed by continued exploration rather than stagnation.

Convergence Behavior under Monotonic Schedules

In standard quantum annealing hardware (e.g., D-Wave), the annealing schedule follows a monotonic crossover: the transverse field $A(s)$ decreases while the problem coupling $B(s)$ increases.
Our SQA baseline mirrors this behavior by geometrically decreasing the temperature while geometrically increasing the coupling strength.

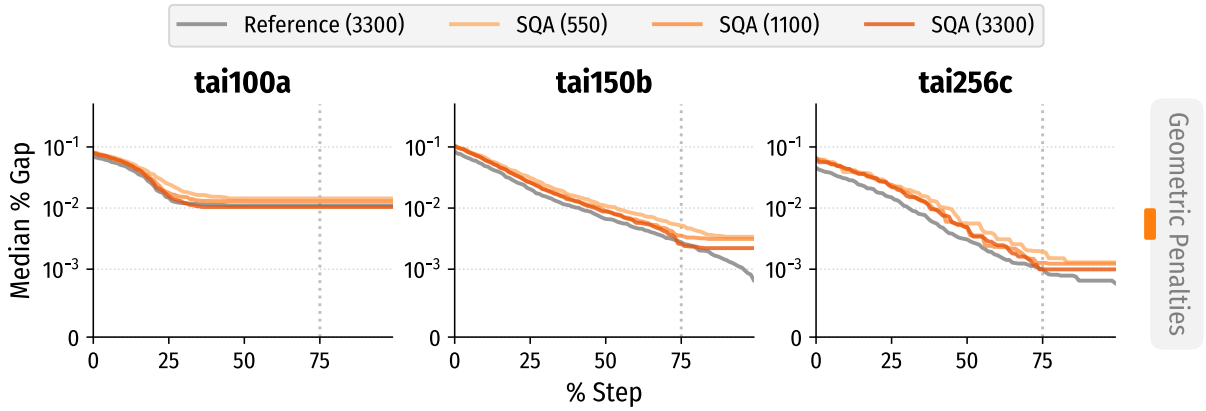


Figure 6.4: Convergence of SQA under a monotonic schedule (geometrically decreasing temperature, geometrically increasing coupling $J_\perp$) on the largest tai instances. The coupling crosses a critical threshold at approximately 75% of the run, forcing all Trotter slices to collapse into a single classical permutation. Beyond this point, the solver is effectively frozen and no further improvement occurs. This single-collapse behavior can prematurely trap the system and thus motivates the self-adaptive coupling mechanism.

The single-collapse structure in Figure 6.4 is the fundamental limitation: once frozen, there is no mechanism for recovery. This motivates the self-adaptive coupling below.

Convergence with Self Adaptive Couplings

To induce multiple controlled collapses, we introduce a self-adaptive penalty mechanism inspired by [80]. Analogous to Connolly's temperature schedule in SA, we cyclically reset the coupling penalty.

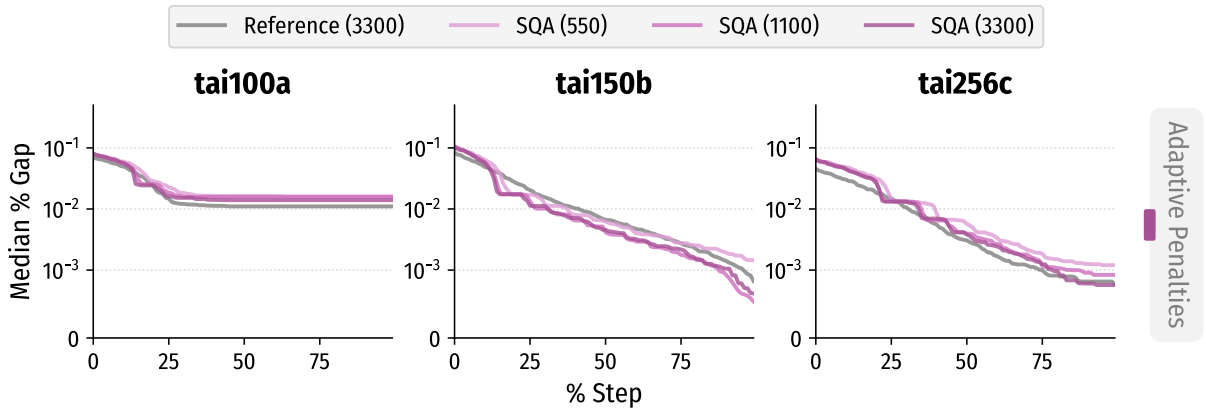


Figure 6.5: Convergence of SQA under the self-adaptive coupling schedule across all three Taillard families at different sweep counts. The cyclic reset mechanism allows the Trotter slices to repeatedly expand into a superstate and re-collapse, producing step-wise improvements in the gap that are absent under the monotonic schedule. On `tai150b` at 1100 sweeps, the adaptive schedule can outperform the SA reference; however, under an equal time budget of 550 sweeps, SQA remains inferior overall, and no consistent improvements over SA are observed on `taiA` or `taiC`.

Figure 6.5 shows that on the largest `tai` instances, the adaptive schedule improves over monotonic SQA.
The anomaly where 1100 sweeps yield better results than 3300 sweeps on `tai150b` is likely a statistical artifact, and no improvements over SA are observed on the remaining families.

The penalty accumulates until the average pairwise Hamming distance (alignment) between slices reaches 40%, at which point it resets to 0. This cyclic reset prevents permanent freezing and allows the slices to re-expand into a *superstate* before collapsing again.

$$J_{\perp_{k+1}} = \begin{cases} J_{\perp_k} + \alpha & \text{if average Hamming distance} > 40\% \\ 0 & \text{otherwise} \end{cases} \tag{6.2}$$

Figure 6.6 confirms the mechanism quantitatively: improvements are not gradual but punctuated, occurring exactly at collapse events. By allowing the system to repeatedly breathe out into a *superstate* and then collapse back, the solver *tunnels* through barriers multiple times within one run.

However, SQA remains expensive, and on smaller `taiA` instances it is still less reliable than the simpler SA baseline.

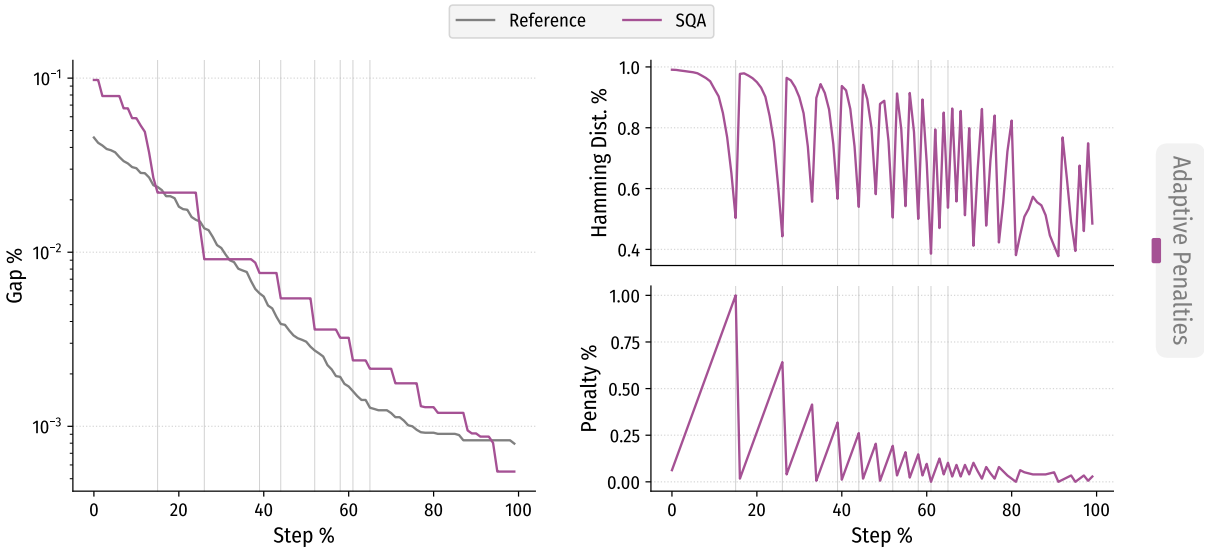


Figure 6.6: Detailed internal dynamics of self-adaptive SQA on `tai150b` using a single replica with 100 Trotter slices. Left panel: optimality gap over time. Right bottom panel: normalized coupling penalty $J_\perp$; right top panel: pairwise normalized Hamming distance between adjacent Trotter slices. Each downward spike in Hamming distance corresponds to a collapse event (slices aligning) and coincides with a step-wise improvement in solution quality. The coupling resets to zero whenever the average Hamming distance falls below 40%, allowing the system to re-expand before tunneling again. The first alignment typically takes the longest and subsequent phases progressively shorten. Note that we log only 100 times per run, such that penalty and Hamming distance do not always reach their exact minima visually.

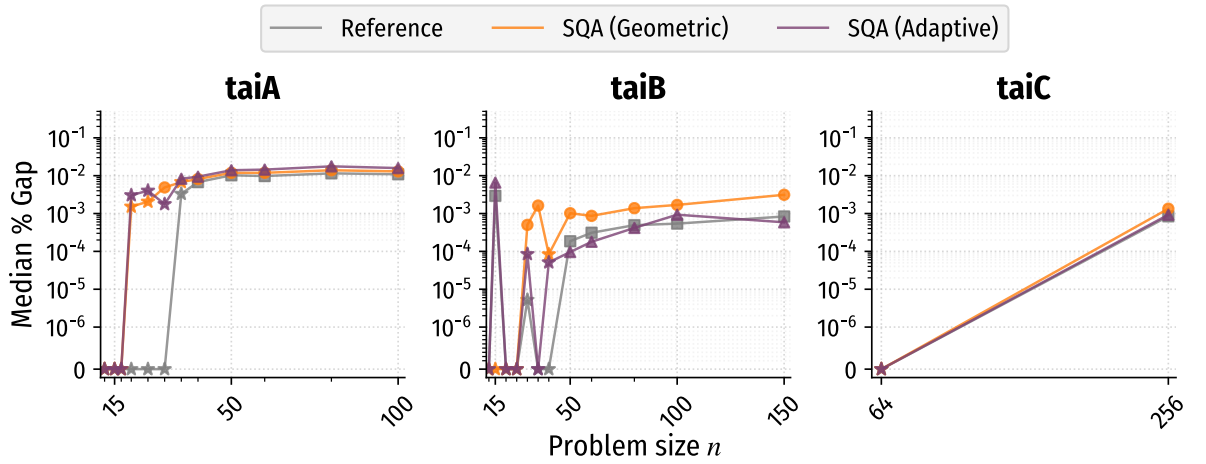


Figure 6.7: Head-to-head comparison of SQA with the monotonic geometric vs self-adaptive coupling schedules at 1100 sweeps across all three Taillard families. The adaptive schedule outperforms the monotonic baseline on several `taiB` instances by inducing multiple tunneling phases, but offers no consistent improvement on the `taiA` and `taiC` families.

Summary

Overall, standalone SQA is interesting but not yet competitive enough to justify its overhead on QAP. Its main strength is that adaptive couplings can induce tunneling be-

havior, but its performance is still highly sensitive to hyperparameters and constrained by coarse discretization.

To mitigate this hyperparameter dependence, combining SQA with PT offers a promising direction, which we evaluate in Section 6.1.5.2. Extending this exchange to two dimensions (PT2D) could theoretically stabilize the solver across both thermal and quantum regimes. However, the multiplicative increase in replicas renders this computationally prohibitive for our current scope, and we leave the evaluation of SQA-PT2D (alongside higher-resolution Trotter grids) to future work.

The torus extension of SQA is discussed in Appendix A.3.

6.1.3 Momentum Annealing

Our evaluation of MA shows a clear trade-off: weak solution quality on QAPLIB, but very fast execution.

On the unstructured `tai` instances, MA often stalls at a high median gap because the temperature and penalty schedules must stay in a very narrow regime for the slices to align without collapsing too early or remaining decoupled.

No single schedule generalizes across QAPLIB: the temperature and penalty must be simultaneously in a narrow symbiotic range, which varies by instance. We settled on:

- 3300 sweeps with a geometric temperature schedule.
- Linear penalty schedule: adaptive and geometric penalties performed worse.
- Cache usage: 5% dropout for standard local sweeps; pseudo-global sweeps were limited to one round to reduce cache overhead.

Figure 6.8 makes the hyperparameter problem concrete: the Hamming distance trajectory directly exposes whether the penalty schedule is too weak (sides stay decoupled) or too strong (sides collapse before temperatures are low enough to exploit the consensus). Neither failure modes can be resolved by a single static schedule.

We tried to solve this hyperparameter-symbiosis issue by using PT2D in Appendix A.4, but the computational overhead of generating the 2D grid proved prohibitive, and we leave a more thorough evaluation of PT2D to future work.

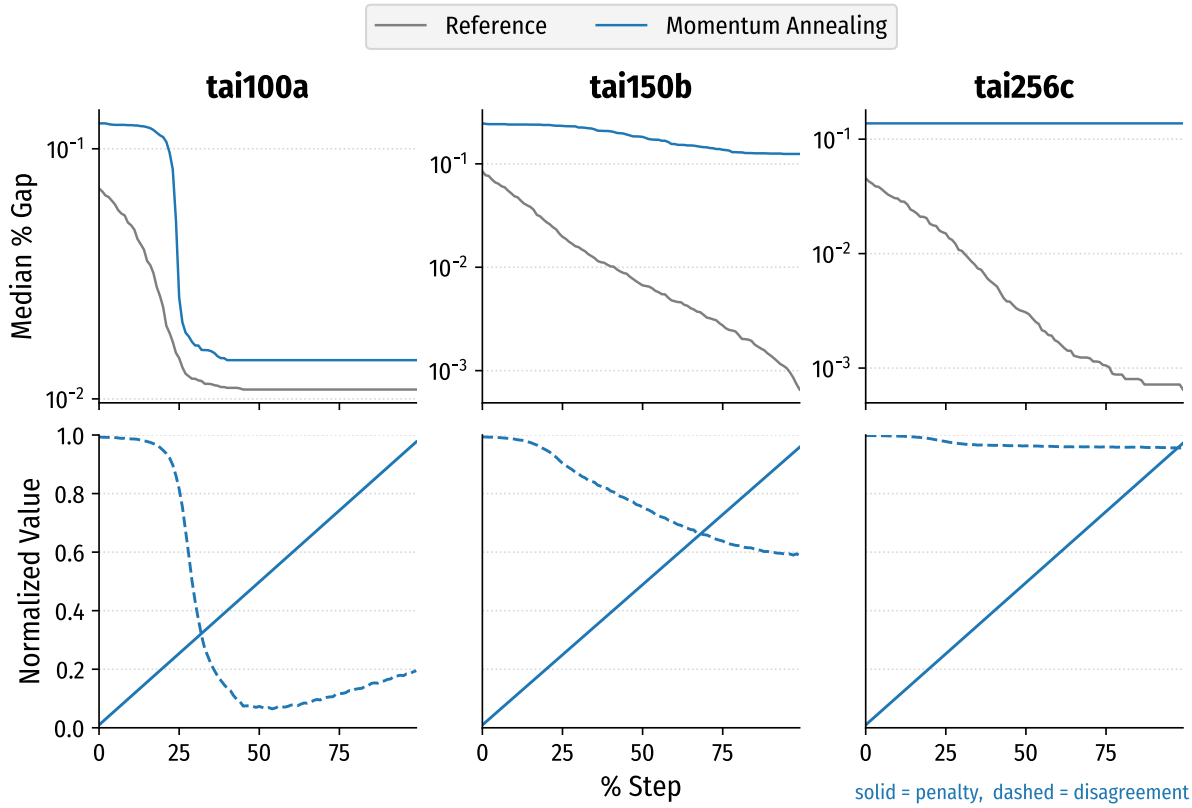


Figure 6.8: MA convergence on the three largest Taillard instances. Top panel: optimality gap trajectory. Bottom panel: normalized linear penalty λ (solid) and Hamming distance between left and right permutations (dashed), indicating the degree of side agreement. On `tai150b` and `tai256c`, the two sides remain poorly aligned (Hamming distance stays high) throughout the run, and the gap does not decrease, indicating that the penalty hyperparameter is not tuned sufficiently high. Only `tai100a` reaches reasonable agreement, but still fails to match the SA baseline. The figure illustrates that effective optimization (in `tai100a`) occurs only during the narrow regime where temperature and penalty are simultaneously in symbiosis, motivating the PT2D extension explored in Appendix A.4.

Execution Speed

The application of MA to the QAP, using a round-robin schedule, already provides a speedup by organizing the $\binom{n}{2}$ pair updates into $n-1$ rounds of $\lfloor \frac{n}{2} \rfloor$ parallel sweeps. The main additional improvement comes from the specialized cache, which reduces the $O(n)$ cost of the 2-opt step. Note that MA benefits from cache reuse throughout the solve, whereas SA mostly benefits from caching in the later stages of annealing.

By combining cache reuse with the parallelizable update structure, MA achieves a clear runtime advantage, as seen in Figure 6.9. The widening gap in Figure 6.9 is what makes Section 6.2 interesting: at n=1000, MA's O(1) cache evaluation keeps per-sweep time very competitive with e.g. FAQ.

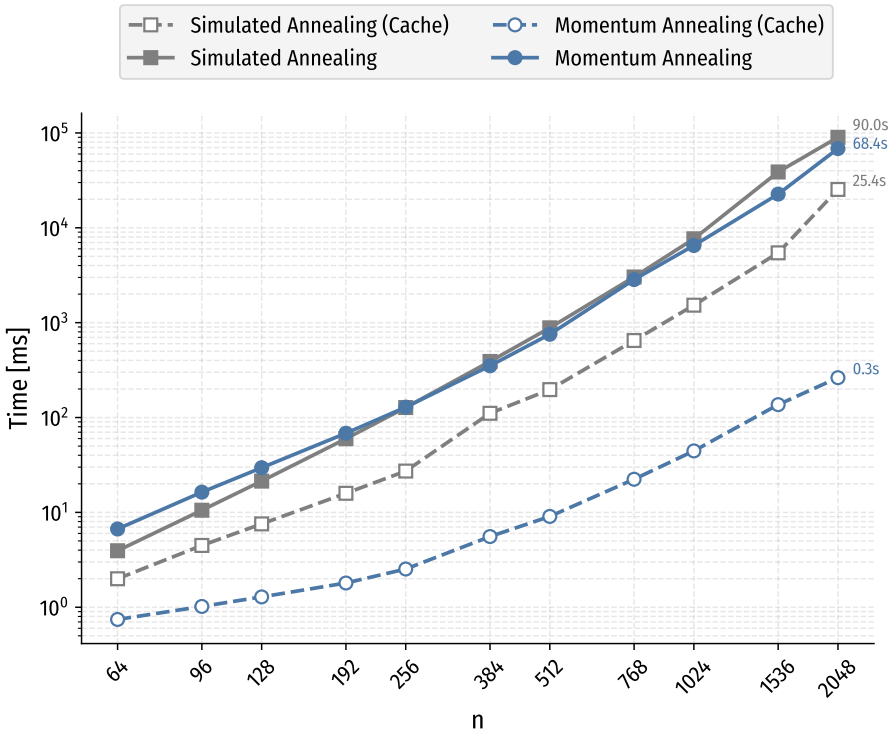


Figure 6.9: Execution time per sweep versus problem size for MA and SA. MA's per-sweep cost grows more slowly due to two compounding advantages: (i) the round-robin tournament scheme parallelizes $\lfloor n \div 2 \rfloor$ conflict-free swaps per round, and (ii) the side-conditioned cache reduces each swap evaluation from $O(n)$ to $O(1)$ and is rebuilt only on side-switches rather than after every accepted move. The gap between the two curves widens with n, motivating MA as the preferred solver for large-scale instances such as the $n = 1000$ connectome graphs in Section 6.2.

6.1.4 Population Annealing

To evaluate PA, we use 10 independent runs with a population of 100 replicas. At each temperature step, we perform 10 sweeps, for a total of 330×10 sweeps per replica. The local solver for each replica is the cache-adaptive SA variant.

We compare standard geometric cooling with an adaptive temperature schedule that keeps the overlap between successive energy histograms in the range $0.83 \leq \alpha \leq 0.89$.

Runs	Replicas	Sweeps per Temperature	Steps
10	100	10	330

Table 6.3: Parameters for PA.

Figure 6.10 shows that PA is the first solver in this evaluation to consistently solve taiB instances to optimality at scale.
Adaptive PA is both faster and more accurate than geometric PA, which at first appears counterintuitive: one might expect a more sophisticated schedule to carry additional overhead. The explanation is that the adaptive schedule cools faster, which activates the $O(1)$ cache earlier, and the resulting speedup more than compensates for the schedule's own computational cost.

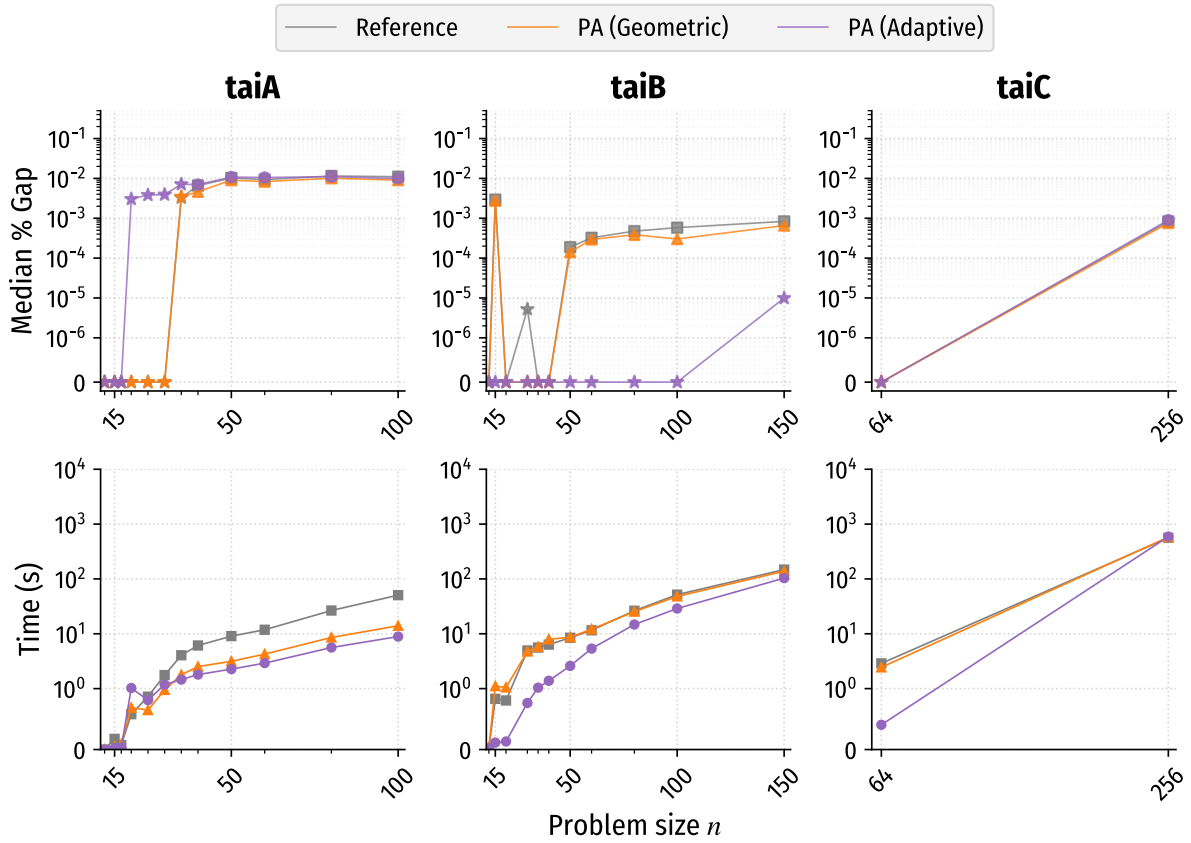


Figure 6.10: Solution quality of PA with geometric and adaptive temperature schedules across all three Taillard families. Using 100 replicas, 10 sweeps per temperature step, and 330 temperature steps. Star markers indicate runs where the best-known solution was found at least once. The adaptive schedule consistently solves all tested `taiB` instances to optimality (0% gap), including sizes up to $n = 150$, while the geometric schedule stalls at sub-optimal solutions. On `taiA` and `taiC`, both schedules show limited improvement over the SA baseline. Remarkably, the adaptive schedule achieves lower runtimes than the geometric one despite its overhead, because faster cooling allows earlier entry into the low-temperature caching regime.

Figure 6.11 reveals a fundamental tension: PA has no mechanism to detect whether the population has converged prematurely. A collapse detection criterion (analogous to the Hamming-based reset in SQA) could trigger a reheating step and would be a natural extension. Another more systematic fix would likely be hybrid PT-PA design, which we leave for future work.

A slower schedule with fewer temperature steps and more sweeps per step improves results on `taiA`, but it also hurts `taiB` by delaying entry into the low-temperature exploitation phase.

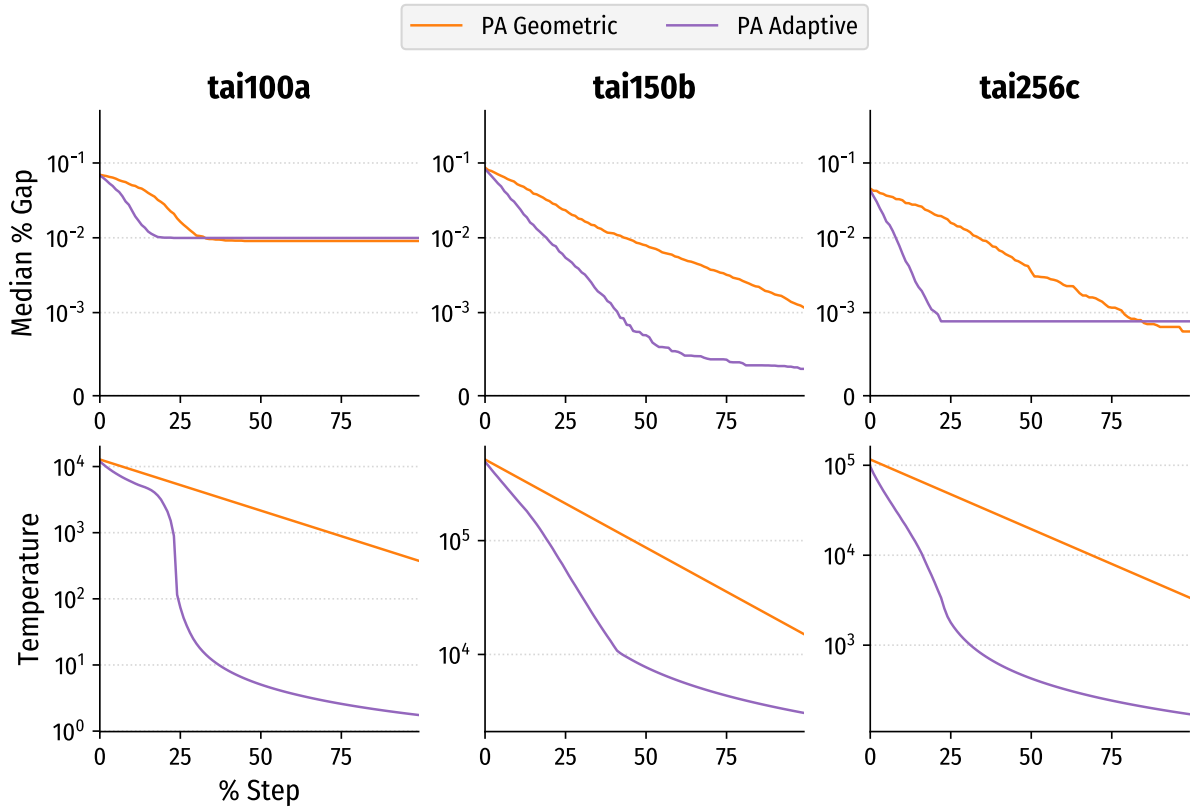


Figure 6.11: Joint evolution of optimality gap and temperature for PA with geometric and adaptive schedules. The adaptive schedule cools substantially faster, spending more time in the low-temperature exploitation phase and reaching a lower final gap. On `tai150b`, this is beneficial: it gives PA a longer low temperature exploitation phase and leads to better solutions than the geometric schedule. On `tai100a` and `tai256c`, this faster cooling can be detrimental, collapsing population diversity before good solutions are found.

6.1.5 Parallel Tempering

Parameters used for PT are shown in Table 6.4. The adaptive temperature sampling takes considerable time; see Appendix A.5 for detailed parameters and runtime.
For `tai256c`, the preprocessing cost is still about 39 minutes, so the adaptive setup is mainly useful for longer runs where this preprocessing cost can be amortized.

Runs	Replicas	Sweeps / Exchange	Steps
10	30	10	330

Table 6.4: Parameters for PT.

PT's core advantage over SA is robustness: a cold replica that has stagnated can escape by temporarily ascending the temperature ladder, eliminating manual restarts. This advantage is most valuable under long wall-clock budgets where the initial ladder construction cost can be amortized. Under the *small* sweep-limited budget used here, however, PT achieves solution quality broadly comparable to the SA baseline

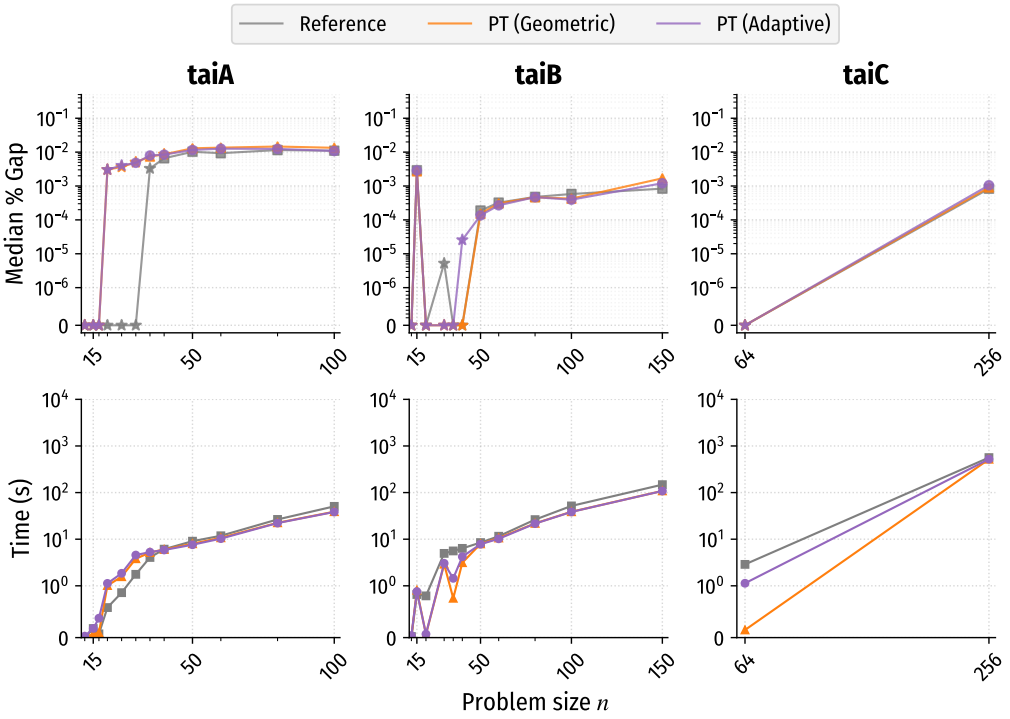


Figure 6.12: Solution quality of PT with geometric and adaptive temperature ladders across all three Taillard families. Using 30 replicas, 10 sweeps per exchange, and 330 exchange steps. PT achieves solution quality broadly comparable to the SA baseline, but its primary advantage is robustness: replica exchange allows trapped cold replicas to escape local minima by temporarily moving to higher temperatures, eliminating the need for manual schedule resets. The adaptive ladder improves mixing but requires significant preprocessing time (see Appendix A.5).

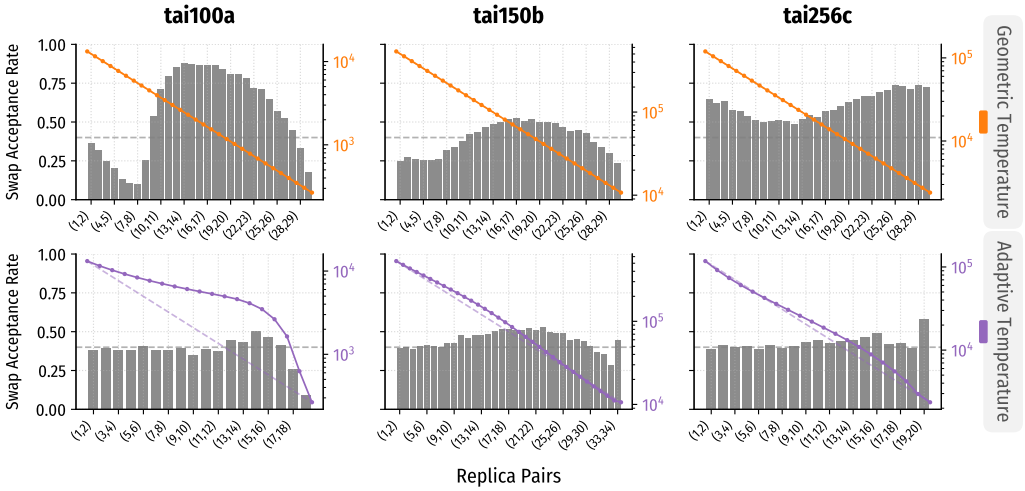


Figure 6.13: Adaptive temperature ladder (bottom axis, line) and resulting per-replica swap acceptance rates (bars) for PT. The adaptive algorithm successfully equalizes acceptance rates across all replica pairs at approximately 40%. It places more temperatures in the high-temperature transition region where acceptance varies rapidly, while using fewer temperatures (replicas) for `tai100a` and `tai256c` and slightly more for `tai150b`. The balanced acceptance rates ensure that configurations can flow efficiently between hot and cold replicas throughout the run.

The instance-dependent replica counts in Figure 6.13 are themselves a diagnostic: fewer replicas needed to achieve uniform 40% acceptance suggests a smoother energy landscape transition, while a higher count hints at a sharper phase boundary that needs finer resolution to bridge.

Cluster Moves

We evaluated PT with 8 copies. To keep the discussion focused, we compare the four exchange variants only at a high level: no spin exchange (*None*), Path Relinking (*PR*), Cyclic Swaps (*CS*), and their combination (*PR+CS*). Restricting cluster moves to the lowest-temperature replicas had no noticeable effect.

Runs	Replicas	Copies	Sweeps per Replica Exchange	Steps
10	Adaptive	8	10	330

Table 6.5: Parameters for PTC.

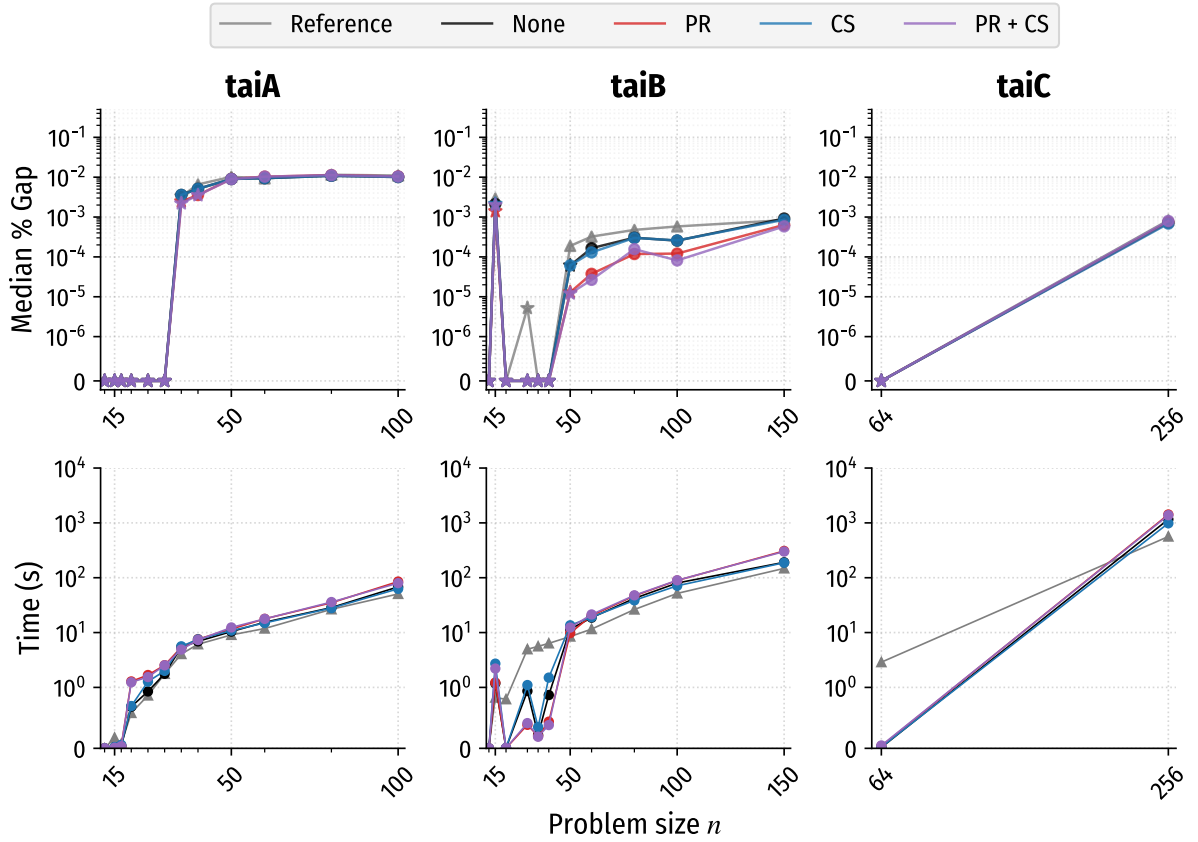


Figure 6.14: Comparison of four PT cluster-move variants across all Taillard families: no spin exchange (None), Path Relinking only (PR), Cyclic Swaps only (CS), and their combination (PR+CS), each using 8 copies and the adaptive temperature ladder. Cluster moves provide the most benefit on `taiB` instances, where PR is the strongest individual variant. CS yields weaker improvements, and combining PR+CS does not significantly change the overall picture. On `taiA` and `taiC`, cluster moves offer only marginal improvements, suggesting that the benefit is landscape-dependent.

Cluster moves help mainly on taiB, where *PR* is the strongest variant. *CS* are weaker, and the combination does not change the overall picture much.

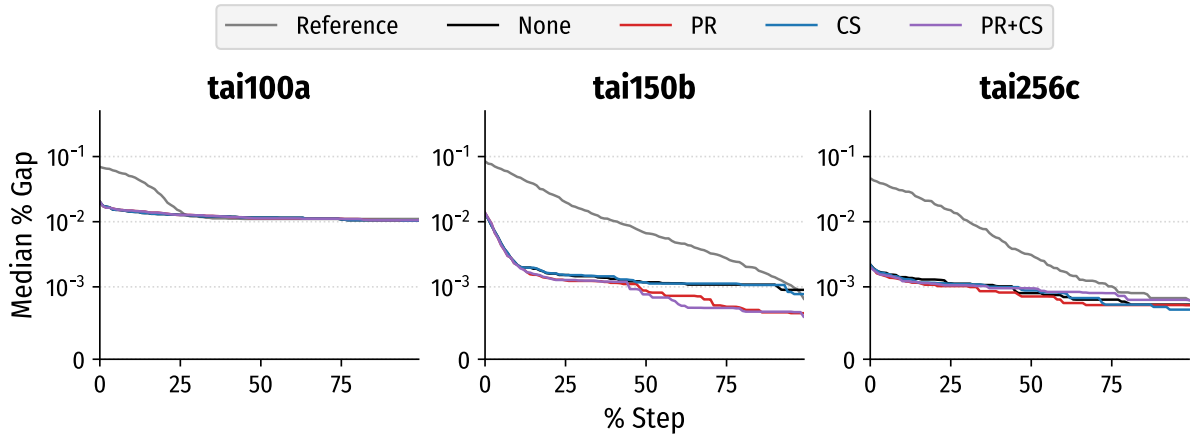


Figure 6.15: Convergence trajectories of PT with and without cluster moves. On tai150b the no-exchange baseline stagnates in a local minimum; PR and PR+CS break through this barrier, leading to sustained improvement throughout the remaining budget.

As illustrated in Figure 6.15, cluster moves improve early convergence on tai150b by helping PT bypass a local minimum.

Implicit Cluster Moves by PT-SQA

We also evaluate PT-SQA, which combines replica exchange from PT with the tunneling dynamics of SQA. For comparability, we use 8 Trotter slices per replica and the same adaptive temperature schedule as for PT.

For the penalty ladder, we try multiple configurations: geometric, inverse linear and a mix of the two. The inverse linear schedule results in only the very last replicas having strong tunneling effects.

We enabled global sweeps and performed in total 330 replica exchanges every 10 sweeps. We suspect that global sweeps introduce unnecessary overhead, since the replica exchange mechanism already enables escape from local minima in collapsed states.

Figure 6.16 establishes PT-SQA as the first quantum-inspired method in this evaluation to match PA on taiB. The inverse-linear advantage is interpretable: concentrating strong tunneling in the coldest replicas means the exploration–exploitation split mirrors the temperature ladder, where hot replicas tunnel weakly and freely, and cold replicas tunnel strongly and precisely.

PT-SQA is a promising hybrid that is especially effective on taiB, suggesting that tunneling-like dynamics are most useful on specific structured landscapes.

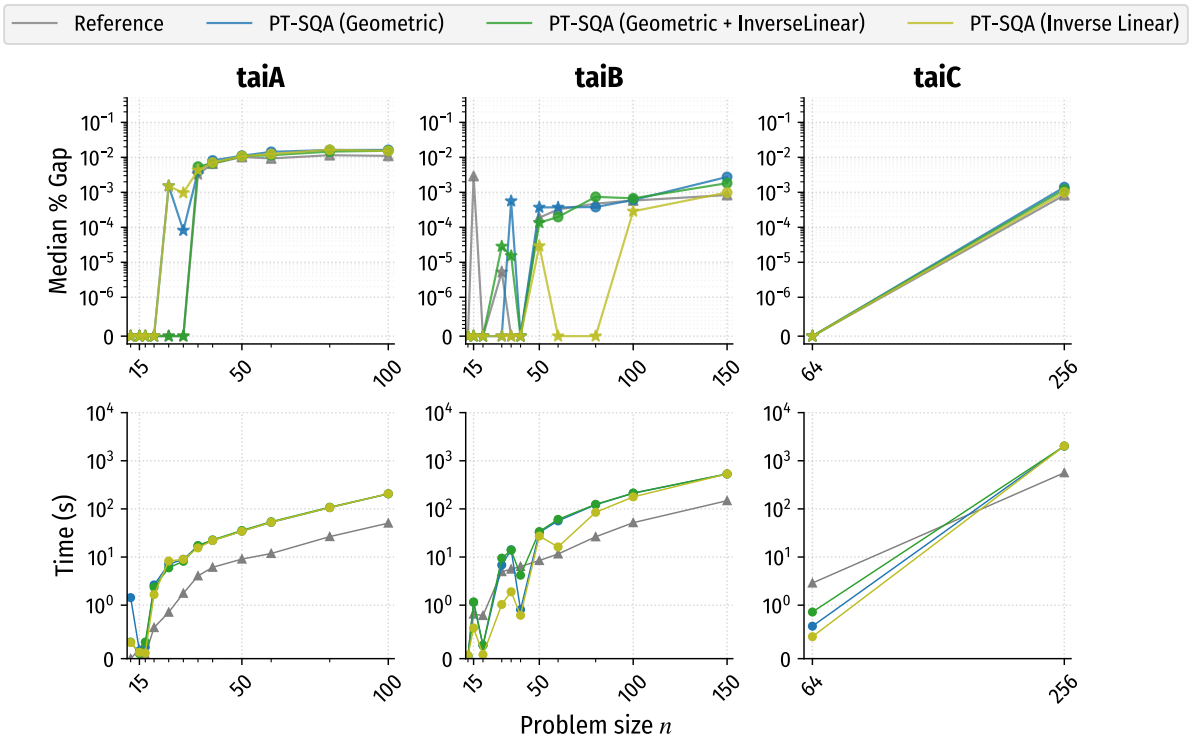


Figure 6.16: Performance of PT-SQA (8 Trotter slices per replica, adaptive temperature ladder) with three penalty schedule configurations: geometric, inverse-linear, and a mixed schedule; compared to the PT baseline across all Taillard families. Implicit cluster moves from tunneling-like dynamics benefit primarily `taiB` instances. The inverse-linear penalty schedule, which concentrates strong tunneling effects in the coldest replicas, finds the optimal solution at least once in all instances up to $n = 100$, matching PA's performance on `taiB` for the first time. On `taiA` and `taiC`, PT-SQA does not offer consistent improvements, confirming that the tunneling advantage is landscape-dependent.

6.1.6 QAPLIB-wide Benchmark Summary

Table 6.6 condenses the full QAPLIB benchmark for our GPU-accelerated solvers. All algorithms are evaluated under a standardized budget of 3300 steps (330×10 for PT and PA) so that the comparison reflects algorithmic efficiency rather than hardware-specific runtime scaling.

We configure SA with a geometric temperature schedule, whereas all other solvers use their respective adaptive schedules. Note that the adaptive schedules used for PT are precomputed beforehand and can take significant time to generate; this preprocessing cost is not reported in the table.
For PTC, we employ both cyclic and path relinking moves. Finally, the PT-SQA configuration reported here utilizes the inverse linear schedule for its penalties. The other parameters are as described in their respective sections.
We exclude MA from this summary, as it generally does not achieve competitive solutions on most instances.

For completeness, we also include the `dre` and `taiE` datasets [93], which serve as recognized supplementary benchmarks beyond the standard QAPLIB. Due to computational

time constraints, our evaluation of the taiE instances is limited to the first of their 20 variations for each size up to $n = 175$, omitting the significantly larger 343 and 729 sizes.

Interestingly, while we solve almost all instances to within a 1% gap, performance degrades significantly on the larger dre instances, where the remaining gap is approximately 50%. This is consistent with the exceptionally high landscape ruggedness reported for these instances (e.g., dre90 has a *ruggedness* of 99.7/100 and a flow dominance of 554 – among the highest in QAPLIB [93]). A high flow dominance indicates that the objective is governed by a small number of particularly strong interactions, whereas high ruggedness implies that neighboring permutations can differ drastically in quality. Together, these properties create a search landscape filled with numerous, sharply separated local optima.

New Best Known Solution (BKS): Both PTC and PA improved upon the previously reported BKS of 59732 for tai175e01. The new BKS is now 59286. One important caveat applies: early stopping was active throughout these experiments, terminating runs once the then-current BKS was matched. This means further improvements beyond what we report here may easily exist. A dedicated extended run without early stopping could confirm whether this BKS is tight. Note that both PA and PTC are atypically fast on this instance.

Overall, the table shows a clear hierarchy: the advanced annealing methods substantially outperform the baseline, but their strengths differ by problem structure. PA is the strongest solver in terms of time-to-solution and is especially effective on the difficult taiB family. PTC is the most robust across instance types, with cluster moves helping it avoid trapping in structured landscapes. The hybrid quantum-inspired PT-SQA can be competitive on selected instances, but its overhead keeps it behind the best classical advanced methods in the overall ranking.

Table 6.6 reports, for each benchmark instance, the median optimality gap $\overline{\theta} = \text{gap} \times 100\%$, the success rate Θ of matching the best-known solution across 10 independent runs, and the wall-clock time in seconds. Instances on which all solvers achieve $\overline{\theta} = 0$ in all 10 runs are considered easy and are omitted. Instances on which no solver succeeds across all 10 runs are highlighted in light gray. Newly found best-known solutions are marked with a star $*$ and their corresponding gap is negative. The bottom row counts how many times each solver achieved the best median gap, best success rate, and best time across all hard instances.

Problem	BKS	SAC			PA			PTC			PT-SQA		
		$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time
bur26b	3817852	**0.00**	**10/10**	1.2	**0.00**	**10/10**	0.2	**0.00**	**10/10**	**0.1**	**0.00**	9/10	0.3
bur26f	3782044	**0.00**	9/10	1.4	**0.00**	**10/10**	0.3	**0.00**	**10/10**	**0.2**	**0.00**	**10/10**	2.2
chr15a	9896	**0.00**	**10/10**	0.1	**0.00**	8/10	0.1	**0.00**	**10/10**	**≈0.0**	**0.00**	**10/10**	0.1
chr15c	9504	**0.00**	**10/10**	0.1	**0.00**	9/10	≈0.0	**0.00**	**10/10**	**≈0.0**	**0.00**	**10/10**	0.2
chr20a	2192	**0.00**	**10/10**	**0.3**	0.18	4/10	1.0	**0.00**	**10/10**	0.6	**0.00**	**10/10**	2.2
chr20b	2298	2.00	3/10	**0.7**	4.18	1/10	1.1	**0.00**	**6/10**	2.5	1.00	5/10	4.6
chr20c	14142	**0.00**	**10/10**	0.3	**0.00**	9/10	0.1	**0.00**	**10/10**	**0.1**	**0.00**	**10/10**	0.2
chr22b	6194	**0.00**	**10/10**	0.7	**0.00**	9/10	**0.2**	**0.00**	**10/10**	0.3	**0.00**	9/10	2.9
chr25a	3796	**0.00**	9/10	0.6	**0.00**	6/10	**0.2**	**0.00**	**10/10**	0.3	**0.00**	**10/10**	3.0
dre21	356	**0.00**	**10/10**	0.3	**0.00**	9/10	0.1	**0.00**	**10/10**	**0.1**	**0.00**	**10/10**	0.5
dre24	396	**0.00**	**10/10**	0.5	**0.00**	8/10	**0.1**	**0.00**	**10/10**	0.5	**0.00**	**10/10**	0.9
dre28	476	**0.00**	**10/10**	0.6	**0.00**	6/10	**0.3**	**0.00**	**10/10**	0.5	**0.00**	7/10	8.1
dre30	508	**0.00**	8/10	**0.9**	21.85	2/10	1.2	**0.00**	**10/10**	1.1	24.41	1/10	10.0
dre42	764	**17.02**	2/10	2.5	26.57	0/10	**1.7**	21.47	**3/10**	8.7	36.52	0/10	19.4
dre56	1086	**41.44**	0/10	4.1	41.44	0/10	**2.7**	45.21	0/10	14.9	58.93	0/10	35.7
dre72	1452	**45.87**	0/10	7.5	46.69	0/10	**3.9**	47.59	0/10	28.5	65.70	0/10	61.8
dre90	1838	56.91	0/10	12.6	**52.67**	0/10	**5.8**	57.18	0/10	49.6	76.44	0/10	102.6
dre110	2479	45.66	0/10	21.2	**44.41**	0/10	**8.9**	47.36	0/10	89.7	64.34	0/10	158.6
dre132	3022	50.17	0/10	35.6	**45.27**	0/10	**14.4**	49.47	0/10	149.2	67.24	0/10	236.5
els19	17212548	**0.00**	**10/10**	0.6	**0.00**	9/10	0.1	**0.00**	**10/10**	**0.1**	**0.00**	**10/10**	1.2
esc32a	130	**0.00**	**10/10**	2.2	**0.00**	**10/10**	**0.2**	**0.00**	9/10	0.9	1.54	4/10	11.8
esc32b	168	4.76	4/10	2.3	**0.00**	**10/10**	**0.2**	8.33	3/10	4.9	**0.00**	8/10	4.8
esc128	64	4.69	1/10	61.1	**0.00**	**10/10**	**4.6**	6.25	0/10	120.0	**0.00**	**10/10**	5.3

Problem	BKS	SAC			PA			PTC			PT-SQA		
		$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time
lipa80a	253195	**0.00**	**6/10**	7.5	0.50	4/10	**5.5**	0.43	2/10	46.6	0.57	1/10	90.8
lipa90a	360630	**0.17**	**5/10**	10.4	0.45	4/10	**6.7**	0.44	3/10	61.4	0.37	4/10	115.0
rou20	725522	**0.00**	**10/10**	0.1	**0.00**	7/10	**0.1**	**0.00**	**10/10**	0.3	**0.00**	**10/10**	0.2
sko49	23386	**0.00**	**10/10**	3.4	**0.00**	8/10	**0.9**	**0.00**	**10/10**	1.9	**0.00**	9/10	14.8
sko72	66256	**0.00**	6/10	10.3	**0.00**	**10/10**	**2.9**	**0.00**	**10/10**	14.9	0.01	1/10	60.3
sko81	90998	**0.00**	**9/10**	13.7	**0.00**	6/10	**4.6**	**0.00**	8/10	29.5	0.01	3/10	81.0
sko90	115534	**0.00**	8/10	17.5	**0.00**	**10/10**	**5.9**	**0.00**	**10/10**	26.4	0.01	3/10	99.1
sko100a	152002	**0.00**	8/10	24.5	**0.00**	8/10	**8.4**	**0.00**	**10/10**	30.2	0.01	4/10	126.7
sko100b	153890	**0.00**	**10/10**	24.6	**0.00**	8/10	**8.7**	**0.00**	**10/10**	26.4	**0.00**	8/10	73.1
sko100c	147862	≈0.0	3/10	24.8	≈0.0	4/10	**10.7**	**0.00**	**6/10**	56.7	≈0.0	3/10	126.3
sko100d	149576	0.01	0/10	25.2	**0.00**	**6/10**	**8.9**	0.01	2/10	72.6	0.01	1/10	126.9
sko100e	149150	**0.00**	**10/10**	24.2	**0.00**	**10/10**	**8.6**	**0.00**	**10/10**	15.9	**0.00**	7/10	85.5
sko100f	149036	**0.00**	**10/10**	24.6	**0.00**	**10/10**	**8.6**	**0.00**	**10/10**	27.4	0.01	4/10	127.0
tai20a	703482	**0.00**	6/10	**0.2**	0.30	1/10	1.1	**0.00**	8/10	1.2	**0.00**	**10/10**	1.0
tai25a	1167256	**0.00**	7/10	**0.5**	0.38	4/10	1.2	**0.00**	**9/10**	1.5	0.15	4/10	8.6
tai30a	1818146	0.01	5/10	**1.3**	0.39	2/10	1.4	**0.00**	**6/10**	1.6	0.10	4/10	11.8
tai35a	2422002	0.33	**4/10**	1.6	0.71	1/10	**1.6**	**0.22**	**4/10**	6.1	0.43	1/10	16.5
tai40a	3139370	0.54	0/10	2.3	0.70	0/10	**1.8**	**0.35**	**1/10**	7.6	0.71	0/10	22.0
tai50a	4938796	0.99	0/10	3.0	1.08	0/10	**2.3**	**0.90**	0/10	12.3	1.08	0/10	34.1
tai60a	7205962	**0.98**	0/10	3.4	1.06	0/10	**2.9**	1.03	0/10	17.6	1.25	0/10	52.6
tai80a	13499184	1.17	0/10	6.2	**1.10**	0/10	**5.3**	1.13	0/10	35.1	1.65	0/10	106.1
tai100a	21044752	1.12	0/10	14.2	**0.99**	0/10	**8.9**	1.04	0/10	79.7	1.52	0/10	206.4
tai15b	51765268	0.18	1/10	1.0	**0.00**	**10/10**	**0.1**	0.20	2/10	2.3	**0.00**	**10/10**	0.3

Problem	BKS	SAC			PA			PTC			PT-SQA		
		$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time	$\overline{\theta}$	Θ	Time
tai30b	637117113	**0.00**	6/10	3.0	**0.00**	**10/10**	0.7	**0.00**	**10/10**	**0.3**	**0.00**	**10/10**	0.8
tai40b	637250948	**0.00**	8/10	4.8	**0.00**	**10/10**	1.3	**0.00**	**10/10**	**0.4**	**0.00**	**10/10**	0.8
tai50b	458821517	0.02	0/10	7.3	**0.00**	**10/10**	**2.4**	≈0.0	3/10	13.9	≈0.0	4/10	35.6
tai60b	608215054	0.04	0/10	10.4	**0.00**	9/10	**4.4**	≈0.0	0/10	21.1	**0.00**	**10/10**	17.0
tai80b	818415043	0.05	0/10	20.5	**0.00**	**8/10**	**11.5**	0.02	0/10	47.4	**0.00**	7/10	86.4
tai100b	1185996137	0.06	0/10	35.4	**0.00**	**8/10**	**24.2**	0.01	0/10	90.2	0.03	2/10	211.7
tai150b	498896643	0.09	0/10	137.0	**≈0.0**	**4/10**	**94.3**	0.06	0/10	303.4	0.10	0/10	532.8
tai256c	44759294	0.08	0/10	**577.8**	0.09	0/10	588.5	**0.08**	0/10	1389.1	0.10	0/10	2033.5
tai45e01	6412	0.05	3/10	4.6	**0.00**	**10/10**	0.8	**0.00**	**10/10**	**0.3**	0.16	5/10	23.0
tai75e01	14488	3.39	0/10	15.0	2.68	**4/10**	**6.5**	**0.03**	2/10	29.4	3.54	0/10	68.3
tai125e01	35426	2.55	0/10	**50.9**	**0.08**	**3/10**	53.3	0.35	0/10	115.8	2.72	0/10	213.0
tai175e01	59732	0.62	0/10	121.0	**−0.20***	**10/10**	38.7	**−0.33***	**10/10**	**21.1**	5.08	0/10	458.3
tho40	240516	0.01	3/10	2.2	0.01	0/10	**2.0**	**0.00**	**6/10**	4.5	0.01	5/10	14.8
tho150	8133398	≈0.0	1/10	62.7	**≈0.0**	**3/10**	**32.6**	0.01	0/10	218.1	0.01	0/10	336.7
wil50	48816	**0.00**	**10/10**	3.7	**0.00**	9/10	**1.0**	**0.00**	**10/10**	1.6	**0.00**	**10/10**	4.1
wil100	273038	**0.00**	7/10	25.4	≈0.0	3/10	**11.1**	**0.00**	**9/10**	32.8	≈0.0	2/10	128.6
# of Best		35	21 (302)	8	**41**	22 (344)	**43**	40	**35** (352)	11	25	17 (295)	0

Table 6.6: QAPLIB results for 62/150 hard instances. Metrics per instance: median gap $\theta = \text{gap} \times 100\%$; success rate Θ out of 10 runs; wall-clock time (s). Light gray rows: no solver found the BKS in any run. ∗: new best-known solution (negative gap). Bottom row: win counts per metric across all hard instances.

6.1.7 Comparison to External Solvers

To contextualize the performance of our GPU-accelerated annealing algorithms, we benchmark them against a diverse set of established QAP solvers:

- **FAQ**: a gradient-descent approximation algorithm provided by *SciPy*;
- **2-opt**: a standard local search heuristic, also accessed via *SciPy*;
- **D-Wave**: a commercial proprietary nonlinear solver for QAPs [94]; and
- **Tabu Search**: a metaheuristic that forbids revisiting recent solutions.[5]

This comparison is not hardware-normalized: our GPU solver and the CPU baselines differ in throughput and energy consumption. We report it as a practical time-to-solution comparison against widely available software, not as a claim of algorithmic superiority per compute unit. While GPU-accelerated variants of metaheuristics like Tabu Search exist in the literature, our objective is to benchmark against the established, readily available software ecosystem.

We evaluate all solvers[6] with a timeout of 150 seconds (restart if time left) and report the median of 10 runs.

As shown in Figure 6.17, Tabu Search is the strongest external baseline across all three Taillard families. It remains the most robust classical comparator in this chapter, while faster methods such as 2-opt and FAQ are competitive only when raw speed matters more than solution quality.

Figure 6.18 compares the best external solver (Tabu Search on CPU) against our GPU-accelerated annealing methods under the same 150-second time limit. We find that PA, in particular, is the strongest method overall on these benchmarks and consistently outperforms Tabu Search on nearly all instance families and sizes.

[5] Available at mistic.iict-heig-vd.ch/taillard/codes.dir/tabou_qap2.c [35]. Modified to run for a maximum time instead of a maximum number of iterations, while keeping the other defaults ($\text{tabu}_{\text{duration}} = 8n$ and $\text{aspiration} = 5n^2$).

[6] D-Wave kindly provided their results on request: github.com/dwavesystems/dwave-optimization/issues/546. We thus did not re-run their experiments and only provide their reported median gaps.

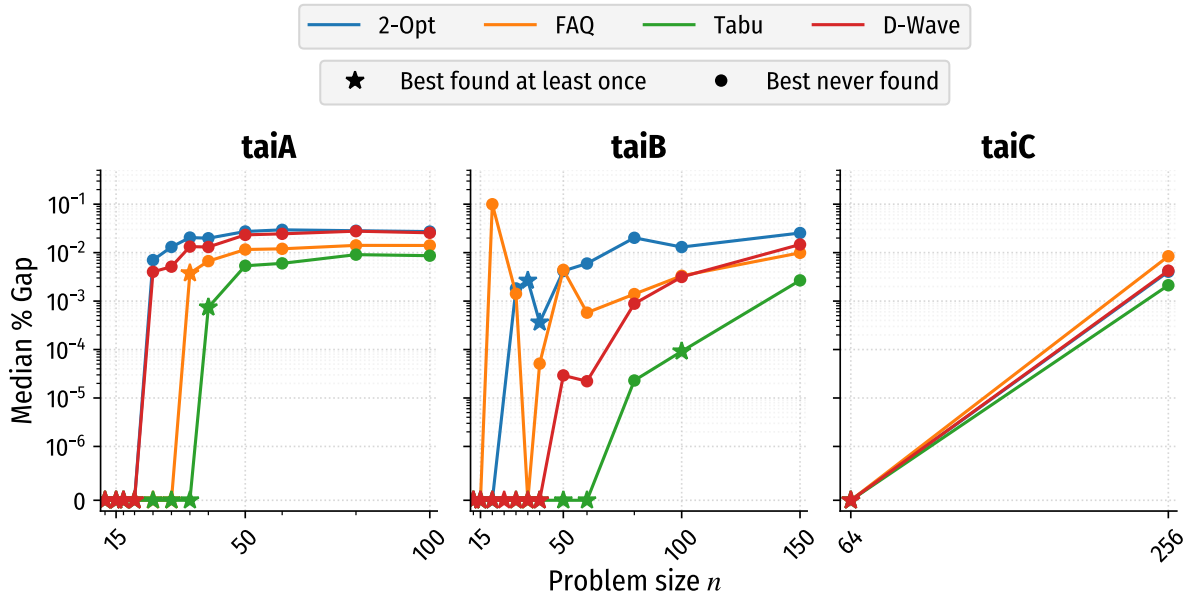


Figure 6.17: Median optimality gap of four external solvers: FAQ (SciPy gradient descent), 2-opt (local search), D-Wave NL (proprietary commercial solver), and Tabu Search (Taillard's CPU implementation). All three Taillard families have a strict 150-second wall-clock limit, with restarts permitted. Tabu Search is the strongest external baseline across all families. Note that the Tabu Search implementation is by Taillard [35] , the creator of the `tai` benchmark instances, and may therefore be especially well-tuned for this family.

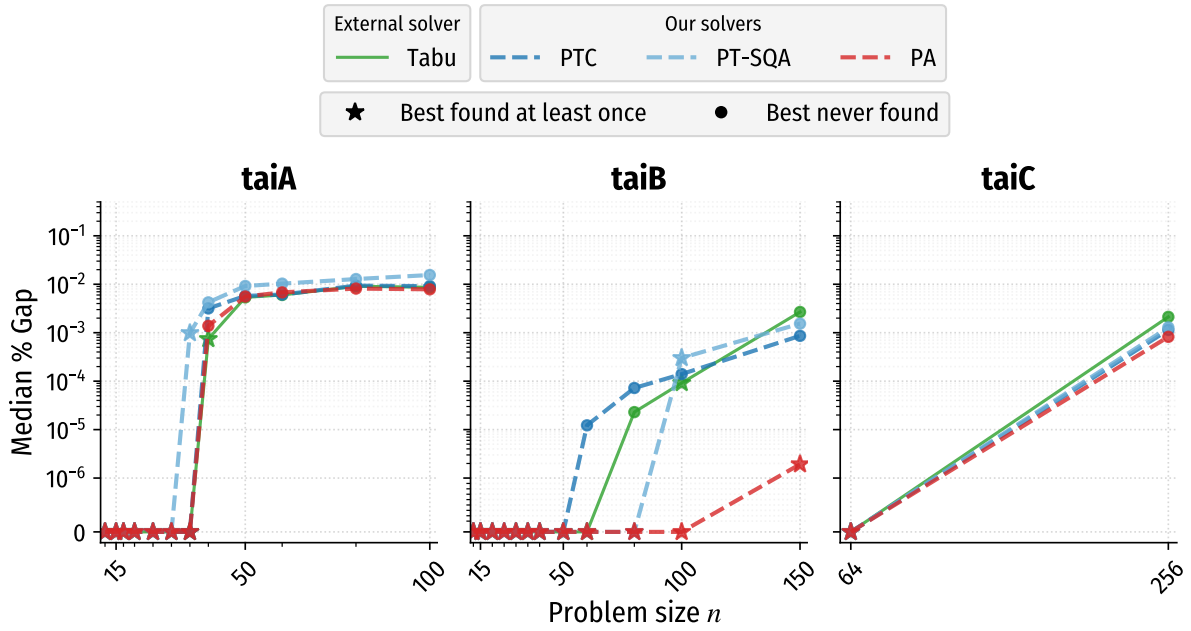


Figure 6.18: Head-to-head comparison of the best external solver (Tabu Search) against our GPU-accelerated annealing methods (PA, PTC, PT-SQA) under an identical 150-second wall-clock limit across all Taillard families. PA is the overall strongest method, consistently outperforming Tabu Search across nearly all instance families and sizes, with `tai40a` as the main exception. The results establish that GPU-native permutation annealing is practically competitive with the best available CPU-based classical metaheuristics under strict time budgets.

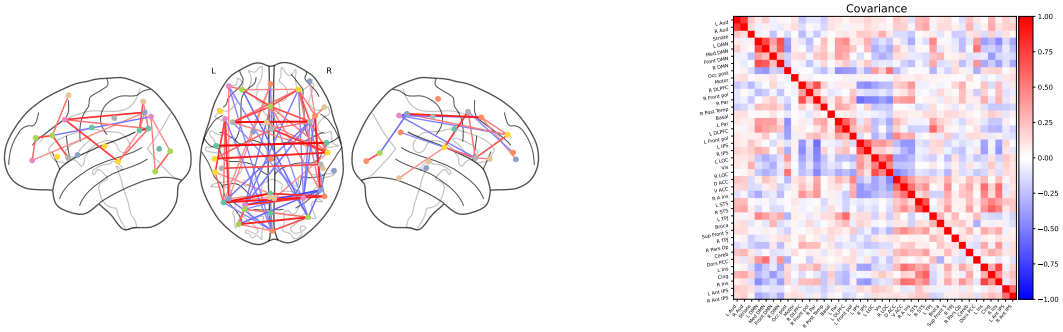
6.2 Human Brain Connectomes

We benchmark using brain scans from [95] and select one representative subject pair for the main 1000-Region of Interest (ROI) evaluation. The MRI data [96] consist of functional scans of children aged 3–12 years and adults while they were watching a short animated film. In this work, we use the data purely as a graph-matching benchmark; however, the same framework could in principle transfer to other functional connectome settings, for example to compare healthy subjects with patient groups and thereby detect structural or functional deviations, such as those associated with e.g., Alzheimer's disease.

Since graph matching is represented by a permutation, its disagreement could be interpreted via swap distance: the number of atlas regions that are matched to different counterparts between two scans [97].

The Schaefer atlas is used to define parcellations with $n \in \{100, 400, 1000\}$ ROIs [98].

Nilearn is used to fetch the data, load the atlas, and extract the functional connectomes, i.e. it performs the neuroimaging preprocessing pipeline for us [99].



(a) Connectome **(b)** Adjacency Matrix

Figure 6.19: Example connectome: the left side shows a functional brain connectome for one subject, visualized as a graph overlaid on a cortical surface using the Schaefer 1000-ROI atlas. Nodes represent brain regions (ROIs), and edge weights encode functional connectivity. Right side shows the corresponding 1000×1000 covariance matrix $\boldsymbol{A}$, color-coded by connectivity strength. The graph matching task seeks a permutation that aligns two such matrices by minimizing their Frobenius distance.

We focus on the 1000-ROI connectomes in the main evaluation, since they represent the intended large-scale setting motivating this benchmark. Smaller 100- and 400-ROI instances were used only for preliminary tuning.

For GPU execution, we convert the connectome matrices to `Float32`. This substantially reduces memory overhead and speeds up the solver kernels. We repeat each experiment across multiple random seeds and report solution quality together with wall-clock time.

All annealing solvers use the fully cached variant, since it substantially reduces per-sweep runtime. On the smaller 100- and 400-ROI connectomes this showed an acceptable performance trade-off.

- **SA**: 50 replicas, geometric temperature and 200 sweeps.
- **MA**: 50 replicas, geometric temperature, linear penalty with 5% dropout, 200 sweeps, no pseudo-global sweep, no full global sweep.
- **PT**: fixed linear temperature ladder with 50 replicas, evaluated via 5/10/15/20-minute checkpoints instead of a fixed 200-sweep run.[7]
- **PA**: 100 replicas with 200 sweeps for geometric schedule and adaptive schedule until reaching the final temperature.
- **FAQ**: 200 maximum iterations with 10 different random initializations.

We additionally report SA with adaptive caching as a comparison point, while MA, PT, and PA are evaluated only in the fully cached setting.

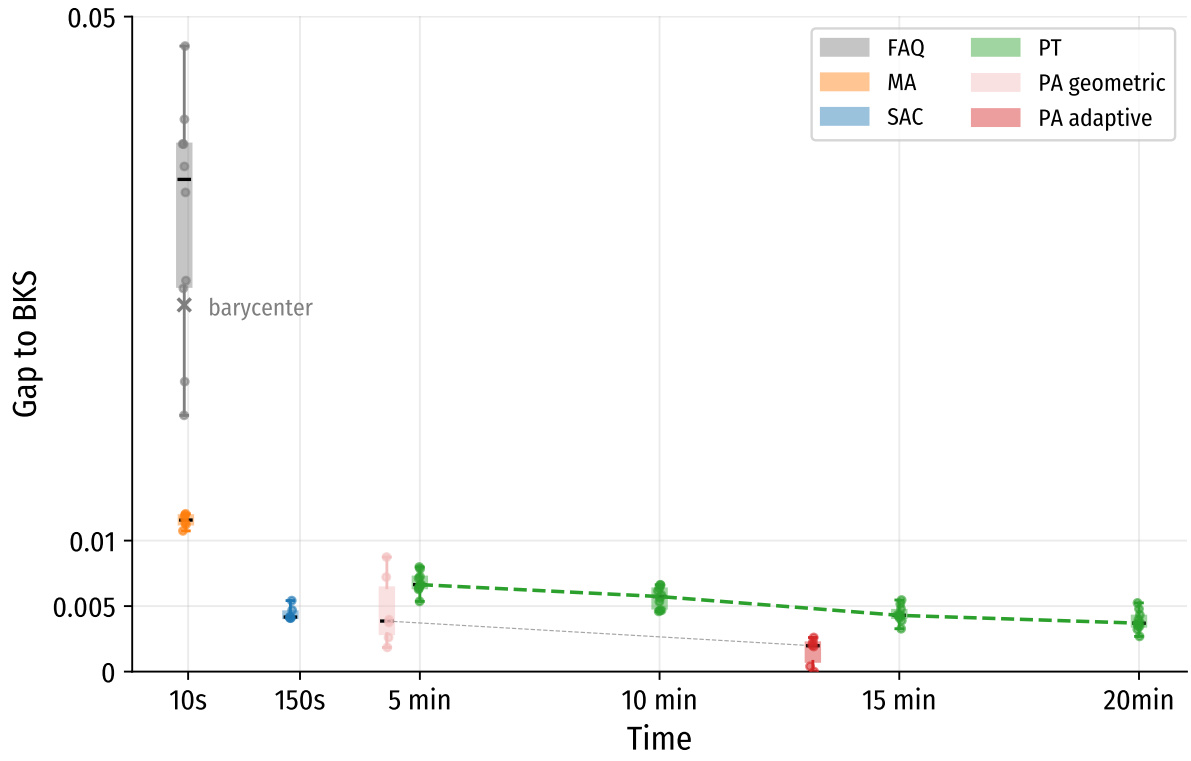


Figure 6.20: Benchmark results for the 1000-ROI functional connectome graph matching task. Reporting the optimality gap (relative to the best solution found by adaptive PA) and wall-clock runtime. MA and FAQ both complete in approximately 10 seconds, but MA achieves a consistently better gap. SA with caching serves as the single-chain baseline. PA with adaptive scheduling achieves the best final solution quality at moderate runtime cost; the geometric-schedule variant is faster but yields a larger gap. All annealers land within approximately 1% of the best-found solution.

[7] Linear ladder showed more balanced replica exchanges than geometric ones. We excluded the adaptive schedule and cluster moves because both substantially increased wall-clock time on the 1000-ROI connectomes.

Figure 6.20 validates the $n = 1000$ scalability claim.
The 1% convergence band across all methods suggests the problem is well-posed but hard: no solver dramatically dominates, and the gap between MA (10 seconds) and adaptive PA (15 minutes) represents a genuine quality-speed tradeoff rather than a dominance relationship.
PT with 50 replicas develops a swap-acceptance bottleneck between replica indices in range 30–40, indicating a coarse ladder in that temperature range. Increasing to 100 replicas resolves the bottleneck but raises per-sweep cost, degrading overall 20-minute performance. A targeted adaptive ladder tuned to this instance would likely close both issues.

7 Conclusion and Outlook

This thesis asked whether physics-inspired optimization dynamics (originally designed for binary spin systems) could be reformulated to operate directly in the natural permutation space of the QAP, bypassing the search-space explosion from $n!$ to 2^{n^2} and the fragile penalty terms of QUBO encodings. The answer is: yes, but with important caveats. Some solvers exceeded expectations; others revealed fundamental limitations. The `dre` instance family resisted all evaluated methods entirely.

Contributions: This thesis makes five main contributions:

1. Closing the permutation-native formulation gap of SQA, MA, and PT for the QAP.
2. Implementing these methods in `ASSA.jl`, a custom GPU-accelerated Julia solver framework built from scratch.
3. Introducing efficient $O(1)$ caching mechanisms: an extension of cached swap evaluation to non-vanishing diagonal matrices, and a novel side-conditioned cache for bipartite MA that is rebuilt only on side-switches. The latter was the key enabler for $n = 1000$ scalability.
4. Proposing new hybrid and ensemble mechanisms, in particular permutation cluster moves for PT and the PT-SQA architecture: the first hybrid combining tunneling with replica exchange for QAP.
5. Validating these ideas empirically on QAPLIB and connectome graph matching. This includes a new best-known solution for `tai175e01`, which was obtained under early stopping, so further improvement likely remains.

Findings: Addressing the three research questions:

1. *"How can spin-glass dynamics be reformulated for integer permutations?"*

The Suzuki–Trotter decomposition of the transverse Ising model extends naturally to permutations by replacing binary spin agreements with Hamming distances between adjacent Trotter slices. The bipartite structure of MA transfers directly via the asymmetric QAP cost. In both cases, the permutation constraint is maintained implicitly through swap-based moves, eliminating the need for additional penalty terms.

2. *"How does quantum-inspired tunneling compare to classical thermal fluctuations?"*

Standalone SQA does not consistently outperform SA within equivalent computational budgets, primarily due to hyperparameter sensitivity and the coarse Trotter discretization enforced by runtime constraints. However, the self-adaptive coupling schedule successfully induces repeated tunneling phases, confirming the validity of the underlying mechanism. When combined with PT, the tunneling dynamics of SQA provide a

qualitative advantage: PT-SQA outperforms standalone SQA and is competitive with the D-Wave commercial solver on `taiB` instances under strict time limits. However, this advantage is highly landscape-dependent: `taiB` instances probably have high but narrow barriers where tunneling-like dynamics can provide a qualitative benefit over purely thermal methods. The unstructured `taiA` family shows no consistent improvement from tunneling, which itself is informative: it suggests that the barriers in `taiA` are broad rather than narrow, a regime where thermal fluctuations are at least as effective as tunneling-like dynamics.

3. *"How can permutation-based annealing be efficiently parallelized on GPUs?"*

The SoA design of `ASSA.jl`, combined with warp-level reductions, enables efficient GPU utilization across all solver families. The round-robin tournament scheme for MA enables conflict-free parallel sweep evaluation. Correctness is verified after every solve via full energy recomputation. Informally, preliminary tests on a workstation machine with an A100 GPU (not the primary evaluation hardware) suggest the framework can sustain approximately 500 replicas on `tai150b` without wall-clock penalty, indicating significant headroom beyond the 100-replica baseline.

Limitations: First, the dre instance family resisted all evaluated solvers, suggesting a qualitative landscape difference that 2-opt-based dynamics alone cannot bridge.
Second, MA's solution quality on QAPLIB is consistently below the SA; the narrow symbiosis regime between temperature and penalty schedule does not generalize across instance families without PT2D, which remains computationally prohibitive at scale. Third, the best-known solution for `tai175e01` was obtained under early stopping: a dedicated extended run without this constraint is needed to confirm whether 59286 is a tight bound. And last, the ring and torus-SQA extensions, do not reliably improve performance due to insufficient Trotter resolution and hyperparameter fragility.

Practical Solver Selection: While solver performance depends on instance structure and runtime budget, the experimental results suggest the following heuristic starting points (Figure 7.1):

- In the connectome experiments, **MA** is the fastest method; its solution quality is typically above FAQ but below our other annealers.
- **SA** benefits from caching once the acceptance rate falls below a certain threshold and thus serves as the backbone of PT and PA.
- **PT** is particularly attractive for long-running optimizations with fixed time budgets because its upfront adaptive temperature-ladder cost can be amortized over long search horizons. However, the current ladder-search procedure becomes expensive on larger instances, so future work should move this adaptation online, allowing replicas or temperatures to be inserted and removed during the run.
  - Path Relinking seems to outperform cycle swaps as a PT cluster move on the `tai` family, but its sequential nature makes it costly for larger instances and thus best applied sparingly.
- **PT-SQA** also looks promising as a cluster-move hybrid, especially on `taiB`, although standalone **SQA** remains relatively slow.

- **PA** with adaptive temperatures achieved the best solution quality among the tested methods, at moderate runtime cost.

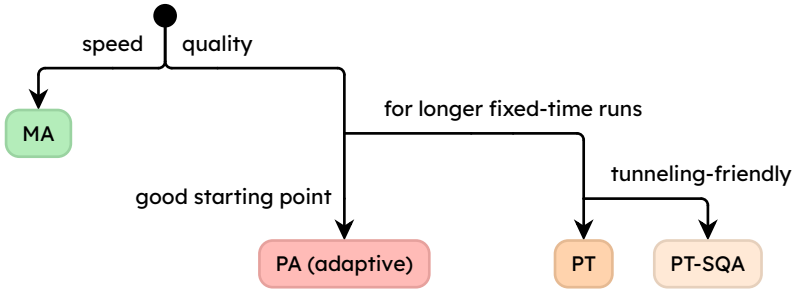


Figure 7.1: Heuristic starting points suggested by our experimental results.

7.1 Future

Among several open questions, one concrete question is whether the number of Trotter slices required for effective QAP optimization with SQA should grow with problem size. Understanding how the required slice number P scales with problem size n could help clarify the limits of the framework.
Analyzing spin-overlap distributions [100] or fitness-distance correlation [37] may help identify instance classes where cluster moves or tunneling are effective. High overlaps or strong fitness correlations suggest a smoother effective landscape with fewer barriers, which in turn may favor tunneling-based methods.

To explore such hypotheses comprehensively, one needs benchmarks that systematically cover a wide range of sizes and structural regimes. The instances by Saifullah Hussin and Stützle [36] are well suited for this purpose: they offer three distinct families, each with QAP sizes up to $n = 1000$ and different sparsity levels, making them useful for comparing not only solver performance but also the behavior of hybrid extensions across instance families. At the same time, their lack of BKS makes them less convenient for direct comparison against exact references.

The permutation-native approach developed here opens several concrete directions: seeded graph matching via GQAP constraints, QAP-aware transverse fields that weight tunneling by the distance matrix, the SQA-MA crossover for massive-scale instances, and more exploratory hybrids such as PT2D combined with MA or Path Relinking cluster moves. Most importantly, the modular architecture of `ASSA.jl` allows some of these ideas to be combined and evaluated systematically in future research. In the following subsections, we discuss some of these ideas in turn.

7.1.1 Seeded Graph Matching

The connectome experiments showed that our solvers scale to $n = 1000$, but the biological constraint that only intra-species pairings are valid was not yet exploited. Seeded graph matching is a variant of graph matching in which some vertex correspondences are fixed a priori, so only the remaining entries of the permutation need to be determined.

A particularly promising use-case of our solvers is the application to the Graph Alignment (GA) step of the GA-IPA (iterative pairing algorithm) pipeline [101]. Gandarilla-Perez et al. [101] currently solve the GA step via SA. Replacing SA with a more powerful solver could yield better matchings for the IPA step.

Crucially, only intra-species pairings are valid, which directly restricts the swap neighborhood [101]. Two concrete extensions to our solver could enable this seeded graph matching [102] application:

(i) introducing the C-matrix of the GQAP with infinite penalties for inter-species pairings, thereby both encoding seeded interaction pairs and enforcing the biological constraint that only intra-species pairs are valid; or alternatively
(ii) directly restricting the swap neighborhood to only propose exchanges within the same species, mirroring the algorithm of [101].

Approach (ii) is arguably preferable in practice: rather than relying on penalty costs to implicitly forbid invalid swaps, the search space is kept clean from the start as already-seeded pairs would never be proposed for swapping. However, approach (i) would likely be easier to add to the current code infrastructure and would make `ASSA.jl` more broadly applicable across GQAP formulations.

7.1.2 SQA: QAP-aware Transverse Field

As shown, SQA's tunneling advantage is sensitive to the coupling schedule and is not uniform across instance families. One potential explanation is that the standard Hamming-distance coupling (as in Equation 4.14) treats all position mismatches equally, regardless of the underlying distance structure. A QAP-aware transverse field addresses this by weighting the inter-slice penalty by the distance matrix B:

$$\mathcal{H}_{d+1} \Rightarrow \sum_{k=1}^{P} z\left(\boldsymbol{\pi}^{[k]}\right) + \lambda \sum_{i}^{n} \frac{\boldsymbol{B}_{\boldsymbol{\pi}_i^{[k]}\boldsymbol{\pi}_i^{[k+1]}} + \boldsymbol{B}_{\boldsymbol{\pi}_i^{[k+1]}\boldsymbol{\pi}_i^{[k]}}}{2}. \tag{7.1}$$

This formulation differs from the standard coupling: instead of uniformly penalizing any mismatch between Trotter slices, it uses the distance matrix B to weight mismatches. By making the penalty symmetric, it naturally handles cases where the distance matrix is asymmetric.

The intuition is that this would encourage tunneling to spatially closer (low-distance) configurations instead of merely lowering the Hamming distance between the permutations. While preliminary tests of this formulation did not yield immediate performance improvements, the concept remains theoretically promising.

Future work could explore whether this heuristic proves beneficial on specific instance classes, particularly those where the coupling matrix is highly sparse. Furthermore, because the QAP objective is symmetric with respect to the flow matrix A and the distance matrix B, their roles can be freely swapped. In instances where B is dense but A is sparse, applying this distance-aware coupling to the flow matrix instead might yield better results.

7.1.3 SQA and MA Crossover

MA's side-conditioned cache was the key enabler for $n = 1000$ scalability, but MA's solution quality on QAPLIB remained disappointing. SQA, by contrast, showed tunneling benefits on `taiB` at the cost of coarse Trotter discretization. Their structural similarity suggests a hybrid that could combine MA's cache efficiency with SQA's tunneling dynamics. Combining both ideas yields a ring of M MA-style replica pairs:

$$\mathcal{H}_{\mathrm{MA}+1} = \sum_{k=1}^{P} \mathrm{tr}\big(\boldsymbol{A} X_k \boldsymbol{B} X_{k+1}^{\top}\big) + \lambda\big(n - \mathrm{tr}\big(X_k X_{k+1}^{\top}\big)\big) \tag{7.2.a}$$

$$= \sum_{k=1}^{P} z_{\mathrm{MA}}\Big(\boldsymbol{\pi}^{[k]}, \boldsymbol{\pi}^{[k+1]}\Big) + \lambda g\Big(\boldsymbol{\pi}^{[k]}, \boldsymbol{\pi}^{[k+1]}\Big), \tag{7.2.b}$$

where each replica interacts with its neighbor via the asymmetric MA cost rather than the symmetric QAP cost. This formulation allows the bipartite parallelism of MA to propagate across a Trotter-like imaginary-time dimension. It also makes each slice interact with both the left and right of $\boldsymbol{B}$, which may be advantageous when $\boldsymbol{B}$ is asymmetric.

For massive QAP instances, this SQA-MA hybrid could potentially replace SQA, leveraging bipartite parallelism to overcome traditional scaling bottlenecks. Exploring this regime is a promising direction for future high-performance computing research.

PT2D on MA Torus with Caching: An ambitious hybrid would apply PT2D on top of the MA torus, yielding a two-dimensional replica-exchange scheme over the MA manifold. The MA structure would handle local mixing within each replica pair, while exchange along both toroidal directions could improve barrier crossing. To remain practical, this variant would need to preserve the $O(1)$ caching strategy.

7.1.4 `ASSA.jl` Improvements

On the software side, the modular nature of the heuristics explored in this work naturally opens the door to numerous hybrid algorithms. A `SolverOrchestrator` abstraction would make the framework substantially more versatile. The small `SolverBase` interface already allows interchangeable heuristics. The `SolverOrchestrator` could manage the execution flow, allowing different heuristics to be combined in a modular way, e.g., by alternating between SA and MA sweeps, or by applying Tabu search after a certain number of PT steps. The strong performance of Tabu Search, even as a single CPU replica, suggests that a GPU-native tabu variant would be a worthwhile future extension of `ASSA.jl`.

One practical limitation of the current `ASSA.jl` implementation is that GPU and CPU support must be updated separately for each algorithm. A future `KernelAbstractions.jl` [88] rewrite would eliminate much of this duplication by providing a single backend-agnostic kernel definition, while still allowing optimized execution on different GPU backends and multi-core CPUs.

Epilogue

Inspired by quantum annealing approaches to combinatorial optimization, this thesis eventually led me to the QAP. My earlier work had focused mainly on annealers for Ising spin-glass models, where optimization problems are first reformulated in binary form.

The QAP is structurally similar in that it also has a quadratic objective, but it differs in an important way: the decision variable must be a permutation matrix with one-hot constraints on every row and column. A direct QUBO encoding therefore turns a clean permutation problem into a much larger binary one, where feasibility must be enforced through penalty terms rather than through the search dynamics themselves.

For this reason, I chose a permutation-native formulation instead. Rather than encoding the constraints into an oversized binary space, I adapted simulated quantum annealing and related methods so that they operate directly on feasible permutations. In this view, multiple replicas play the role of an extended state that gradually collapses toward a single permutation, which can be interpreted as a heuristic analogue of tunneling through energy barriers.

This route is not without drawbacks. The multi-replica structure of SQA introduces significant computational overhead, and the basic swap moves used in permutation annealing are inherently sequential in dense QAP instances.

The most serendipitous moment of the whole project came during a walk with my dog. The idea for the MA cache suddenly occurred to me: because MA fixes one side of the bipartite permutation while optimizing the other, the cache remains valid for the entire sweep, no incremental updates needed. I came home, worked out the formulas, verified they gave correct results, and then integrated the scheme into the codebase. Only then did the full implication become clear: the cache could be rebuilt just once per side-switch, and be exploited aggressively for every single swap evaluation in between. This is far more powerful than the SA-style cache, which must be updated after every accepted move. This became one of the most satisfying technical insights of the project.

Before this cache was introduced, MA did not yet show the large performance gains that later made it so compelling. That also explains why the SQA–MA crossover was not explored as deeply: the hybrid did not yet have the same practical appeal without the cache-accelerated MA backbone.

The cache also shaped the algorithmic design in ways I did not fully anticipate. Because it strongly rewards local sweeps, the dropout mechanism becomes the natural tool for re-introducing exploration (occasionally decoupling the two sides to prevent premature freeze) rather than the more expensive global sweep schemes I had relied on before.

Not every discovery was rewarding. MA's solution quality on QAPLIB remains disappointing relative to the other annealers. The narrow window in which temperature and

penalty must be in symbiosis is genuinely hard to hit, and most of the optimization collapses into a single short phase. The speed advantage is real and substantial (and on the large connectomes it matters) but the quality gap is frustrating. I suspect a dropout ladder combined with PT2D could help, but this remains untested.

PA exceeded all my expectations. I initially adopted it mainly because its independent-replica structure maps cleanly to GPU execution. But its strength runs deeper: PA sits at the intersection of simulated annealing and genetic algorithms (GA). The temperature schedule and Boltzmann weights are SA; the population and resampling step are GA; survival of the fittest, without crossover. This selection pressure continuously concentrates the population near promising solution candidates while independent SA runs between steps maintain diversity. The physics framing is thin, but the algorithmic idea is elegant. The quality advantage (especially on the `taiB` family) was a genuine surprise.

Looking back, the central lesson this work taught me is that physics analogies are most useful as design heuristics rather than performance guarantees. The collapse dynamics of SQA, the bipartite parallelism of MA, and the replica coupling of PT all contributed something. But which mechanism dominates depends heavily on the problem structure. The unstructured `taiA` instances resisted everything roughly equally; the structured `taiB` instances rewarded tunneling-like dynamics and population pressure. The `dre` instances defeated all evaluated methods entirely. Monotonic schedules freeze dynamics too early, and the narrow symbiosis regime of penalty-based methods motivates richer approaches like PT2D: not because the physics demands it, but because the optimization landscape does. That tension between physical intuition and empirical reality turned out to be the most interesting thing about this project.

A Appendix

A.1 Reservoir sampling on GPU

In the following we give the pseudocode for picking a random accepted swap as used in our SAC implementation.

GPU version of: filter(!iszero, flags) |> rand

```
function pick_random_nonzero(flags)
    stride = blockDim().x
    tid = threadIdx().x

    # each thread does local reservoir with some flags
    counts, candidates = zeros(Int, stride), zeros(Int, stride)
    for i in tid:stride:length(flags)
        flags[i] == 0 && continue # skip
        counts[tid] += 1
        if rand() < 1 / counts[tid]; candidates[tid] = flags[i]; end
    end

    total_counts = naive_parallel_cumsum(counts)
    picked_count = rand() * total_counts[end] |> ceil
    picked_flag = 0

    count_before = tid == 1 ? 0 : total_counts[tid - 1]
    if count_before < picked_count ⩽ total_counts[tid]
        picked_flag = candidates[tid]
    end

    return picked_flag
end
```

Algorithm A.1: Pseudocode for picking random nonzero on GPU

A.2 SA Always using Cache

We evaluate the performance of the fully cached SA approach on the largest instances of each Taillard family. To maintain comparable execution times with the baseline results from Figure 6.2, we roughly tailored the total number of sweeps for each instance, as shown in Table A.1.

Problem	**Geometric** *(time)*	**Inverse Linear** *(time)*	Reference *(time)*
tai100a	45000 *(52s)*	75000 *(50s)*	2400 *(52s)*
tai150b	15500 *(155s)*	17000 *(159s)*	2400 *(150s)*
tai256c	23000 *(703s)*	24000 *(712s)*	2400 *(649s)*

Table A.1: Number of sweep setup for SA cached.

Determining equivalent sweep counts is challenging because the cache's computational cost is dynamic and highly instance-dependent. For example, tai100a quickly falls into local minima and accepts very few moves, resulting in fast execution times per sweep due to few cache updates. Conversely, tai150b continuously explores and accepts new states, requiring frequent, expensive updates. Coupled with varying cooling schedules, this instance-specific acceptance rate makes it difficult to align runtimes using a fixed sweep count.

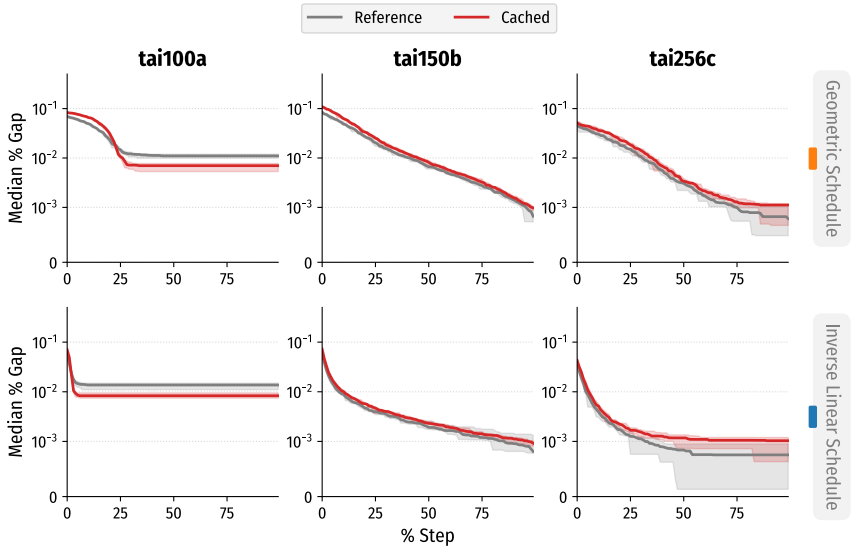


Figure A.1: Convergence comparison of non-cached and fully cached version.

As illustrated in Figure A.1, tai100a utilizes many more sweeps, allowing the system to cool much more slowly. This slower cooling trajectory likely contributes to its improved results, an observation that aligns well with the adiabatic theorem. The behavior of tai150b is highly comparable. In contrast, tai256c exhibits slightly inferior performance despite running for approximately 50 seconds longer than the baseline.
This underperformance might be explained by the instance's sparsity; it causes the cache to frequently sample and accept $\Delta E = 0$ moves, potentially hindering effective exploration.

A.3 SQA on Torus: Dimension + 2

To maintain a similar number of total slices as SA, we configure SQA with 6 replicas and 4×4 Trotter slices, resulting in a total of 96 slices per 1100-sweep run.

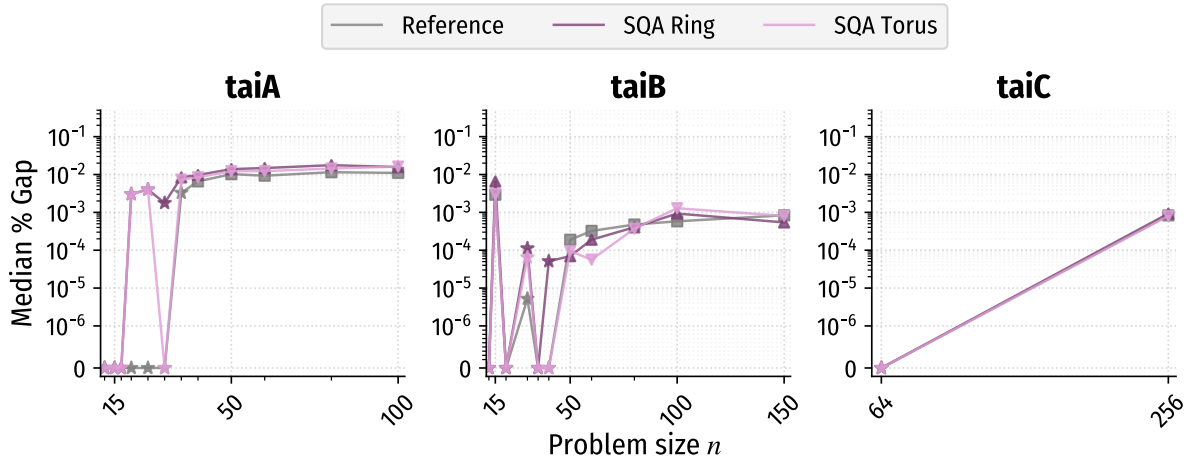


Figure A.2: SQA torus vs ring topology.

As illustrated in Figure A.2, drawing conclusions regarding the torus topology is difficult due to instance-dependent results. For instance, the $d + 2$ configuration achieves a lower gap on `tai60b`, but performs worse on `tai100b`.

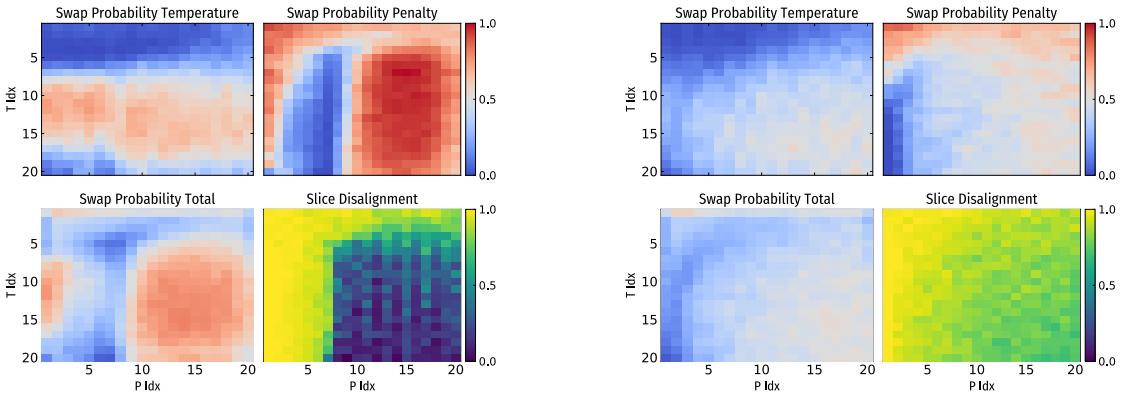
We observed that in two-dimensional time, the slices generally converge slightly faster and at a lower penalty threshold. This behavior is expected: in a $d + 2$ grid topology, each Trotter slice is coupled to four neighbors rather than just two in the $d + 1$ ring. This higher connectivity results in a stronger cumulative inter-slice penalty, forcing the system to collapse into a classical state faster.

These results serve primarily as a proof of concept. They demonstrate that a two-dimensional Trotter space can be successfully implemented and guided by a self-adaptive penalty mechanism. However, the theoretical advantage of higher-dimensional tunneling does not reliably translate into better optimization performance for the QAP.

This lack of improvement is likely due to the extremely coarse discretization of the torus. Constrained by computational overhead, the Trotter space was restricted to a minimal 4×4 grid. This low resolution is presumably insufficient to accurately simulate the quantum fluctuations needed to traverse broad energy barriers.

A.4 PT2D for MA

To resolve MA's hyperparameter fragility, PT2D over temperature and penalty is a natural solution. However, the current adaptive implementations require computationally expensive pre-sampling to generate the 2D grid, which massively inflates runtime.
We thus conducted preliminary experiments with a grid of geometric temperatures and linear penalties in Figure A.3.



(a) tai100a **(b)** tai150b

Figure A.3: Results of PT2D with geometric temperatures and linear penalties.

As seen in Figure A.3, the ideal slice misalignment (a smooth gradient from 1.0 to 0.0) is not achieved. On tai100a slices collapse too fast. Conversely, on tai150b, the slices remain decoupled (stuck near 0.6). Furthermore, the swap probabilities are highly heterogeneous, failing to maintain the desired ~ 0.4 acceptance rate, which demonstrates why static grids fail and computationally expensive adaptive grids are required.

Figure A.4 shows a representative adaptive grid. Compared with the static grid in Figure A.3, it yields a smoother slice-alignment gradient and more uniform swap rates, illustrating the potential benefit of adaptive scheduling. However, a more thorough study is beyond the scope of this thesis, as generating these schedules is computationally expensive and introduces additional parameter tuning, especially for the learning rates used by the adaptive schedule finder [103].

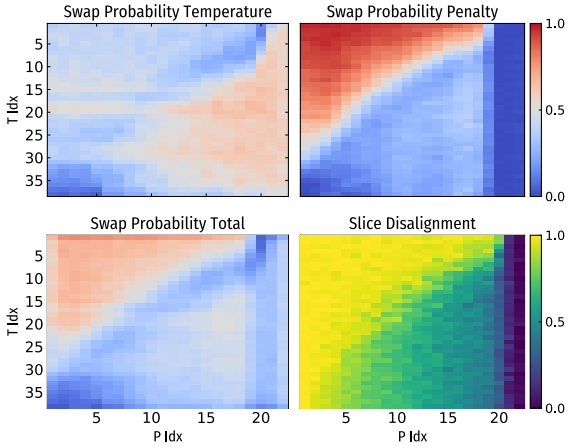


Figure A.4: Adaptive PT2D for tai100b.

A.5 Parameters for Adaptive Temperature Schedule in PT

Our implementation is heavily inspired by [49]. However, instead of stopping when reaching a minimum energy standard deviation σ_E, we stop when reaching our target inverse temperature β_{final}. We made this adjustment because we found it difficult to decide on a universally good threshold for σ_E, as it is highly problem-dependent. The

temperature bounds are again determined by the scheme presented in Section 4.2.2. We used a learning rate of 1.25 to achieve a swap-rate of approximately 0.4.

Parameter	Default Value	Description
Initial Inverse Temp	*Misevičius [56] scheme*	Starting point
Final Inverse Temp	*Misevičius [56] scheme*	Target point
Learning Rate	1.25	Temperature step size
Warmup Sweeps	200	Initial thermalization
Sampling Sweeps	500	Sweeps for std. dev.
Number of Chains	100	Parallel replicas

Table A.2: Parameters used to generate the adaptive temperature ladders for PT.

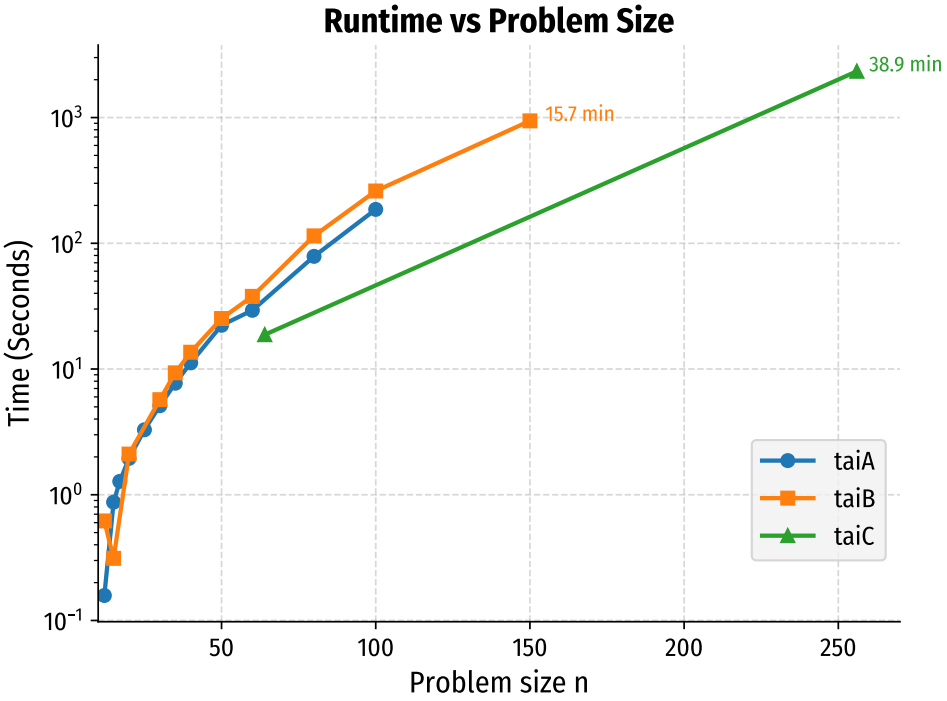


Figure A.5: Runtime for sampling adaptive temperatures for PT

The runtimes can be reduced by more aggressive usage of the cached variants and the `accept_only_one_j_per_i` toggle.

List of Acronyms

BKS **– Best Known Solution**

FAQ **– Fast Approximate QAP Algorithm**

GA **– Graph Alignment**

GMP **– Graph Matching Problem**

GQAP **– Generalized Quadratic Assignment Problem**

IQAP **– Independent Quadratic Assignment Problem**

MA **– Momentum Annealing**

PA **– Population Annealing**

PT **– Parallel Tempering**

PT2D **– Parallel Tempering in Two Dimensions**

PTC **– Parallel Tempering with cluster moves**

QA **– Quantum Annealing**

QAP **– Quadratic Assignment Problem**

QUBO **– Quadratic Unconstrained Binary Optimization**

ROI **– Region of Interest**

SA **– Simulated Annealing**

SAC **– Simulated Annealing with Caching**

SoA **– Structure-of-Arrays**

SQA **– Simulated Quantum Annealing**

TSP **– Traveling Salesman Problem**

List of Figures

Figure 2.1 Visualizing QAP objective quality 5
Figure 2.2 Search-space growth as a function of problem size 5
Figure 2.3 Graph matching applied to medical imaging 6
Figure 2.4 A small 2D Ising lattice 7
Figure 2.5 Two binary encodings for an integer variable over five values 10
Figure 4.1 Taxonomy of metaheuristic optimization strategies 15
Figure 4.2 A 2-opt swap applied to positions 3 and 7 of the permutation 16
Figure 4.3 Four cooling schedules 18
Figure 4.4 Comparison of classical and quantum escape mechanisms 21
Figure 4.5 From single SA-slice to SQA-slice topology 23
Figure 4.6 Ring topology of Trotter slices under periodic boundary conditions 23

Figure 4.7 Extension of SQA to a two-dimensional Trotter space . . . 24
Figure 4.8 Bipartite remapping in Momentum Annealing . . . 25
Figure 4.9 Round-robin tournament scheme for the MA local sweep . . . 28
Figure 4.10 Two novel global sweep types introduced for MA . . . 28
Figure 4.11 Escape mechanism of Parallel Tempering . . . 29
Figure 4.12 Isoenergetic cluster move on a 2D Ising lattice . . . 30
Figure 4.13 Path relinking between an initial permutation and a guide permutation . . . 32
Figure 4.14 Architecture of the PT-SQA hybrid . . . 34
Figure 4.15 Two-dimensional Parallel Tempering . . . 35
Figure 4.16 Systematic resampling for Population Annealing with 10 replicas . . . 36
Figure 5.1 `ASSA.jl`, the custom Julia solver framework developed for this thesis . . . 37
Figure 5.2 Class diagram of the `ASSA.jl` solver hierarchy . . . 37
Figure 5.3 Generic control flow of the `solve!` entry point shared by all `ASSA.jl` solvers. . . . 39
Figure 5.4 Specialized `step!` implementations for each solver family. . . . 39
Figure 6.1 Solution quality of SA across all three Taillard families(`taiA` , `taiB` , `taiC`). . . . 49
Figure 6.2 Convergence trajectories of SA on the three largest instances of each family. . 49
Figure 6.3 Effect of adaptive cache activation on `tai100a` over 3300 sweeps . . . 50
Figure 6.4 Convergence of SQA under a monotonic schedule(geometrically decreasing temperature, geometrically increasing coupling) on the largest `tai` instances. . 51
Figure 6.5 Convergence of SQA under the self-adaptive coupling schedule across all three Taillard families at different sweep counts. . . . 52
Figure 6.6 Detailed internal dynamics of self-adaptive SQA on `tai150b` using a single replica with 100 Trotter slices. . . . 53
Figure 6.7 Head-to-head comparison of SQA with the monotonic geometric vs self-adaptive coupling schedules at 1100 sweeps across all three Taillard families. .53
Figure 6.8 MA convergence on the three largest Taillard instances. . . . 55
Figure 6.9 Execution time per sweep versus problem size for MA and SA. . . . 56
Figure 6.10 Solution quality of PA with geometric and adaptive temperature schedules across all three Taillard families. . . . 57
Figure 6.11 Joint evolution of optimality gap and temperature for PA with geometric and adaptive schedules. . . . 58
Figure 6.12 Solution quality of PT with geometric and adaptive temperature ladders across all three Taillard families. . . . 59
Figure 6.13 Adaptive temperature ladder(bottom axis, line) and resulting per-replica swap acceptance rates(bars) for PT. . . . 59
Figure 6.14 Comparison of four PT cluster-move variants across all Taillard families . . . 60
Figure 6.15 Convergence trajectories of PT with and without cluster moves. . . . 61
Figure 6.16 Performance of PT-SQA(8 Trotter slices per replica, adaptive temperature ladder) with three penalty schedule configurations . . . 62
Figure 6.17 Median optimality gap of four external solvers . . . 68
Figure 6.18 Head-to-head comparison of the best external solver(Tabu Search) against our GPU-accelerated annealing methods(PA, PTC, PT-SQA) under an identical 150-second wall-clock limit across all Taillard families. . . . 68
Figure 6.19 Example connectome . . . 69
Figure 6.20 Benchmark results for the 1000-ROI functional connectome graph matching task. . . . 70
Figure 7.1 Heuristic starting points suggested by our experimental results. . . . 74
Figure A.1 Convergence comparison of non-cached and fully cached version. . . . 80
Figure A.2 SQA torus vs ring topology. . . . 81

Figure A.3 Results of PT2D with geometric temperatures and linear penalties. 82
Figure A.4 Adaptive PT2D for tai100b 82
Figure A.5 Runtime for sampling adaptive temperatures for PT 83

List of Tables

Table 5.1 Summary of CUDA grid and block configurations for the implemented solvers. . 40
Table 6.1 Parameters for SA. 48
Table 6.2 Parameters for SQA. 51
Table 6.3 Parameters for PA. 56
Table 6.4 Parameters for PT. 58
Table 6.5 Parameters for PTC. 60
Table 6.6 QAPLIB results for 62/150 hard instances. 64
Table A.1 Number of sweep setup for SA cached. 80
Table A.2 Parameters used to generate the adaptive temperature ladders for PT. 83

List of Algorithms

Algorithm 4.1 SA flow for the QAP(adapted from) 19
Algorithm 5.1 Cached SA inner loop for the QAP. 41
Algorithm 5.2 Cached MA local sweep with dropout 44
Algorithm A.1 Pseudocode for picking random nonzero on GPU 79

Bibliography

[1] A. Misevičius, “Letter: New best known solution for the most difficult QAP instance “tai100a,” *Memetic Computing*, vol. 11, no. 3, pp. 331–332, Sep. 2019, doi: 10.1007/s12293-019-00289-y.

[2] T. C. Koopmans and M. Beckmann, “Assignment Problems and the Location of Economic Activities,” *Econometrica*, vol. 25, no. 1, p. 53, Jan. 1957, doi: 10.2307/1907742.

[3] L. Steinberg, “The Backboard Wiring Problem: A Placement Algorithm,” *SIAM Review*, vol. 3, no. 1, pp. 37–50, Jan. 1961, doi: 10.1137/1003003.

[4] C. Bierwirth, D. C. Mattfeld, and H. Kopfer, “On permutation representations for scheduling problems,” *Parallel Problem Solving from Nature — PPSN IV*, vol. 1141. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 310–318, 1996. doi: 10.1007/3-540-61723-X_995.

[5] R. E. Burkard and J. Offermann, “Entwurf von Schreibmaschinentastaturen mittels quadratischer Zuordnungsprobleme,” *Zeitschrift für Operations Research*, vol. 21, no. 4, pp. B121–B132, Aug. 1977, doi: 10.1007/BF01918175.

[6] A. N. Elshafei, “Hospital Layout as a Quadratic Assignment Problem,” *Operational Research Quarterly (1970-1977)*, vol. 28, no. 1, p. 167, 1977, doi: 10.2307/3008789.

[7] J. Dickey and J. Hopkins, “Campus building arrangement using topaz,” *Transportation Research*, vol. 6, no. 1, pp. 59–68, Mar. 1972, doi: 10.1016/0041-1647(72)90111-6.

[8] J. Bos, “Zoning in Forest Management: a Quadratic Assignment Problem Solved by Simulated Annealing,” *Journal of Environmental Management*, vol. 37, no. 2, pp. 127–145, Feb. 1993, doi: 10.1006/jema.1993.1010.

[9] R. Abdolahi, M. R. Soleymani, and W. Hamouda, “A Genetic Algorithm with Quantum Circuit Objective Function for Optimizing Communication Network Design: A Quadratic Assignment Approach,” in *ICC 2025 - IEEE International Conference on Communications*, Jun. 2025, pp. 1–6. doi: 10.1109/ICC52391.2025.11161958.

[10] R. E. Burkard, E. Çela, P. Pardalos, and L. Pitsoulis, “The quadratic assignment problem,” in *Handbook of Combinatorial Optimization*, vol. 2, Netherlands: Kluwer Academic Publishers, 1998, pp. 241–337.

[11] T. Mzili, M. Mzili, A. Abatal, A. K. Arya, and N. Bacanin, “A spotted hyena algorithm for quadratic allocation problems: a novel bio-inspired solution,” *Annals of Operations Research*, Jul. 2025, doi: 10.1007/s10479-025-06707-4.

[12] É. D. Taillard, *Design of Heuristic Algorithms for Hard Optimization: With Python Codes for the Travelling Salesman Problem*. in Graduate Texts in Operations Research. Cham: Springer International Publishing, 2023. doi: 10.1007/978-3-031-13714-3.

[13] Z. Drezner, "A new heuristic for the quadratic assignment problem," *Journal of Applied Mathematics and Decision Sciences*, vol. 6, no. 3, pp. 143–153, Jan. 2002, doi: 10.1155/S1173912602000093.

[14] E. M. Loiola, N. M. M. De Abreu, P. O. Boaventura-Netto, P. Hahn, and T. Querido, "A survey for the quadratic assignment problem," *European Journal of Operational Research*, vol. 176, no. 2, pp. 657–690, Jan. 2007, doi: 10.1016/j.ejor.2005.09.032.

[15] S. Sahni and T. Gonzalez, "P-Complete Approximation Problems," *Journal of the ACM*, vol. 23, no. 3, pp. 555–565, Jul. 1976, doi: 10.1145/321958.321975.

[16] T. Stützle, "Iterated local search for the quadratic assignment problem," *European Journal of Operational Research*, vol. 174, no. 3, pp. 1519–1539, Nov. 2006, doi: 10.1016/j.ejor.2005.01.066.

[17] D. Harel, *Computers Ltd: What they really can't do*. Oxford University PressOxford, 2003. doi: 10.1093/oso/9780198604426.001.0001.

[18] C. Oyarzun Laura, S. Wesarg, and G. Sakas, "Graph matching survey for medical imaging: On the way to deep learning," *Methods*, vol. 202, pp. 3–13, Jun. 2022, doi: 10.1016/j.ymeth.2021.06.008.

[19] A. Saad-Eldin, B. D. Pedigo, C. E. Priebe, and J. T. Vogelstein, "Graph Matching via Optimal Transport." Accessed: Apr. 09, 2025. [Online]. Available: http://arxiv.org/abs/2111.05366

[20] J. T. Vogelstein *et al.*, "Fast Approximate Quadratic Programming for Graph Matching," *PLOS ONE*, vol. 10, no. 4, p. e121002, Apr. 2015, doi: 10.1371/journal.pone.0121002.

[21] S. Arajs and R. V. Colvin, "Ferromagnetic-Paramagnetic Transition in Iron," *Journal of Applied Physics*, vol. 35, no. 8, pp. 2424–2426, Aug. 1964, doi: 10.1063/1.1702873.

[22] K. Binder and A. P. Young, "Spin glasses: Experimental facts, theoretical concepts, and open questions," *Reviews of Modern Physics*, vol. 58, no. 4, pp. 801–976, Oct. 1986, doi: 10.1103/RevModPhys.58.801.

[23] A. Abbas *et al.*, "Challenges and Opportunities in Quantum Optimization," *Nature Reviews Physics*, vol. 6, no. 12, pp. 718–735, Oct. 2024, doi: 10.1038/s42254-024-00770-9.

[24] M. S. Benkner, V. Golyanik, C. Theobalt, and M. Moeller, "Adiabatic Quantum Graph Matching with Permutation Matrix Constraints," in *2020 International Conference on 3D Vision (3DV)*, Nov. 2020, pp. 583–592. doi: 10.1109/3DV50981.2020.00068.

[25] E. Alessandroni, S. Ramos-Calderer, I. Roth, E. Traversi, and L. Aolita, "Alleviating the quantum Big-M problem," *npj Quantum Information*, vol. 11, no. 1, p. 125, Jul. 2025, doi: 10.1038/s41534-025-01067-0.

[26] P. Codognet, “Comparing QUBO models for quantum annealing: integer encodings for permutation problems,” *International Transactions in Operational Research*, vol. 32, no. 1, pp. 18–37, Jan. 2025, doi: 10.1111/itor.13471.

[27] S. Kikuchi, K. Takahashi, and S. Tanaka, “Performance of Domain-Wall Encoding in Digital Ising Machine.” Accessed: Apr. 07, 2025. [Online]. Available: http://arxiv.org/abs/2410.11198

[28] M. Bagherbeik, P. Ashtari, K. Kanda, H. Tamura, and A. Sheikholeslami, “MOCQA: A Multi-Core Optimizer for Constrained Quadratic Assignment,” *IEEE Access*, vol. 13, pp. 156870–156890, 2025, doi: 10.1109/ACCESS.2025.3603171.

[29] N. K. Meli, V. Golyanik, M. S. Benkner, and M. Moeller, “QuCOOP: A Versatile Framework for Solving Composite and Binary-Parametrised Problems on Quantum Annealers.” Accessed: Jun. 25, 2025. [Online]. Available: http://arxiv.org/abs/2503.19718

[30] D. Abramson and M. Randall, “A simulated annealing code for general integer linear programs,” *Annals of Operations Research*, vol. 86, no. 0, pp. 3–21, Jan. 1999, doi: 10.1023/A:1018915104438.

[31] R. E. Burkard, E. Çela, P. M. Pardalos, and L. S. Pitsoulis, “The Quadratic Assignment Problem,” *Handbook of Combinatorial Optimization*. Springer US, Boston, MA, pp. 1713–1809, 1998. doi: 10.1007/978-1-4613-0303-9_27.

[32] R. E. Burkard and F. Rendl, “A thermodynamically motivated simulation procedure for combinatorial optimization problems,” *European Journal of Operational Research*, vol. 17, no. 2, pp. 169–174, Aug. 1984, doi: 10.1016/0377-2217(84)90231-5.

[33] A. Franzin and T. Stützle, “Revisiting simulated annealing: A component-based analysis,” *Computers & Operations Research*, vol. 104, pp. 191–206, Apr. 2019, doi: 10.1016/j.cor.2018.12.015.

[34] F. Glover and M. Laguna, “Tabu Search,” *Handbook of Combinatorial Optimization*. Springer US, Boston, MA, pp. 2093–2229, 1998. doi: 10.1007/978-1-4613-0303-9_33.

[35] E. Taillard, “Robust taboo search for the quadratic assignment problem,” *Parallel Computing*, vol. 17, no. 4, pp. 443–455, Jul. 1991, doi: 10.1016/S0167-8191(05)80147-4.

[36] M. Saifullah Hussin and T. Stützle, “Tabu search vs. simulated annealing as a function of the size of quadratic assignment problem instances,” *Computers & Operations Research*, vol. 43, pp. 286–291, Mar. 2014, doi: 10.1016/j.cor.2013.10.007.

[37] U. Benlic and J.-K. Hao, “Memetic search for the quadratic assignment problem,” *Expert Systems with Applications*, vol. 42, no. 1, pp. 584–595, Jan. 2015, doi: 10.1016/j.eswa.2014.08.011.

[38] T. Achary, S. Pillay, S. M. Pillai, M. Mqadi, E. Genders, and A. E. Ezugwu, “A performance study of meta-heuristic approaches for quadratic assignment problem,” *Concurrency and Computation: Practice and Experience*, vol. 33, no. 17, p. e6321, Sep. 2021, doi: 10.1002/cpe.6321.

[39] A. Misevičius, A. Andrejevas, A. Ostreika, D. Verenė, and G. Žekienė, “An Improved Hybrid Genetic-Hierarchical Algorithm for the Quadratic Assignment Problem,” *Mathematics*, vol. 12, no. 23, p. 3726, Nov. 2024, doi: 10.3390/math12233726.

[40] W. Chmiel and J. Kwiecień, “Quantum-Inspired Evolutionary Approach for the Quadratic Assignment Problem,” *Entropy*, vol. 20, no. 10, p. 781, Oct. 2018, doi: 10.3390/e20100781.

[41] H. Kang, Y. Lee, and H. Yoe, “A Design of a GPU Based Solver for Quadratic Assignment Problems,” in *2024 15th International Conference on Information and Communication Technology Convergence (ICTC)*, Oct. 2024, pp. 635–640. doi: 10.1109/ICTC62082.2024.10827321.

[42] S. Iida and R. Yasudo, “Solving the QAP by Two-Stage Graph Pointer Networks and Reinforcement Learning.” Accessed: Jun. 16, 2025. [Online]. Available: http://arxiv.org/abs/2404.00539

[43] N. Ito, S. Kim, M. Kojima, A. Takeda, and K.-C. Toh, “BBCPOP: A Sparse Doubly Nonnegative Relaxation of Polynomial Optimization Problems with Binary, Box and Complementarity Constraints.” Accessed: Jun. 08, 2026. [Online]. Available: http://arxiv.org/abs/1804.00761

[44] S. Kim, M. Kojima, and K.-C. Toh, “A Newton-bracketing method for a simple conic optimization problem,” *Optimization Methods and Software*, vol. 36, no. 2, pp. 371–388, May 2021, doi: 10.1080/10556788.2020.1782906.

[45] P. Virtanen *et al.*, “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020, doi: 10.1038/s41592-019-0686-2.

[46] M. S. Benkner, M. Krahn, E. Tretschk, Z. Lähner, M. Moeller, and V. Golyanik, “QuAnt: Quantum Annealing with Learnt Couplings.” Accessed: Mar. 31, 2025. [Online]. Available: http://arxiv.org/abs/2210.08114

[47] D. Laplace Mermoud, A. Simonetto, and S. Elloumi, “Variational quantum algorithms for permutation-based combinatorial problems: Optimal ansatz generation with applications to quadratic assignment problems and beyond,” *Quantum*, vol. 10, p. 1998, Feb. 2026, doi: 10.22331/q-2026-02-09-1998.

[48] N. Daly, T. Krauss, and J. Shapiro, “Quantum Approaches to the Quadratic Assignment Problem.” Accessed: Jun. 28, 2026. [Online]. Available: http://arxiv.org/abs/2505.00182

[49] S. Chowdhury *et al.*, “Pushing the Boundary of Quantum Advantage in Hard Combinatorial Optimization with Probabilistic Computers,” *Nature Communications*, vol. 16, no. 1, p. 9193, Oct. 2025, doi: 10.1038/s41467-025-64235-y.

[50] S. Bunaiyan, C. Delacour, S. Chowdhury, K. Lee, and K. Y. Camsari, “IsingFormer: Augmenting Parallel Tempering With Learned Proposals.” Accessed: Jan. 07, 2026. [Online]. Available: http://arxiv.org/abs/2509.23043

[51] R. Martoňák, G. E. Santoro, and E. Tosatti, “Quantum annealing of the traveling-salesman problem,” *Physical Review E*, vol. 70, no. 5, p. 57701, Nov. 2004, doi: 10.1103/PhysRevE.70.057701.

[52] R. Burkard, M. Dell'Amico, and S. Martello, *Assignment Problems: Revised Reprint*. Society for Industrial, Applied Mathematics, 2012. doi: 10.1137/1.9781611972238.

[53] R. E. Burkard, "Quadratic assignment problems," *European Journal of Operational Research*, vol. 15, no. 3, pp. 283–289, Mar. 1984, doi: 10.1016/0377-2217(84)90093-6.

[54] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983, doi: 10.1126/science.220.4598.671.

[55] D. T. Connolly, "An improved annealing scheme for the QAP," *European Journal of Operational Research*, vol. 46, no. 1, pp. 93–100, May 1990, doi: 10.1016/0377-2217(90)90301-Q.

[56] A. Misevičius, "A Modified Simulated Annealing Algorithm for the Quadratic Assignment Problem," *Informatica*, vol. 14, no. 4, pp. 497–514, Jan. 2003, Accessed: Mar. 05, 2025. [Online]. Available: https://content.iospress.com/articles/informatica/inf14-4-07

[57] M. J. Kochenderfer and T. A. Wheeler, *Algorithms for optimization*. Cambridge: The MIT press, 2019.

[58] G. Paul, "An efficient implementation of the simulated annealing heuristic for the quadratic assignment problem." Accessed: Mar. 12, 2025. [Online]. Available: http://arxiv.org/abs/1111.1353

[59] M. Bagherbeik, P. Ashtari, S. F. Mousavi, K. Kanda, H. Tamura, and A. Sheikholeslami, "A Permutational Boltzmann Machine with Parallel Tempering for Solving Combinatorial Optimization Problems," *Parallel Problem Solving from Nature – PPSN XVI*, vol. 12269. Springer International Publishing, Cham, pp. 317–331, 2020. doi: 10.1007/978-3-030-58112-1_22.

[60] M. Aramon, G. Rosenberg, E. Valiante, T. Miyazawa, H. Tamura, and H. G. Katzgraber, "Physics-Inspired Optimization for Quadratic Unconstrained Problems Using a Digital Annealer," *Frontiers in Physics*, vol. 7, p. 48, Apr. 2019, doi: 10.3389/fphy.2019.00048.

[61] R. Martí, M. Sevaux, and K. Sörensen, "Fifty years of metaheuristics," *European Journal of Operational Research*, vol. 321, no. 2, pp. 345–362, Mar. 2025, doi: 10.1016/j.ejor.2024.04.004.

[62] P. Atchade-Adelomou, "Quantum Algorithms for solving Hard Constrained Optimisation Problems." Accessed: Dec. 17, 2025. [Online]. Available: https://arxiv.org/abs/2202.13125

[63] M. Suzuki, "Generalized Trotter's formula and systematic approximants of exponential operators and inner derivations with applications to many-body problems," *Communications in Mathematical Physics*, vol. 51, no. 2, pp. 183–190, Jun. 1976, doi: 10.1007/BF01609348.

[64] V. S. Denchev *et al.*, "What is the Computational Value of Finite Range Tunneling?," *Physical Review X*, vol. 6, no. 3, p. 31015, Aug. 2016, doi: 10.1103/PhysRevX.6.031015.

[65] B. Heim, T. F. Rønnow, S. V. Isakov, and M. Troyer, “Quantum versus classical annealing of Ising spin glasses,” *Science*, vol. 348, no. 6231, pp. 215–217, Apr. 2015, doi: 10.1126/science.aaa4170.

[66] R. Martoňák, G. E. Santoro, and E. Tosatti, “Quantum annealing by the path-integral Monte Carlo method: The two-dimensional random Ising model,” *Phys. Rev. B*, vol. 66, no. 9, p. 94203, Sep. 2002, doi: 10.1103/PhysRevB.66.094203.

[67] T. Okuyama, T. Sonobe, K.-i. Kawarabayashi, and M. Yamaoka, “Binary optimization by momentum annealing,” vol. 100, no. 1, p. 12111, Jul. 2019, doi: 10.1103/PhysRevE.100.012111.

[68] W. Yang, Y. Wang, A. Ćustić, and A. P. Punnen, “Experimental analysis of algorithms for the independent quadratic assignment problem,” *Computers & Operations Research*, vol. 168, p. 106704, Aug. 2024, doi: 10.1016/j.cor.2024.106704.

[69] K. Hukushima and K. Nemoto, “Exchange Monte Carlo Method and Application to Spin Glass Simulations,” *Journal of the Physical Society of Japan*, vol. 65, no. 6, pp. 1604–1608, Jun. 1996, doi: 10.1143/JPSJ.65.1604.

[70] B. Verity and P. Winskill, “drjacoby: Flexible Markov Chain Monte Carlo via Reparameterization.” 2024.

[71] N. Rathore, M. Chopra, and J. J. De Pablo, “Optimal allocation of replicas in parallel tempering simulations,” *The Journal of Chemical Physics*, vol. 122, no. 2, p. 24111, Jan. 2005, doi: 10.1063/1.1831273.

[72] Z. Zhu, C. Fang, and H. G. Katzgraber, “borealis - A generalized global update algorithm for Boolean optimization problems.” Accessed: Sep. 29, 2025. [Online]. Available: https://arxiv.org/abs/1605.09399

[73] Z. Zhu, A. J. Ochoa, and H. G. Katzgraber, “Efficient Cluster Algorithm for Spin Glasses in Any Space Dimension,” *Physical Review Letters*, vol. 115, no. 7, p. 77201, Aug. 2015, doi: 10.1103/PhysRevLett.115.077201.

[74] J. King *et al.*, “Quantum-Assisted Genetic Algorithm.” Accessed: Dec. 29, 2025. [Online]. Available: http://arxiv.org/abs/1907.00707

[75] J. Houdayer, “A Cluster Monte Carlo Algorithm for 2-Dimensional Spin Glasses,” *The European Physical Journal B*, vol. 22, no. 4, pp. 479–484, Aug. 2001, doi: 10.1007/PL00011151.

[76] V. A. Cicirello, “Cycle Mutation: Evolving Permutations via Cycle Induction,” *Applied Sciences*, vol. 12, no. 11, p. 5506, May 2022, doi: 10.3390/app12115506.

[77] M. Laguna and R. Marti, “GRASP and Path Relinking for 2-Layer Straight Line Crossing Minimization,” *INFORMS Journal on Computing*, vol. 11, no. 1, pp. 44–52, Feb. 1999, doi: 10.1287/ijoc.11.1.44.

[78] C. A. S. Oliveira, P. M. Pardalos, and M. G. C. Resende, “GRASP with Path-Relinking for the Quadratic Assignment Problem,” *Experimental and Efficient Algorithms*, vol. 3059. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 356–368, 2004. doi: 10.1007/978-3-540-24838-5_27.

[79] P. Ashtari, “Two-dimensional parallel tempering for solving constraint-satisfaction problems,” Master's Thesis, University of Toronto, 2021.

[80] C. Delacour, “Self-Adaptive Ising Machines for Constrained Optimization,” in *2025 Design, Automation & Test in Europe Conference (DATE)*, Lyon, France: IEEE, Mar. 2025, pp. 1–7. doi: 10.23919/DATE64628.2025.10992982.

[81] M. M. H. Sajeeb, C. Delacour, K. Callahan-Coray, S. Seshan, T. Srimani, and K. Y. Camsari, “Probabilistic Computers for MIMO Detection: From Sparsification to 2D Parallel Tempering.” Accessed: Mar. 19, 2026. [Online]. Available: http://arxiv.org/abs/2601.09037

[82] K. Hukushima, “Population Annealing and Its Application to a Spin Glass,” in *AIP Conference Proceedings*, Los Alamos, New Mexico (USA): AIP, 2003, pp. 200–206. doi: 10.1063/1.1632130.

[83] L. Y. Barash, M. Weigel, M. Borovský, W. Janke, and L. N. Shchur, “GPU accelerated population annealing algorithm,” *Computer Physics Communications*, vol. 220, pp. 341–350, Nov. 2017, doi: 10.1016/j.cpc.2017.06.020.

[84] D. Gessert, W. Janke, and M. Weigel, “Resampling schemes in population annealing: Numerical and theoretical results,” 2023, doi: 10.48550/ARXIV.2305.19994.

[85] J. Bezanson, A. Edelman, S. Karpinski, and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017, doi: 10.1137/141000671.

[86] T. Besard, C. Foket, and B. De Sutter, “Effective Extensible Programming: Unleashing Julia on GPUs,” *IEEE Transactions on Parallel and Distributed Systems*, 2018, doi: 10.1109/TPDS.2018.2872064.

[87] T. Besard, V. Churavy, A. Edelman, and B. De Sutter, “Rapid software prototyping for heterogeneous and distributed platforms,” *Advances in Engineering Software*, vol. 132, pp. 29–46, 2019.

[88] V. Churavy, “KernelAbstractions.jl.” [Online]. Available: https://github.com/JuliaGPU/KernelAbstractions.jl

[89] M. Harris, S. Sengupta, and J. D. Owens, “Parallel prefix sum (scan) with CUDA,” *GPU gems*, vol. 3, no. 39, pp. 851–876, 2007.

[90] R. Burkard, E. Çela, S. Karisch, F. Rendl, M. Anjos, and P. Hahn, “QAPLIB - A Quadratic Assignment Problem Library - Problem instances and solutions.” Accessed: Apr. 01, 2026. [Online]. Available: https://datashare.ed.ac.uk/handle/10283/4390

[91] K. Fujii, S. Kim, M. Kojima, H. D. Mittelmann, and Y. Shinano, “An exceptionally difficult binary quadratic optimization problem with symmetry: a challenge for the largest unsolved QAP instance Tai256c,” *Optimization Letters*, Nov. 2024, doi: 10.1007/s11590-024-02157-2.

[92] S. Verel, S. Thomson, and O. Rifki, “Where the Really Hard Quadratic Assignment Problems Are: the QAP-SAT instances,” 2024, doi: 10.48550/ARXIV.2403.02783.

[93] Z. Drezner, P. M. Hahn, and É. D. Taillard, “Recent Advances for the Quadratic Assignment Problem with Special Emphasis on Instances that are Difficult for Meta-Heuristic Methods,” *Annals of Operations Research*, vol. 139, no. 1, pp. 65–94, Oct. 2005, doi: 10.1007/s10479-005-3444-z.

[94] D-Wave, “D-WAVE PERFORMANCE STUDY: FACILITY PLACEMENT.” Accessed: Mar. 17, 2026. [Online]. Available: https://www.dwavequantum.com/resources/white-paper/d-wave-performance-study-facility-placement/

[95] H. Richardson, G. Lisandrelli, A. Riobueno-Naylor, and R. Saxe, “Development of the social brain from age three to twelve years,” *Nature Communications*, vol. 9, no. 1, p. 1027, Mar. 2018, doi: 10.1038/s41467-018-03399-2.

[96] H. Richardson, G. Lisandrelli, A. Riobueno-Naylor, and R. Saxe, “MRI data of 3-12 year old children and adults during viewing of a short animated film.” Accessed: Jun. 25, 2025. [Online]. Available: https://openneuro.org/datasets/ds000228/versions/1.0.0

[97] H. Bukhari, C. Su, E. Dhamala, Z. Gu, K. Jamison, and A. Kuceyeski, “Graph-matching distance between individuals' functional connectomes varies with relatedness, age, and cognitive score,” *Human Brain Mapping*, vol. 44, no. 9, pp. 3541–3554, 2023, doi: 10.1002/hbm.26296.

[98] A. Schaefer *et al.*, “Local-Global Parcellation of the Human Cerebral Cortex from Intrinsic Functional Connectivity MRI,” *Cerebral Cortex*, vol. 28, no. 9, pp. 3095–3114, Sep. 2018, doi: 10.1093/cercor/bhx179.

[99] N. contributors, “nilearn.” [Online]. Available: https://github.com/nilearn/nilearn

[100] H. G. Katzgraber, F. Hamze, Z. Zhu, A. J. Ochoa, and H. Munoz-Bauza, “Seeking Quantum Speedup Through Spin Glasses: The Good, the Bad, and the Ugly,” *Physical Review X*, vol. 5, no. 3, Sep. 2015, doi: 10.1103/physrevx.5.031026.

[101] C. A. Gandarilla-Perez, S. Pinilla, A.-F. Bitbol, and M. Weigt, “Combining phylogeny and coevolution improves the inference of interaction partners among paralogous proteins,” *PLOS Computational Biology*, vol. 19, no. 3, p. e1011010, Mar. 2023, doi: 10.1371/journal.pcbi.1011010.

[102] D. E. Fishkind *et al.*, “Seeded Graph Matching.” Accessed: Apr. 09, 2025. [Online]. Available: http://arxiv.org/abs/1209.0367

[103] C. Delacour, M. M. H. Sajeeb, J. P. Hespanha, and K. Y. Camsari, “Two-dimensional Parallel Tempering for Constrained Optimization,” *Physical Review E*, vol. 112, no. 2, p. L23301, Aug. 2025, doi: 10.1103/mr2n-qqrb.